



CS4001NI Programming

30% Individual Coursework

2022-23 Autumn

Student Name: Nirajan Shah

London Met ID: 22067846

College ID: NP01CP4A220232

Group: C10

Assignment Due Date: Wednesday, May 10, 2023

Assignment Submission Date: Wednesday, May 10, 2023

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Abstract

When the three classes were initialized at the very start, we did not have to make a GUI as there already was a form of it provided by the BlueJ IDE. However, the GUI provided was only for learning and testing purposes. A functioning GUI was developed using swing library of Java to initialize the three classes developed at the very start and to actually develop a banking application. Through an object i.e., the main GUI itself, instantiation of the objects of the other two classes was made possible. Important concepts such as iteration, event handling and exception handling were learned throughout the construction of this project. Unlike before, a main method was used to call an anonymous object of the BankGUI class.

This project aims to simulate an actual banking application used in real world applications. Although, the GUI made could be described to be as crude and simple, there is no denying that this is the first actual software with an action GUI that was developed by us students using Java.

Table of Contents

Introduction	1
1.1 BlueJ	1
1.2 Draw.io	2
1.3 Java	2
Class Diagram.....	3
2.1 BankCard.....	3
2.2 DebitCard	4
2.3 CreditCard	5
2.4 BankGUI	6
2.5 Relationship diagram.....	7
Pseudocode	8
3.1 BankGUI	8
Method descriptions	43
4.1 BankGUI	44
4.1.1 Buttons	44
4.1.2 KeyListener text fields	47
Evidence	48
5.1 Test 1	49
5.2 Test 2	53
5.3 Test 3	71
Error detection and correction	86
6.1 Syntax error	86
6.2 Semantic error	88
6.3 Logical error.....	89
Conclusion	91
7.1 Evaluation.....	91
7.2 Reflection on acquired knowledge	91
7.3 Difficulties encountered	92
7.4 How I overcame the said difficulties	93
References.....	94
Appendix	A
Appendix A: Code.....	A
Appendix A.1: BankGUI.....	A

Table of figures

Figure 1: Class diagram of BankCard	3
Figure 2: Class diagram of DebitCard	4
Figure 3: Class diagram of CreditCard	5
Figure 4: Class diagram of BankGUI.....	6
Figure 5: Figure showing the inheritance relationship between the four classes.....	7
Figure 6: Screenshot of compiling the BankGUI class	50
Figure 7: Screenshot of running the BankGUI class	52
Figure 8: Screenshot of running the BankGUI class	52
Figure 9: Screenshot displaying the entered text field values	54
Figure 10: Screenshot after clicking on the add button	54
Figure 11: Screenshot displaying the entered text field values	56
Figure 12: Screenshot after clicking on the add button	56
Figure 13: Screenshot displaying the entered text field values	58
Figure 14: Screenshot after clicking on add button	58
Figure 15: Screenshot after clicking on Withdraw button	59
Figure 17: Screenshot displaying the entered text field values	61
Figure 18: Screenshot after clicking on add button	61
Figure 19: Screenshot after clearing out the PIN number text field value.....	62
Figure 20: Screenshot after clearing out the Card Id text field value	62
Figure 21: Screenshot after re-entering the value of card id	63
Figure 22: Screenshot after re-entering the value of pin number	63
Figure 23: Screenshot after entering a value for withdrawal amount and pressing enter	64
Figure 24: Screenshot after clicking on OK on the dialog box.....	64
Figure 26: Screenshot displaying the entered text field values	66
Figure 27: Screenshot after clicking on add button	66
Figure 28: Screenshot after entering the values for credit limit and grace period.....	67
Figure 29: Screenshot after clicking on Set Limit button	67
Figure 31: Screenshot displaying the entered text field values	69
Figure 32: Screenshot after clicking on the add button	69
Figure 33: Screenshot after clicking on the Set Limit button after entering values for credit limit and grace period	70
Figure 34: Screenshot after clicking on the Cancel button	70
Figure 36: Screenshot displaying the entered text field values	72
Figure 37: Screenshot after clicking on the add button	72
Figure 38: Screenshot after clicking on add button	73
Figure 40: Screenshot displaying the entered text field values	75
Figure 41: Screenshot after clicking on the add button	75
Figure 43: Screenshot displaying the entered text field values	77
Figure 44: Screenshot after clicking on the add button	77
Figure 46: Screenshot displaying the entered text field values.	79
Figure 47: Screenshot after clicking on the Withdraw button	79
Figure 49: Screenshot displaying the entered text field values	81
Figure 50: Screenshot after clicking on the add button	81
Figure 51: Screenshot after clicking on the withdraw button	82

Figure 53: Screenshot displaying the entered text field values	84
Figure 54: Screenshot after clicking on the add button	84
Figure 55: Screenshot after clicking on the Set Limit button	85
Figure 56: Screenshot of the syntax error	87
Figure 57: Screenshot of the fix	87
Figure 58: Screenshot of the semantic error	88
Figure 59: Screenshot of the fix	89
Figure 60: Screenshot of the logical error	90
Figure 61: Screenshot of the fix	90

List of Tables

Table 1: To compile the BankGUI class	50
Table 2: To run the BankGUI class	51
Table 3: To check if an object of DebitCard was added to the array list after clicking on add button	53
Table 4: To check if credit card is added to the array list after clicking on add button	55
Table 5: To check if a certain amount would be withdrawn from an instance of debit card from the GUI.....	57
Table 6: To check if dialog messages are displayed after entering valid debit card attribute values	60
Table 7: To check if a certain amount of credit limit would be set for an instance of credit card from the GUI	65
Table 8: To check if credit limit would be cancelled for an instance of credit card from the GUI.....	68
Table 9: To check if a dialog box informing about a duplicate would appear upon adding a credit card entry twice.....	71
Table 10: To check if a dialog box informing that an invalid value for card Id was entered.	74
Table 11: To check if a dialog box informing that an empty value for card Id was entered would appear.....	76
Table 12: To check if a dialog box would appear informing about empty database when withdraw button is clicked before adding any entries into the array list.....	78
Table 13: To check if a dialog box informing that withdrawal was not successful would appear.....	80
Table 14: To check if a dialog box would appear when an invalid amount of credit limit is attempted to be set for a given card id	83
Table 15: Syntax error.....	86
Table 16: Semantic error.....	88
Table 17: Logical error	90

Introduction

Swing is a library of java that is imported and used to develop graphical user interfaces for any written program (Dale, et al., 2003). Application developed using swing framework can work across various platforms. Swing contains additional components that do not use the system resources heavily. In this project, a banking application was developed to simulate a real world banking system. The project required the importation of many classes from many packages to build the GUI.

Exceptions were handled when the user entered string values for where numeric values were asked using try and catch blocks. Action listeners were added to each of the buttons so that they could be mapped to different workings later in the action performed method. We learned how a datatype i.e., an object itself was stored in another datatype i.e., an ArrayList. The question regarding why there were so many getter and setter methods defined was finally answered these methods were used to get and set the value of the instance variables on each iteration and on each button click.

1.1 BlueJ

BlueJ is a Java IDE which was built specially to introduce the Java programming language to beginner learners. The learner or the user is able to directly interact with classes and objects in Unified Modelling Language (UML) which displays the relationship between different classes and hence the structure of the program (Kölling, et al., 2003). The constructor of any classes can be executed by right clicking the rectangular icons denoting the classes and selecting new ClassName(datatype variable, ...).

1.2 Draw.io

Draw.io is a free and open-source software used to build diagrams, flowcharts, charts, wireframes, class diagrams and many more type of diagrams (JGraph, 2021). This online software allowed for the creation of the class diagram of all three individual classes and a class diagram showing the relation between each.

1.3 Java

Java is an all-purpose object-oriented programming language developed by Sun Microsystems in the middle of the 1990's. Its write once and run anywhere credo can be linked to its purpose of creation which was to develop embedded software for consumer household devices such as toaster and microwave oven (O'Regan, 2018). According to a recent Stack Overflow survey which was taken by almost 90000 software developers from more than 170 countries, Java stands at the 5th position as one of the most popular programming languages (Stack Overflow, 2019). The project was built using Java software development kit.

Class Diagram

Unified model language (UML), a standard used to dictate how object-oriented system is represented in graphical format, is a widely adapted standard (Jacobson, et al., 1999) developed by Object Management Group which is a non-profit technology standards consortium (OMG, 2017). Class diagram is an architectural representation of object-oriented system which shows the individual classes and the relationships between them. A class diagram itself comprises of details such as class name, attribute, methods etc (Kulkarni & Prasad, 2021). The class diagram which is a part of UML has a close resemblance to ERD's used in database design (Rumbaugh, 2006).

2.1 BankCard

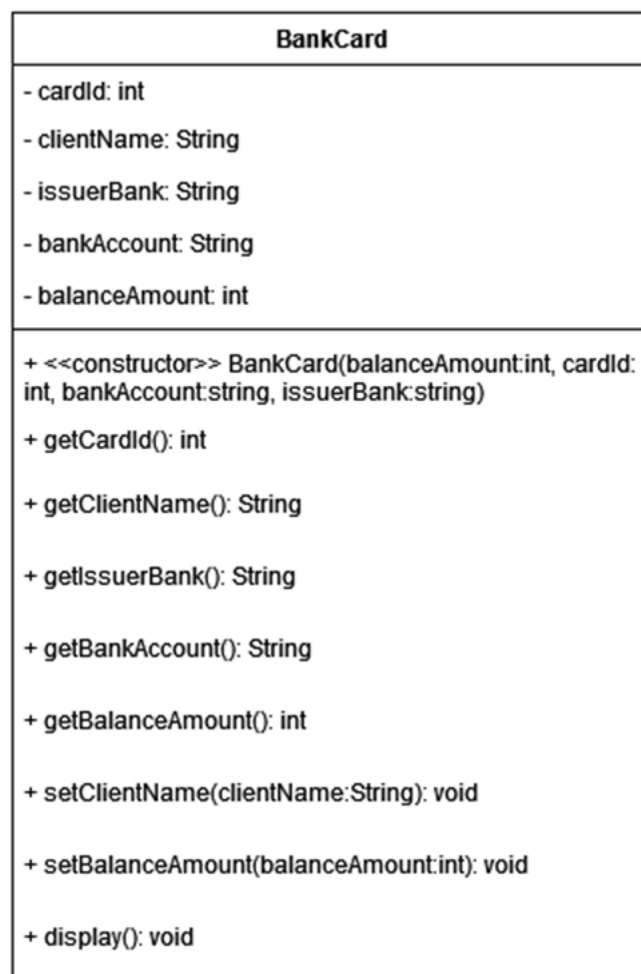


Figure 1: Class diagram of BankCard

2.2 DebitCard



Figure 2: Class diagram of DebitCard

2.3 CreditCard

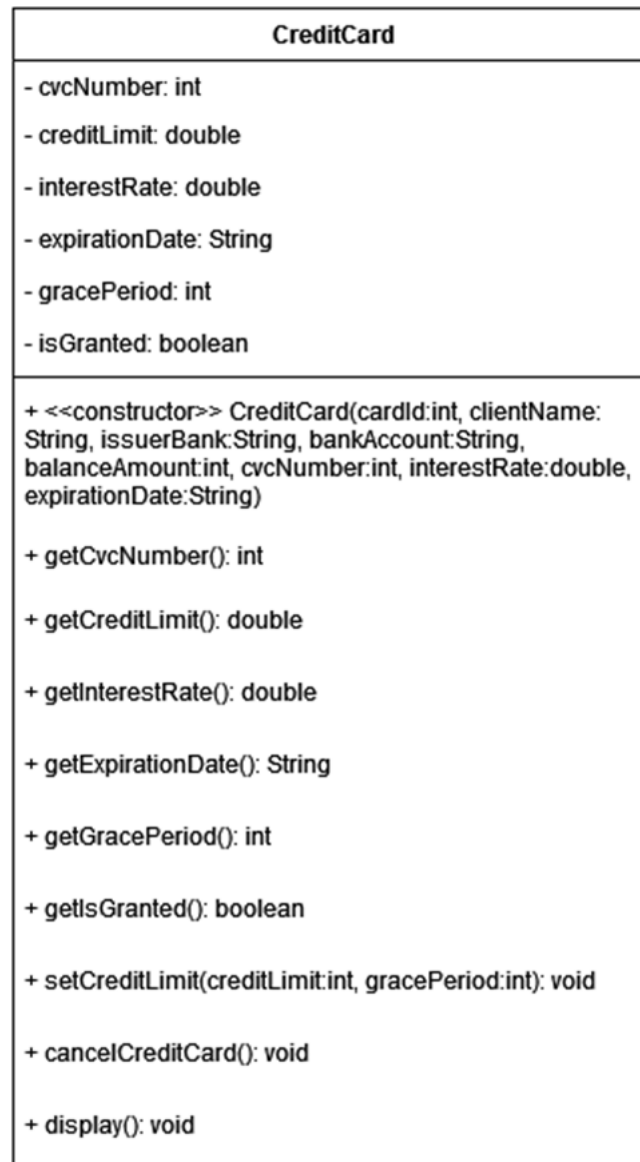


Figure 3: Class diagram of CreditCard

2.4 BankGUI



Figure 4: Class diagram of BankGUI

2.5 Relationship diagram

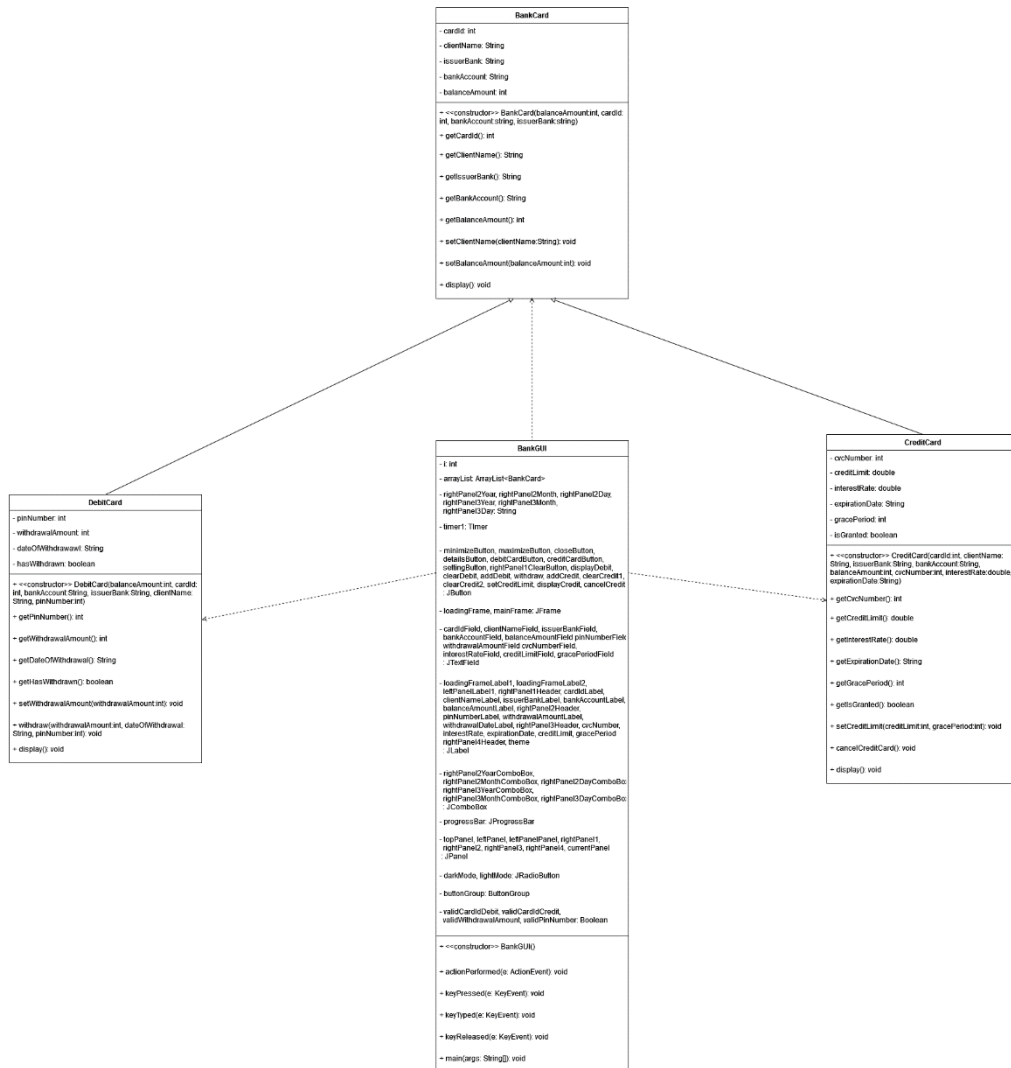


Figure 5: Figure showing the inheritance relationship between the four classes

Pseudocode

Pseudocode is the natural language (English) equivalent of the source code which explains the workings of each line of statement or of the program as a whole in a detailed and concise manner (Oda, et al., 2015). It simplifies the source code so that those who are new to programming are able to understand the source code or the working mechanism of the program without much difficulty.

3.1 BankGUI

IMPORT all components of swing library.

IMPORT the Font class from awt package.

IMPORT the Color class from awt package.

IMPORT all event classes from awt package.

IMPORT the FlowLayout class from awt package.

IMPORT the Image class from awt package.

IMPORT the Dimension class from awt package.

IMPORT the GridLayout class from awt package.

IMPORT the ArrayList class from util package.

IMPORT the Iterator class from the util package.

CREATE a class BankGUI that implements Action Listener and Key Listener interface.

DO

DECLARE instance variable i;

DECLARE instance variable arrayList as ArrayList of BankCard type using private access modifier.

DECLARE instance variables rightPanel2Year, rightPanel2Month, rightPanel2Day, rightPanel3Year, rightPanel3Month, rightPanel3Day as String using private access modifier.

DECLARE instance variable timer1 as Timer using private access modifier.

DECLARE instance variables minimizeButton, maximizeButton, closeButton, detailsButton, debitCardButton, creditCardButton, settingButton, rightPanel1ClearButton as JButton using private access modifier.

DECLARE instance variables displayDebit, clearDebit, addDebit, withdraw as JButton using private access modifier.

DECLARE instance variables addCredit, clearCredit1, clearCredit2, setCreditLimit, displayCredit, cancelCredit as JButton using private access modifier.

DECLARE instance variables loadingFrame, mainframe as JFrame using private access modifier.

DECLARE instance variables loadingFrameLabel1, loadingFrameLabel2, leftPanelLabel1, rightPanel1Header as JLabel using private access modifier.

DECLARE instance variables cardIdLabel, clientNameLabel, issuerBankLabel, bankAccountLabel, balanceAmountLabel JLabel using private access modifier.

DECLARE instance variables cardIdField, clientNameField, issuerBankField, bankAccountField, balanceAmountField as JTextField using private access modifier.

DECLARE instance variables rightPanel2Header, pinNumberLabel, withdrawalAmountLabel, withdrawalDateLabel as JLabel using private access modifier.

DECLARE instance variables pinNumberField, withdrawalAmountField as JTextField using private access modifier.

DECLARE instance variables rightPanel3Header, cvcNumber, interestRate, expirationDate, creditLimit, gracePeriod as JLabel using private access modifier.

DECLARE instance variables cvcNumberField, interestRateField, creditLimitField, gracePeriodField as JTextField using private access modifier.

DECLARE instance variables rightPanel4Header, theme as JLabel using private access modifier.

DECLARE instance variables rightPanel2YearComboBox, rightPanel2MonthComboBox, rightPanel2DayComboBox as JComboBox using private access modifier.

DECLARE instance variables rightPanel3YearComboBox, rightPanel3MonthComboBox, rightPanel3DayComboBox as JComboBox using private access modifier.

DECLARE instance variable progressBar as JProgressBar using private access modifier.

DECLARE instance variables topPanel, leftPanel, leftPanelPanel, rightPanel1, rightPanel2, rightPanel3, rightPanel4, currentPanel as JPanel using private access modifier.

DECLARE instance variables darkMode, lightMode as JRadioButton using private access modifier.

DECLARE instance variable buttonGroup as ButtonGroup using private access modifier.

DECLARE instance variable validCardIdDebit as boolean using private access modifier and **INITIALIZE** it to false.

DECLARE instance variable validCardIdCredit as boolean using private access modifier and **INITIALIZE** it to false.

DECLARE instance variable validWithdrawalAmount as boolean using private access modifier and **INITIALIZE** it to false.

DECLARE instance variable validPinNumber as boolean using private access modifier and **INITIALIZE** it to false.

CREATE a constructor of class BankCard which accepts four parameters which are balanceAmount, cardId, bankAccount and issuerBank

DO

INITIALIZE loadingFrameLabel2

DECLARE and **INITIALIZE** image as ImageIcon to get background image.

DECLARE and **INITIALIZE** image1 as Image to scale the background image.

INITIALIZE image back to ImageIcon

SET the icon of the loadingFrameLabel2

SET the bounds of loadingFrameLabel2

INITIALIZE loadingFrame

INITIALIZE mainFrame

INITIALIZE progressBar

SET foreground color of progress bar.

SET bounds of progress bar.

SET the visibility of the percentage value to true.

INITIALIZE loadingFrameLabel1

SET bounds of loadingFrameLabel1.

SET font of loadingFrameLabel1.

SET foreground color of loadingFrameLabel1.

INITIALIZE timer1

START timer1.

ADD progressBar to loadingFrame

ADD loadingFrameLabel1 to loadingFrame

ADD loadingFrameLabel2 to loadingFrame

SET resizability of loadingFrame to false.
SET background color of loadingFrame.
REMOVE the encapsulating bars of loadingFrame.
SET the layout of loadingFrame to null.
SET the size of the loadingFrame.
SET the visibility of the loadingFrame.
SET the location of the loadingFrame to the centre of the screen when it first appears.

DECLARE and **INITIALIZE** yearArray
DECLARE and **INITIALIZE** monthArray
DECLARE and **INITIALIZE** dayArray

FOR twelve times
 INITIALIZE monthArray till eleventh index.
END FOR

FOR thirty one times
 INITIALIZE monthArray till thirtieth index.
END FOR

INITIALIZE topPanel
SET layout of top panel to FlowLayout
DECLARE and **INITIALIZE** minimizeImage.
DECLARE and **INITIALIZE** scaledMinimizeImage
REASSIGN minimizeImage

SET bounds of topPanel
SET background color of topPanel
ADD topPanel to mainFrame

INITIALIZE minimizeButton

SET attributes of minimizeButton which are background, borderPainted, focusable, icon and preferredSize to appropriate values.

ADD minimizeButton to topPanel

ADD ActionListener to minimizeButton

DECLARE and **INITIALIZE** maximizeImage.

DECLARE and **INITIALIZE** scaledMaximizeImage

REASSIGN maximizeImage

INITIALIZE maximizeButton

SET attributes of maximizeButton which are background, borderPainted, icon and preferredSize to appropriate values.

ADD ActionListener

ADD minimizeButton to topPanel

DECLARE and **INITIALIZE** closeImage

DECLARE and **INITIALIZE** scaledCloseImage

REASSIGN closeImage

INITIALIZE closeButton

SET attributes of closeButton which are background, borderPainted, focusable, icon and preferredSize to appropriate values.

ADD minimizeButton to closeButton

ADD ActionListener to closeButton

INITIALIZE leftPanel

SET attributes of leftPanel which are layout, bounds and background to appropriate values

ADD leftPanel to mainframe

INITIALIZE leftPanelLabel1

SET attributes of leftPanelLabel1 which are bounds, font and foreground to appropriate values.

ADD leftPanelLabel1 to leftPanel

INITIALIZE leftPanelPanel

SET layout of leftPanelPanel to GridLayout

ADD leftPanelLabel1 to leftPanelPanel

INITIALIZE detailsButton

SET attributes of detailsButton which are foreground, background, font, focusable and borderPainted to appropriate values.

ADD ActionListener to detailsButton

ADD detailsButton to leftPanelPanel

INITIALIZE debitCardButton

SET attributes of debitCardButton which are foreground, background, font, focusable and borderPainted to appropriate values.

ADD ActionListener to debitCardButton

ADD debitCardButton to leftPanelPanel

INITIALIZE creditCardButton

SET attributes of creditCardButton which are foreground, background, font, focusable and borderPainted to appropriate values

ADD ActionListener to creditCardButton

ADD creditCardButton to leftPanelPanel

INITIALIZE settingButton

SET attributes of settingButton which are foreground, background, font, focusable and borderPainted to appropriate values

ADD ActionListener to settingButton

ADD settingButton to leftPanelPanel

ADD leftPanelPanel to leftPanel

SET bounds of leftPanelPanel

INITIALIZE rightPanel1

SET attributes of rightPanel1 which are layout, bounds, background and visible to appropriate values.

ADD rightPanel1 to mainFrame

INITIALIZE rightPanel1Header

SET attributes of rightPanel1Header which are foreground, font and bounds to appropriate values.

ADD rightPanel1Header to rightPanel1

INITIALIZE clientNameLabel

SET attributes of clientNameLabel which are foreground, font and bounds to appropriate values.

ADD clientNameLabel to rightPanel1

INITIALIZE clientNameField

SET attributes of clientNameField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD clientNameField to rightPanel1

INITIALIZE cardIdLabel

SET attributes of cardIdLabel which are foreground, font and bounds to appropriate values.

ADD cardIdLabel to rightPanel1

INITIALIZE cardIdField

SET attributes of cardIdField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD keyListener to cardIdField

ADD cardIdField to rightPanel1

INITIALIZE bankAccountLabel

SET attributes of bankAccountLabel which are foreground, font and bounds to appropriate values.

ADD bankAccountLabel to rightPanel1

INITIALIZE bankAccountField

SET attributes of bankAccountField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD bankAccountField to rightPanel1

INITIALIZE issuerBankLabel

SET attributes of issuerBankLabel which are foreground, font and bounds to appropriate values.

ADD issuerBankLabel to rightPanel1

INITIALIZE issuerBankField

SET attributes of issuerBankField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD issuerBankField to rightPanel1

INITIALIZE balanceAmountLabel

SET attributes of balanceAmountLabel which are foreground, font and bounds to appropriate values.

ADD balanceAmountLabel to rightPanel1

INITIALIZE balanceAmountField

SET attributes of balanceAmountField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD balanceAmountField to rightPanel1

INITIALIZE rightPanel1ClearButton

SET attributes of rightPanel1ClearButton which are foreground, background, border, font, bounds and focusable to appropriate values

ADD actionListener to rightPanel1ClearButton

ADD rightPanel1ClearButton to leftPanelPanel

INITIALIZE currentPanel to rightPanel1

INITIALIZE rightPanel2

SET attributes of rightPanel2 which are Layout, bounds, background and visible to appropriate values.

ADD rightPanel2 to mainFrame

INITIALIZE rightPanel2Header

SET attributes of rightPanel2Header which are foreground, font and bounds to appropriate values.

ADD rightPanel2Header to rightPanel2

INITIALIZE pinNumberLabel

SET attributes of pinNumberLabel which are foreground, font and bounds to appropriate values.

ADD pinNumberLabel to rightPanel2

INITIALIZE pinNumberField

SET attributes of clientNameField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD keyListener to pinNumberField

ADD pinNumberField to rightPanel2

INITIALIZE withdrawalAmountLabel

SET attributes of withdrawalAmountLabel which are foreground, font and bounds to appropriate values.

ADD withdrawalAmountLabel to rightPanel2

INITIALIZE pinNumberField

SET attributes of clientNameField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD keyListener to pinNumberField

ADD pinNumberField to rightPanel2

INITIALIZE withdrawalAmountField

SET attributes of withdrawalAmountField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD actionListener to withdrawalAmountField

ADD keyListener to withdrawalAmountField

ADD withdrawalAmountField to rightPanel2

INITIALIZE withdrawalDateLabel

SET attributes of withdrawalDateLabel which are foreground, font and bounds to appropriate values.

ADD withdrawalDateLabel to rightPanel2

INITIALIZE rightPanel2YearComboBox to String datatype

SET attributes of rightPanel2YearComboBox which are background, foreground, font, focusable and bounds to appropriate values.

ADD actionListener to rightPanel2YearComboBox

ADD rightPanel2YearComboBox to rightPanel2

INITIALIZE rightPanel2MonthComboBox to String datatype

SET attributes of rightPanel2MonthComboBox which are background, foreground, font, focusable and bounds to appropriate values.

ADD actionListener to rightPanel2MonthComboBox

ADD rightPanel2MonthComboBox to rightPanel2

INITIALIZE rightPanel2DayComboBox to String datatype

SET attributes of rightPanel2DayComboBox which are background, foreground, font, focusable and bounds to appropriate values.

ADD actionListener to rightPanel2DayComboBox

ADD rightPanel2DayComboBox to rightPanel2

INITIALIZE addDebit

SET attributes of addDebit which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to addDebit

ADD addDebit to rightPanel2

INITIALIZE withdraw

SET attributes of withdraw which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to withdraw

ADD withdraw to rightPanel2

INITIALIZE displayDebit

SET attributes of displayDebit which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to displayDebit

ADD displayDebit to rightPanel2

INITIALIZE clearDebit

SET attributes of clearDebit which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to clearDebit

ADD clearDebit to rightPanel2

INITIALIZE rightPanel3

SET attributes of rightPanel3 which are Layout, bounds, background and visible to appropriate values.

ADD rightPanel3 to mainFrame

INITIALIZE rightPanel3Header

SET attributes of rightPanel2Header which are foreground, font and bounds to appropriate values.

ADD rightPanel3Header to rightPanel3

INITIALIZE cvcNumber

SET attributes of cvcNumber which are foreground, font and bounds to appropriate values.

ADD cvcNumber to rightPanel3

INITIALIZE cvcNumberField

SET attributes of cvcNumberField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD cvcNumberField to rightPanel3

INITIALIZE interestRate

SET attributes of interestRate which are foreground, font and bounds to appropriate values.

ADD interestRate to rightPanel3

INITIALIZE interestRateField

SET attributes of interestRateField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD interestRateField to rightPanel3

INITIALIZE expirationDate

SET attributes of expirationDate which are foreground, font and bounds to appropriate values.

ADD expirationDate to rightPanel3

INITIALIZE rightPanel3YearComboBox to String datatype

SET attributes of rightPanel3YearComboBox which are background, foreground, font, focusable and bounds to appropriate values.

ADD actionListener to rightPanel3YearComboBox

ADD rightPanel3YearComboBox to rightPanel3

INITIALIZE rightPanel3MonthComboBox to String datatype

SET attributes of rightPanel3MonthComboBox which are background, foreground, font, focusable and bounds to appropriate values.

ADD actionListener to rightPanel3MonthComboBox

ADD rightPanel3MonthComboBox to rightPanel3

INITIALIZE rightPanel3DayComboBox to String datatype

SET attributes of rightPanel3DayComboBox which are background, foreground, font, focusable and bounds to appropriate values.

ADD actionListener to rightPanel3DayComboBox

ADD rightPanel3DayComboBox to rightPanel3

INITIALIZE addCredit

SET attributes of addCredit which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to addCredit

ADD addCredit to rightPanel3

INITIALIZE clearCredit1

SET attributes of clearCredit1 which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to clearCredit1

ADD clearCredit1 to rightPanel3

INITIALIZE creditLimit

SET attributes of creditLimit which are foreground, font and bounds to appropriate values.

ADD creditLimit to rightPanel3

INITIALIZE creditLimitField

SET attributes of creditLimitField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD creditLimitField to rightPanel3

INITIALIZE gracePeriod

SET attributes of gracePeriod which are foreground, font and bounds to appropriate values.

ADD gracePeriod to rightPanel3

INITIALIZE gracePeriodField

SET attributes of gracePeriodField which are background, foreground, font, border, caretColor and bounds to appropriate values.

ADD gracePeriodField to rightPanel3

INITIALIZE clearCredit2

SET attributes of clearCredit2 which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to clearCredit2

ADD clearCredit2 to rightPanel3

INITIALIZE cancelCredit

SET attributes of cancelCredit which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to cancelCredit

ADD cancelCredit to rightPanel3

INITIALIZE setCreditLimit

SET attributes of setCreditLimit which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to setCreditLimit

ADD setCreditLimit to rightPanel3

INITIALIZE displayCredit

SET attributes of displayCredit which are foreground, background, border, font, bounds and focusable to appropriate values.

ADD actionListener to displayCredit

ADD displayCredit to rightPanel3

INITIALIZE rightPanel4

SET attributes of rightPanel4 which are Layout, bounds, background and visible to appropriate values.

ADD rightPanel4 to mainFrame

INITIALIZE rightPanel4Header

SET attributes of rightPanel4Header which are foreground, font and bounds to appropriate values.

ADD rightPanel4Header to rightPanel4

INITIALIZE theme

SET attributes of theme which are foreground, font and bounds to appropriate values.

ADD theme to rightPanel4

INITIALIZE darkMode

ADD actionListener to darkMode

SET the attributes which are selected, bounds, background, font, foreground and focusable to appropriate values.

INITIALIZE lightMode

ADD actionListener to lightMode

SET the attributes which are bounds, background, font, foreground and focusable to appropriate values.

INITIALIZE buttonGroup

ADD darkMode to buttonGroup

ADD lightMode to buttonGroup

ADD darkMode to rightPanel4

ADD lightMode to rightPanel4

SET attributes of mainFrame which are resizable, undecorated, layout, size, visible, defaultCloseOperation, LocationRelativeTo to appropriate values.

END DO

IMPLEMENT actionPerformed method ActionListener class with no return type

DO

DECLARE instance variables interestRate and creditLimit as double

DECLARE instance variables cvcNumber, balanceAmount, cardId, pinNumber, withdrawalAmount, gracePeriod as int

DECLARE and **INITIALIZE** instance variables catchBalanceAmount, catchCardId, catchPinNumber, catchCvcNumber, catchInterestRate, emptyBalanceAmount, emptyCardId, emptyPinNumber, emptyBankAccount, emptyIssuerBank, emptyClientName, emptyCvcNumber, emptyInterestRate, emptyWithdrawalAmount, emptyCreditLimit, emptyGracePeriod, isEmptyForDebit, isEmptyForCredit, isEmptyForWithdrawal, isEmptyForSetLimit, incorrectValueForDebit, incorrectValueForCredit, incorrectValueForWithdrawal, incorrectValueForSetLimit, catchBankAccount, catchIssuerBank, catchClientName, catchWithdrawalAmount, catchCreditLimit, catchGracePeriod, exception and isMatch as boolean to false.

IF the user has clicked addCredit or if the user has clicked addDebit

TRY

INITIALIZE balanceAmount to text field value of balance amount which is cast as int.

END TRY

CATCH NumberFormatException as exception1

INITIALIZE catchBalanceAmount to true.

IF the text field value of balance amount is empty

INITIALIZE emptyBalanceAmount to true

END IF

END CATCH

TRY

INITIALIZE cardId to text field value of card Id which is cast as int.

END TRY

CATCH NumberFormatException as exception1

INITIALIZE cardId to true.

IF the text field value of cardId is empty

INITIALIZE emptyCardId to true

END IF

END CATCH

TRY

INITIALIZE pinNumber to text field value of pinNumber which is cast as int.

END TRY

CATCH NumberFormatException as exception1

INITIALIZE catchPinNumber to true.

IF the text field value of pinNumber is empty

INITIALIZE emptyPinNumber to true

END IF

END CATCH

TRY

INITIALIZE cvcNumber to text field value of cvcNumber which is cast as int.

END TRY

CATCH NumberFormatException as exception1

INITIALIZE catchCvcNumber to true.

IF the text field value of cvcNumber is empty

INITIALIZE emptyCvcNumber to true

END IF

END CATCH

TRY

INITIALIZE interestRate to text field value of interest rate which is cast as double.

END TRY

CATCH NumberFormatException as exception1

INITIALIZE catchInterestRate to true.

IF the text field value of interestRate is empty

INITIALIZE emptyInterestRate to true

END IF

END CATCH

TRY

INITIALIZE withdrawalAmount to text field value of withdrawalAmount which is cast as int.

END TRY

CATCH NumberFormatException as exception1

INITIALIZE catchWithdrawalAmount to true.

IF the text field value of withdrawalAmount is empty

INITIALIZE emptyWithdrawalAmount to true

END IF

END CATCH

TRY

INITIALIZE creditLimit to text field value of creditLimit which is cast as double.

END TRY

CATCH NumberFormatException as exception1

INITIALIZE catchCreditLimit to true.

IF the text field value of creditLimit is empty

INITIALIZE emptyCreditLimit to true

END IF

END CATCH

TRY

INITIALIZE gracePeriod to text field value of gracePeriod which is cast as int.

END TRY

CATCH NumberFormatException as exception1

INITIALIZE catchGracePeriod to true.

IF the text field value of gracePeriod is empty

INITIALIZE emptyGracePeriod to true

END IF

END CATCH

IF bankAccountField is empty

INITIALIZE emptyBankAccount to true

END IF

IF issuerBankField is empty

INITIALIZE emptyIssuerBank to true

END IF

IF clientNameField is empty

INITIALIZE emptyClientName to true

END IF

IF bankAccount is empty or if issuerBank is empty or if clientName is empty
or if balanceAmount is empty or if cardId is empty or if pinNumber is empty

INITIALIZE isEmptyForDebit to true

END IF

IF exception is caught in balanceAmount or if exception is caught in cardId or
if exception is caught in pinNumber

INITIALIZE incorrectValueForDebit to true

END IF

IF bankAccount is empty or if issuerBank is empty or if clientName is empty
or if balanceAmount is empty or if cardId id empty or if cvcNumber is empty
or if interestRate is empty

INITIALIZE isEmptyForCredit to true

END IF

IF exception is caught in balanceAmount or if exception is caught in
catchCardId or if exception is caught in cvcNumber or if exception is caught
in interestRate

INITIALIZE incorrectValueForCredit to true a

END IF

END IF

Increment the value of i by 1

IF i is equal to 10

SET text of loadingFrameLabel1 to "Loading.."

END IF

IF i is equal to 20

SET text of loadingFrameLabel1 to "Loading..."

END IF

IF i is equal to 30

SET text of loadingFrameLabel1 to "Loading."

END IF

IF i is equal to 40

SET text of loadingFrameLabel1 to "Loading.."

END IF

IF i is equal to 50

SET text of loadingFrameLabel1 to "Loading..."

END IF

IF i is equal to 55

SET text of loadingFrameLabel1 to "Loading."

END IF

IF i is equal to 70

SET text of loadingFrameLabel1 to "Loading.."

END IF

IF i is equal to 80

SET text of loadingFrameLabel1 to "Loading..."

END IF

IF i is equal to 90

SET text of loadingFrameLabel1 to "Loading."

END IF

IF i is equal to 100

SET text of loadingFrameLabel1 to "Loading.."

STOP timer1

DISPOSE loadingFrame

SET visibility of mainFrame to true

END IF

SET value of progressBar to i

IF the user has clicked closeButton

DISPOSE mainFrame

END IF

ELSE the user has clicked maximizeButton

ELSE the user has clicked minimizeButton

MINIMIZE mainFrame

ELSE the user has clicked detailsButton

SET the visibility of currentPanel to false

INITIALIZE currentPanel to rightPanel1

SET the visibility of rightPanel1 to true

ELSE the user has clicked debitCardButton

SET the visibility of currentPanel to false

INITIALIZE currentPanel to rightPanel2

SET the visibility of rightPanel2 to true

ELSE the user has clicked creditCardButton

SET the visibility of currentPanel to false

INITIALIZE currentPanel to rightPanel3

SET the visibility of rightPanel3 to true

ELSE the user has clicked settingButton

SET the visibility of currentPanel to false

INITIALIZE currentPanel to rightPanel4

SET the visibility of rightPanel4 to true

ELSE the user has clicked addDebit

TRY

IF bankAccount is empty or if issuerBank is empty or if clientName is empty or if balanceAmount is empty or if cardId is empty or if pinNumber is empty or if exception is caught in balance amount or if exception is caught in cardId or if exception is caught in pinNumber

THROW a NumberFormatException

END IF

INITIALIZE bankAccount, issuerBank and clientName as String and to values extracted from corresponding text fields.

INITIALIZE balanceAmount, cardId and pinNumber as int and to values extracted from corresponding text fields.

INITIALIZE itr to an iterator of the arrayList

IF arrayList is empty

DECLARE and **INITIALIZE** obj as DebitCard and pass balanceAmount, cardId, bankAccount, issuerBank, clientName and pinNumber to its constructor in that order

ADD obj to arrayList

SHOW a message dialog informing the user that the debit card has been added successfully.

END IF

ELSE

WHILE there is an element to the right of the cursor of iterator

DOWNCAST the object returned by itr.next() to BankCard as bankCard

IF bankcard is an instance of DebitCard

IF the cardId of the bankcard is equal to the cardId entered by the user

INITIALIZE isMatch to true

SHOW a message box informing the user that he/she cannot add the same debit card twice

BREAK

END IF

END WHILE

IF isMatch is equal to false

DECLARE and **INITIALIZE** obj as DebitCard and pass balanceAmount, cardId, bankAccount, issuerBank, clientName and pinNumber to its constructor in that order

ADD obj to arrayList

SHOW a message dialog informing the user that the debitCard has been added successfully.

END IF

END TRY

CATCH NumberFormatException as exception1

SHOW a message dialog informing the user that wrong values were entered.

END CATCH

ELSE the user has clicked addCredit

TRY

IF bankAccount is empty or if issuerBank is empty or if clientName is empty or if balanceAmount is empty or if cardId is empty or if cvcNumber is empty or if interest rate is empty or if exception is caught in balance amount or if exception is caught in cardId or if exception is caught in cvcNumber or if exception is caught in interest rate

THROW a NumberFormatException

END IF

INITIALIZE bankAccount, issuerBank and clientName as String and to values extracted from corresponding text fields.

INITIALIZE rightPanel3Year, rightPanel3Month, rightPanel3Day as String and to values extracted from corresponding combo boxes.

INITIALIZE balanceAmount, cardId, cvcNumber as int and to values extracted from corresponding text fields.

INITIALIZE interest rate as double to value extracted from interest rate text field

INITIALIZE itr to an iterator of the arrayList

```

    IF arrayList is empty

        DECLARE and INITIALIZE obj as CreditCard and pass cardId,
        clientName, issuerBank, bankAccount, balanceAmount, cvcNumber,
        interestRate and expirationDate to its constructor in that order

        ADD obj to arrayList

        SHOW a message dialog informing the user that the credit card has been
        added successfully.

    END IF

ELSE

    WHILE there is an element to the right of the cursor of iterator

        DOWNCAST the object returned by itr.next() to BankCard as bankCard

        IF bankcard is an instance of CreditCard

            IF the cardId of the bankcard is equal to the cardId entered by the
            user

                INITIALIZE isMatch to true

                SHOW a message box informing the user that he/she cannot add
                the same credit card twice

                BREAK

            END IF

        END WHILE

        IF isMatch is equal to false

            DECLARE and INITIALIZE obj as CreditCard and pass
            balanceAmount, cardId, bankAccount, issuerBank, clientName and pinNumber
            to its constructor

            ADD obj to arrayList

            SHOW a message dialog informing the user that the credit card
            has been added successfully.

        END IF

    END TRY

```

CATCH NumberFormatException as exception1

SHOW a message dialog informing the user that wrong values were entered.

END CATCH

ELSE the user has clicked withdraw

TRY

INITIALIZE cardId, pinNumber and withdrawal amount as int and to values extracted from corresponding text fields.

DECLARE and **INITIALIZE** isMatchCardId and isMatchPinNumber as boolean and to values extracted from corresponding text fields.

INITIALIZE rightPanel2Year, rightPanel2Month, rightPanel2Day as String and to values extracted from corresponding combo boxes.

INITIALIZE dateOfWithdrawal to concatenated value of rightPanel2Year, rightPanel2Month and rightPanel2Day

IF arrayList is empty

SHOW a message dialog informing the user that the database is empty.

END IF

ELSE

INITIALIZE itr to an iterator of the arrayList

WHILE there is an element to the right of the cursor of iterator

DOWNCAST the object returned by itr.next() to BankCard as bankCard

IF bankcard is an instance of DebitCard

DOWNCAST the object bankCard to DebitCard as debitCard

IF the cardId of the debit card is equal to the cardId entered by the user and if the pin number entered by the user is equal to the pin number of the debit card

CALL withdraw method of the debitCard

SHOW appropriate message box informing the user if the withdrawal was successful or not.

BREAK

END IF

END IF

END WHILE

IF isMatch is equal to false

SHOW a message dialog informing the user that a valid card ID was not entered

ELSE

SHOW a message dialog informing the user that a valid pin number was not entered

END IF

END TRY

CATCH NumberFormatException as exception1

SHOW a message dialog informing the user that wrong values were entered.

END CATCH

ELSE the user has clicked setCreditLimit

TRY

INITIALIZE cardId and grace period as int and to values extracted from corresponding text fields.

INITIALIZE creditLimit as double and to the value extracted from creditLimit text field.

DECLARE and **INITIALIZE** isMatchCredit as boolean to false

```

        IF arrayList is empty
            SHOW a message dialog informing the user that the database is
empty.
        END IF

    ELSE
        INITIALIZE itr to an iterator of the arrayList
        WHILE there is an element to the right of the cursor of iterator
            DOWNCAST the object returned by itr.next() to BankCard as
bankCard
            IF bankcard is an instance of CreditCard
                DOWNCAST the object bankCard to CreditCard as creditCard
                IF the cardId of the card card is equal to the cardId entered by
the user
                    CALL setCreditLimit method of the creditCard
                    INITIALIZE isMatchCredit to true
                    SHOW appropriate message box informing the user if credit
limit was granted or not.
                BREAK
            END IF
        END IF
    END WHILE

    IF isMatchCredit is equal to false
        SHOW a message dialog informing the user that a valid card
ID was not entered
    ELSE
        SHOW a message dialog informing the user that there is no
entry of credit card of entered card id in the database.
    END IF

```

END TRY

CATCH NumberFormatException as exception1

SHOW a message dialog informing the user that wrong values were entered.

END CATCH

ELSE the user has clicked cancelCredit

TRY

INITIALIZE cardId as int and to the value extracted from cardId text field.

INITIALIZE itr to an iterator of the arrayList

DECLARE and **INITIALIZE** isMatchCredit and isGranted as boolean to false

IF arrayList is empty

SHOW a message dialog informing the user that the database is empty.

END IF

ELSE

WHILE there is an element to the right of the cursor of iterator

DOWNCAST the object returned by itr.next() to BankCard as bankCard

IF bankcard is an instance of CreditCard

DOWNCAST the object bankCard to CreditCard as creditCard

IF the cardId of the card card is equal to the cardId entered by the user

CALL setCreditLimit method of the creditCard

SHOW appropriate message box informing that credit limit was cancelled.

BREAK

END IF
END IF
END WHILE

IF isMatchCredit is equal to false
SHOW a message dialog informing the user that there is no entry of credit card of entered card id in the database
ELSE
SHOW a message dialog informing the user that credit limit is not granted or it has been cancelled before
END IF

END TRY

CATCH NumberFormatException as exception1

SHOW a message dialog informing the user that wrong values were entered.

END CATCH

ELSE the user has clicked displayDebit

TRY

INITIALIZE cardId as int and to the value extracted from cardId text field.

INITIALIZE isMatch as boolean to false

INITIALIZE itr to an iterator of the arrayList

IF arrayList is empty

SHOW a message dialog informing the user that the database is empty.

END IF

```

ELSE
    WHILE there is an element to the right of the cursor of iterator
        DOWNCAST the object returned by itr.next() to BankCard as
bankCard
        IF bankcard is an instance of DebitCard
            DOWNCAST the object bankCard to DebitCard as debitCard
            IF the cardId of the card card is equal to the cardId entered by
the user
                CALL display method of the DebitCard
                SHOW a message informing that debit card details were
                displayed successfully.
            BREAK
        END IF
    END IF
END WHILE

IF isMatch is equal to false
    SHOW a message dialog informing the user that there is no
entry of debit card of entered card id in the database
END IF

END TRY
CATCH NumberFormatException as exception1
    SHOW a message dialog informing the user that wrong values were
entered.
END CATCH

```

```

ELSE the user has clicked displayCredit
    TRY

        INITIALIZE cardId as int and to the value extracted from cardId text
field.

        INITIALIZE isMatch as boolean to false
        INITIALIZE itr to an iterator of the arrayList

        IF arrayList is empty
            SHOW a message dialog informing the user that the database is
empty.
        END IF

        ELSE

            WHILE there is an element to the right of the cursor of iterator
                DOWNCAST the object returned by itr.next() to BankCard as
bankCard

                IF bankcard is an instance of CreditCard
                    DOWNCAST the object bankCard to CreditCard as creditCard
                    IF the cardId of the card card is equal to the cardId entered by
the user

                        CALL display method of the CreditCard

                        SHOW a message informing that credit card details were
displayed successfully.

                    BREAK

                END IF
            END IF
        END WHILE

```

IF isMatch is equal to false

SHOW a message dialog informing the user that there is no entry of credit card of entered card id in the database

END IF

END TRY

CATCH NumberFormatException as exception1

SHOW a message dialog informing the user that wrong values were entered.

END CATCH

END DO

CREATE the main method of java with arbitrary number of arguments

DO

CALL an anonymous object of BankGUI

END DO

END DO

Method descriptions

Constructor

The design of the whole of the GUI was done in the constructor of the BankGUI class. Instantiating it as an anonymous object in main method makes the GUI appear to the user. All the instance variables related to the creation of the GUI is initialized here. The attributes of all the components such as bounds, font, foreground color, background color etc was set inside the constructor.

actionPerformed

The actionPerformed accepts(ActionEvent) as its parameters. Any event generated is passed inside this method and onto the respective if or else blocks so that a certain fix action occurs when a certain button is clicked.

keyPressed

This method overrides the keyPressed method of KeyListener interface and is left empty as there was no functionality need from this method.

keyTyped

This method overrides the keyTyped method of KeyListener interface and is left empty as there was no functionality need from this method.

keyReleased

This method overrides the keyReleased method of KeyListener interface. This method enables iteration over the arraylist on each keystroke so that when valid card id is entered into the GUI, an appropriate message displaying all the needed fields is displayed.

4.1 BankGUI

4.1.1 Buttons

addDebit

The try catch block encapsulates whole of this button. If any of the text fields are empty or if any exception is caught for its text field values, an exception thrown redirects the flow of code towards the catch block. The necessary variables storing the corresponding values of the text fields are initialized here. The variables are bank account, client name, issuer bank, balance amount, card id and pin number.

If the arrayList is empty, then the user is able to add an object of DebitCard class after entering all the necessary text field values. A message that a debit card is added is displayed afterwards.

For the second time when the user tries to add an entry into the arrayList, an iteration which checks for instance of the current object in the first step and card id in the second step evaluates if there ever was a debitCard object with the card id matching to that of the entered card id value. If there was no match, the user is able to add an object of debit card. Else, only a message is displayed that the user cannot add the same debit card twice.

addCredit

The try catch block encapsulates whole of this button. If any of the text fields are empty or if any exception is caught for its text field values, an exception thrown redirects the flow of code towards the catch block. The necessary variables storing the corresponding values of the text fields are initialized here. The variables are bank account, client name, issuer bank, balance amount, card id, cvc number, interest rate and expiration date.

If the arrayList is empty, then the user is able to add an object of CreditCard class after entering all the necessary text field values. A message that a credit card is added is displayed afterwards.

For the second time when the user tries to add an entry into the arrayList, an iteration which checks for instance of the current object in the first step and card id in the second step evaluates if there ever was a credit card object with the card id matching to that of the entered card id value. If there was no match, the user is able

to add an object of credit card. Else, only a message is displayed that the user cannot add the same credit card twice.

withdraw

The try catch block encapsulates whole of this button. The necessary variables storing the corresponding values of the text fields are initialized here. The variables stored which are necessary for withdrawal are cardId, pin number and withdrawal amount.

If the arrayList is empty, a message that there are no entries in the database is displayed. If the arrayList is not empty, instance of current object is checked in the first step and card id and pin number is checked in the second step after downcasting. If the card id and pin number of the current object matches with that of the values entered by the user on the respective text fields, the withdraw method is called through downcasted object and an appropriate message is displayed informing that withdrawal was successful afterwards. If any of the values do not match, an appropriate message is displayed. .

setCreditLimit

The try catch block encapsulates whole of this button. The necessary variables storing the corresponding values of the text fields are initialized here. The variables stored which are necessary for withdrawal are cardId, grace period and credit limit.

If the arrayList is empty, a message that there are no entries in the database is displayed. If the arrayList is not empty, instance of current object is checked to verify if is of CreditCard in the first step and card id in the second step after downcasting. If the card id entered by the user on the text field matches with the card id of the current object of the credit card, the setCreditLimit method is called through downcasted object and an appropriate message informing that the credit limit was set successfully is displayed afterwards. If any of the values do not match, an appropriate message is displayed.

cancelCredit

The try catch block encapsulates whole of this button. The necessary variables storing the corresponding values of the text fields are initialized here. The variable stored which is necessary for cancelling the credit is cardId only.

If the arrayList is empty, a message that there are no entries in the database is displayed. If the arrayList is not empty, instance of current object is checked to verify if it is of CreditCard in the first step and card id in the second step after downcasting. If the card id entered by the user on the text field matches with the card id of the current object of the credit card, the cancelCreditCard method is called through downcasted object and an appropriate message informing that the credit limit was cancelled is displayed afterwards. If the card id does not match, an appropriate message is displayed.

displayDebit

The try catch block encapsulates whole of this button. The necessary variables storing the corresponding values of the text fields are initialized here. The variable stored which is necessary for displaying the details of the debit card is cardId only.

If the arrayList is empty, a message that there are no entries in the database is displayed. If the arrayList is not empty, instance of current object is checked to verify if it is of DebitCard in the first step and card id in the second step after downcasting. If the card id entered by the user on the text field matches with the card id of the current object of the debit card, the display method is called through downcasted object and an appropriate message informing that the details were displayed successfully is displayed afterwards. If the card id does not match, an appropriate message is displayed.

displayCredit

The try catch block encapsulates whole of this button. The necessary variables storing the corresponding values of the text fields are initialized here. The variable stored which is necessary for displaying the details of the credit card is cardId only.

If the arrayList is empty, a message that there are no entries in the database is displayed. If the arrayList is not empty, instance of current object is checked to verify if it is of Credit Card in the first step and card id in the second step after downcasting. If the card id entered by the user on the text field matches with the card id of the current object of the credit card, the display method is called through downcasted object and an appropriate message informing that the details were displayed successfully is displayed afterwards. If the card id does not match, an appropriate message is displayed.

withdrawalAmountField

The main purpose of adding action listener to this text field is if the user presses enter after entering any withdrawal amount, an iteration takes places which determines if the withdrawal amount entered is less than or equal to the balance amount of the card Id present in the text field of card id. If it is, a message dialog appears informing the user that a valid withdrawal amount for that card id was entered.

4.1.2 KeyListener text fields

cardIdField

A keylistener is added to this text field so that for each key entered and released by the user, two iterator iterate over the arrayList. The first iterator iterates to check if the current object is an instance of DebitCard while the second object iterates to check if the current object is an instance of credit card. The reason why two iterators were used instead of one is because two bank card object of different type may have same card id. If so, appropriate values are only displayed for one type of object after entering in the valid card id for the case where two different types apart from bank card may have the same card id.

If the valid conditions are met for each of the types, either one or two dialog box from here is displaye. The condition for validity is checked respectively for each type just after the iteration for that type.

withdrawalAmountField

It would be largely inconvenient to withdraw 1000 from a balance amount of 10000 if after each key stroke to type the whole withdrawal amount, a dialog box would be displayed. '1', '10 ', '100 ' and '1000 ' are all valid withdrawal amounts. The user would have to close four dialog boxes confirming that the amount after the key released was a valid withdrawal amount. As such, only validity is checked in each iteration and the value of the instance variable is initialized accordingly. If the user wants to check if the withdrawal amount entered is valid for that card id, the user may press enter on the

text field after entering the withdrawal amount. The card id text field should not be empty if the user wants to check if a valid withdrawal amount was entered.

pinNumberField

Different from withdrawal amount, a pin number is said to be valid if and only if the pin number entered by the user matches with the pin number of the entered card id. The user would not be prompted with multiple valid pin number prompts because there can only be one valid pin number, no more and no less. Therefore, a JDialog box would be displayed if the user has entered valid pin number for the card id value present in the card id text field.

Evidence

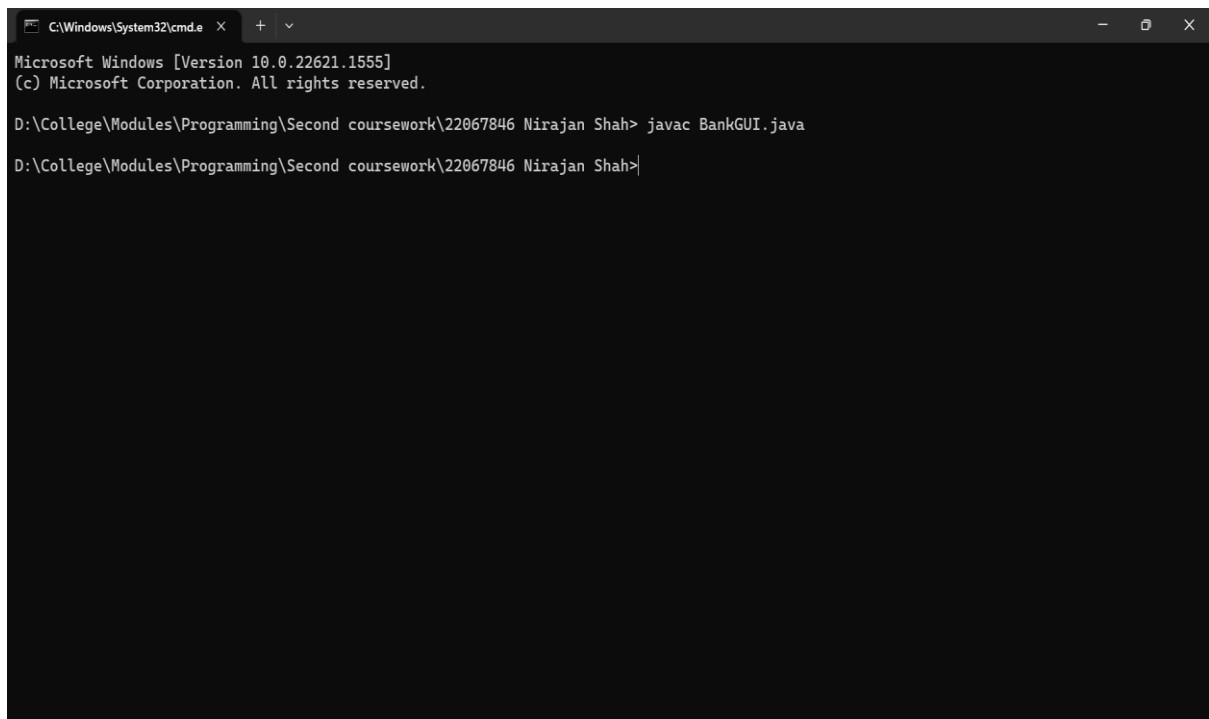
Testing of the software is a vital part of the software development process. Testing determines if the program is functioning as intended or not. A test case is the most basic unit of testing where a certain logical part of the program is tested by supplying input values and comparing the actual output with the expected output. If the actual output of the program is equal to the expected output, the test passes (Alexander, et al., 2002). Otherwise, the test fails. A test set is a group of all test cases. In this testing part of the project, we aim to simulate test cases and test set defined in the paper. The primary objective is to obtain a successful test set.

5.1 Test 1

5.1.1 Test case 1

Test case:	1
Objective:	To check if the program can be compiled using the command prompt.
Action:	a. The location of the project is navigated. b. Command prompt is opened in that location. c. The class BankGUI is compiled by entering the command javac BankGUI.java
Expected Result:	The BankGUI class would be compiled.
Actual Result:	The BankGUI class was compiled.
Conclusion:	Test case is successful.

Table 1: To compile the BankGUI class



```
C:\Windows\System32\cmd.e  x  +  v
Microsoft Windows [Version 10.0.22621.1555]
(c) Microsoft Corporation. All rights reserved.

D:\College\Modules\Programming\Second coursework\22067846 Nirajan Shah> javac BankGUI.java

D:\College\Modules\Programming\Second coursework\22067846 Nirajan Shah>
```

Figure 6: Screenshot of compiling the BankGUI class

5.1.2 Test case 2

Test case:	2
Objective:	To run the BankGUI class using command prompt
Action:	a. The location of the project is navigated. b. Command prompt is opened in that location. c. The class BankGUI is run by entering the command <code>java BankGUI.java</code>
Expected Result:	The BankGUI class would be run.
Actual Result:	The BankGUI class was run.
Conclusion:	Test case is successful.

Table 2: To run the BankGUI class

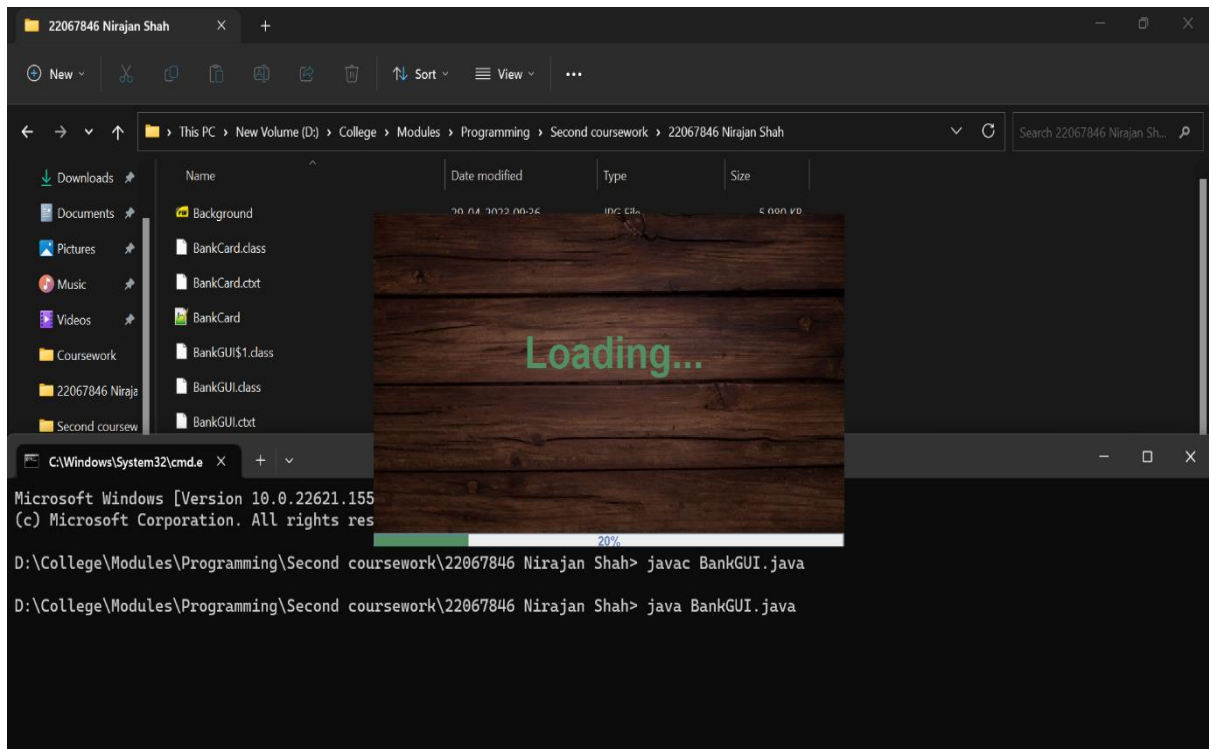


Figure 7: Screenshot of running the BankGUI class

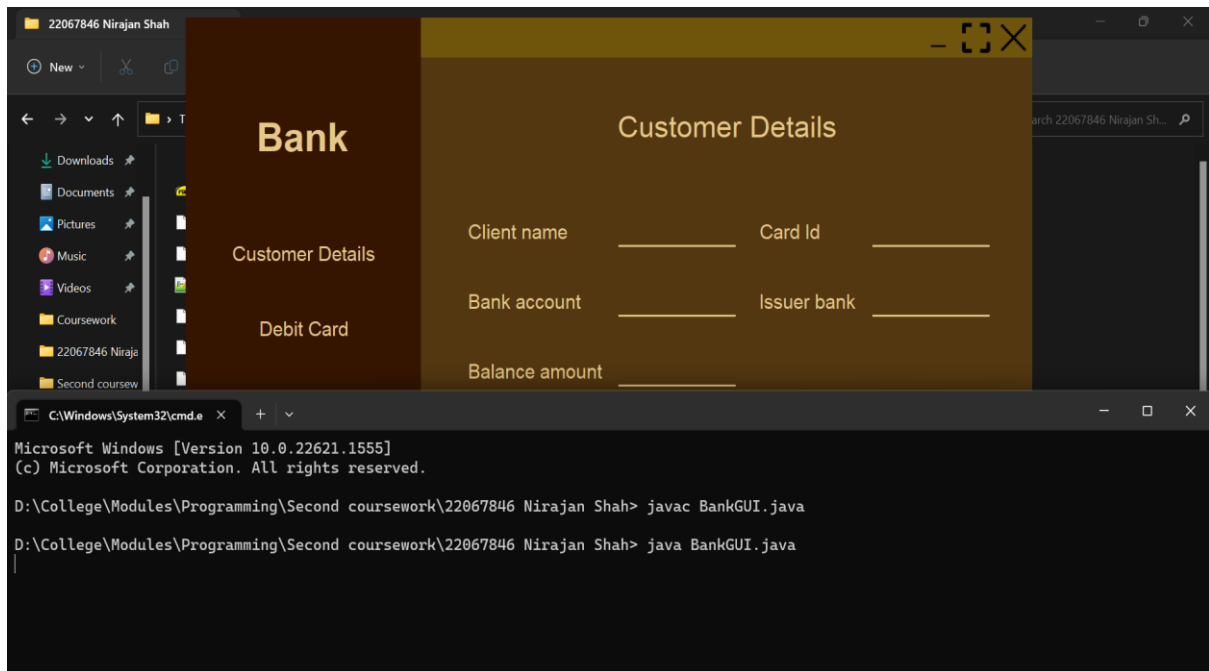


Figure 8: Screenshot of running the BankGUI class

5.2 Test 2

5.2.1 Test case 1

Test case:	1
Objective:	To check if debit card is added to the array list after clicking on add debit.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of debit card Balance amount = 50000 Card Id = 17320943 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Pin number = 5340 b. The Add button is clicked.
Expected Result:	There would be an object added to the array list and the entered text field values would match with the printed values.
Actual Result:	There was an object added to the array list and the entered text field values matched with the printed values.
Conclusion:	The test case is successful.

Table 3: To check if an object of DebitCard was added to the array list after clicking on add button

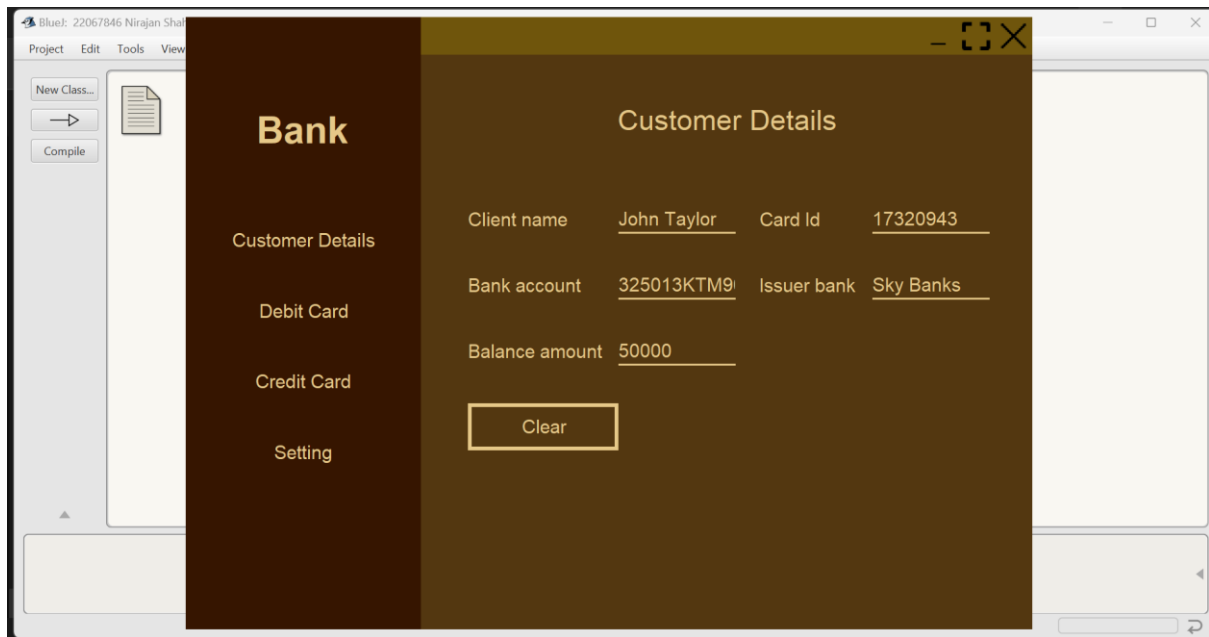


Figure 9: Screenshot displaying the entered text field values

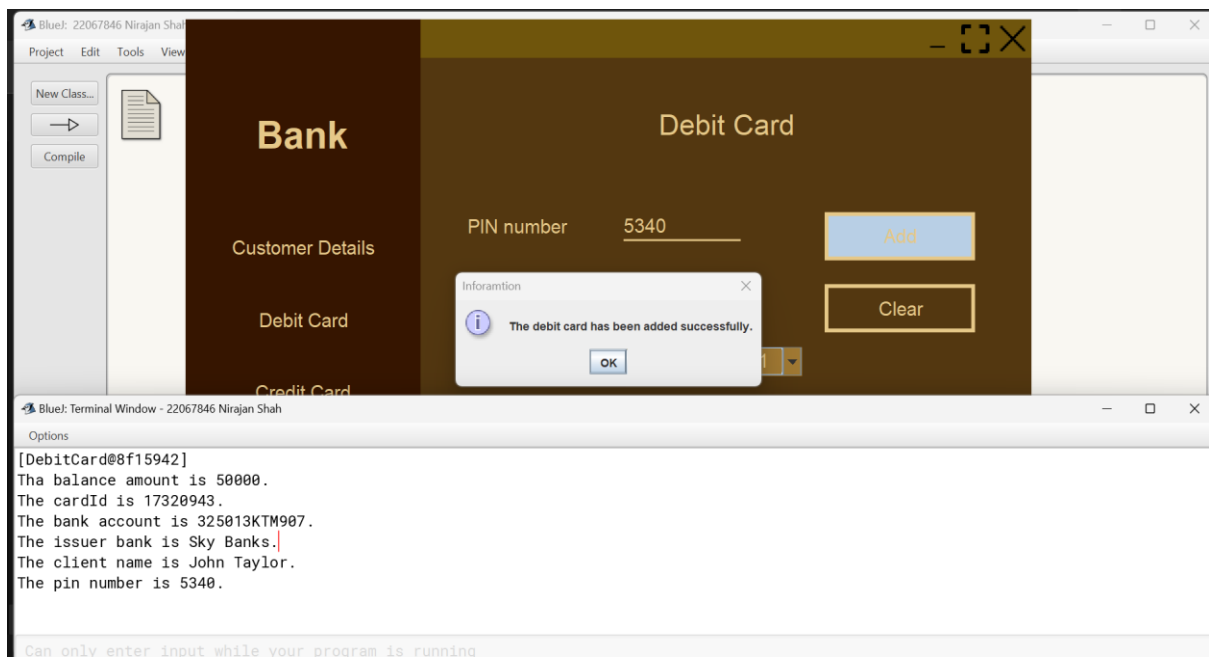


Figure 10: Screenshot after clicking on the add button

5.2.2 Test case 2

Test case:	2
Objective:	To check if credit card is added to the array list after clicking on add credit.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of credit card Balance amount = 50000 Card id = 54432934 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Cvc number = 890 interestRate= 16.0 b. The Add button is clicked.
Expected Result:	There would be an object added to the array list and the entered text field values would match with the printed values.
Actual Result:	There was an object added to the array list and the entered text field values matched with the printed values.
Conclusion:	The test case is successful.

Table 4: To check if credit card is added to the array list after clicking on add button

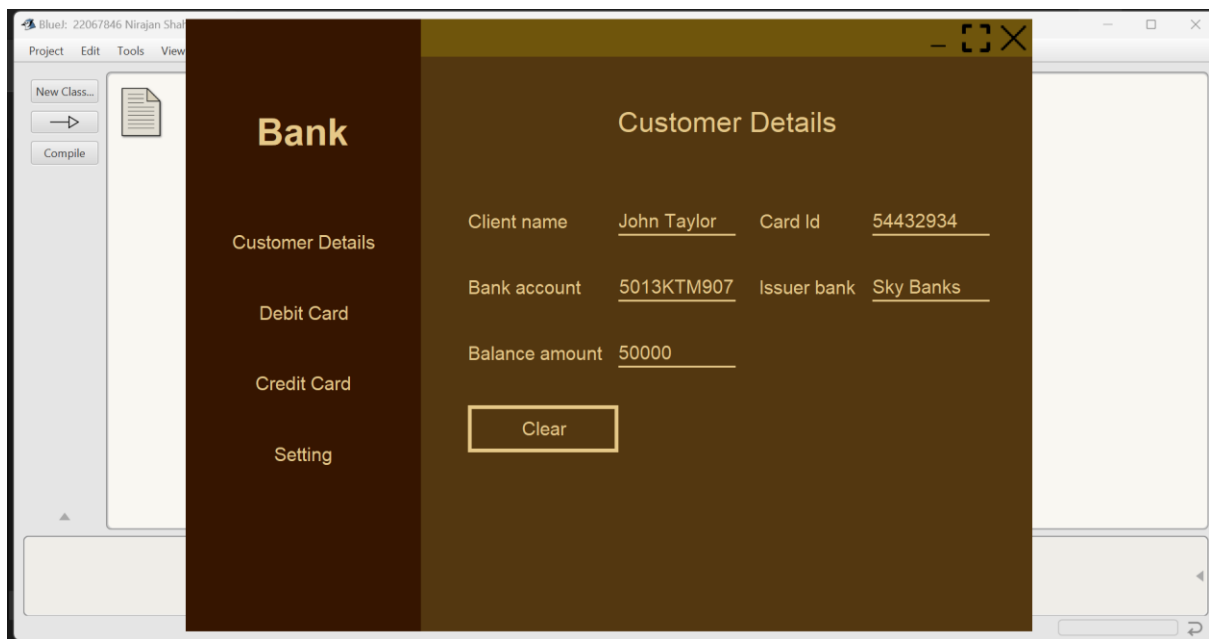


Figure 11: Screenshot displaying the entered text field values

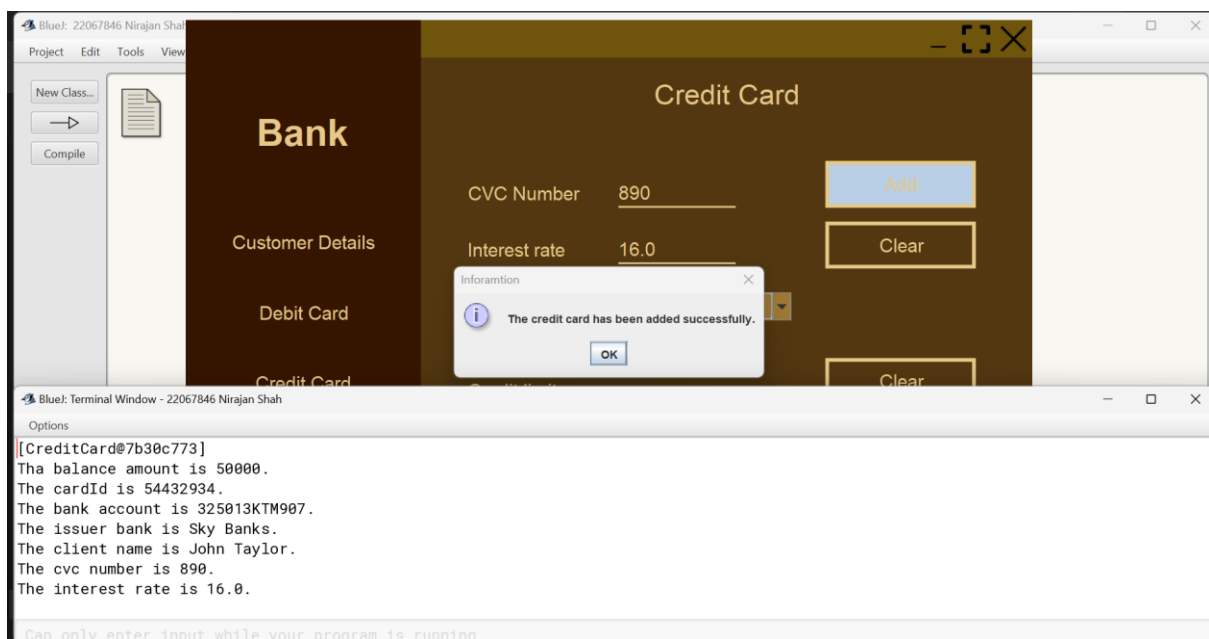


Figure 12: Screenshot after clicking on the add button

5.2.3 Test case 3

Test case:	3
Objective:	To check if a certain amount would be withdrawn from an instance of debit card from the GUI
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of debit card Balance amount = 50000 Card Id = 17320943 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Pin number = 5340 b. A certain amount is withdrawn with a certain withdrawal date. Withdrawal amount = 5000 Date of withdrawal = 2023/04/08 c. The withdraw button is clicked.
Expected Result:	A certain amount would be withdrawn and appropriate values would be displayed both in dialog box and in printing console.
Actual Result:	A certain amount was withdrawn and appropriate values were displayed both in dialog box and in printing console.
Conclusion:	The test case is successful.

Table 5: To check if a certain amount would be withdrawn from an instance of debit card from the GUI

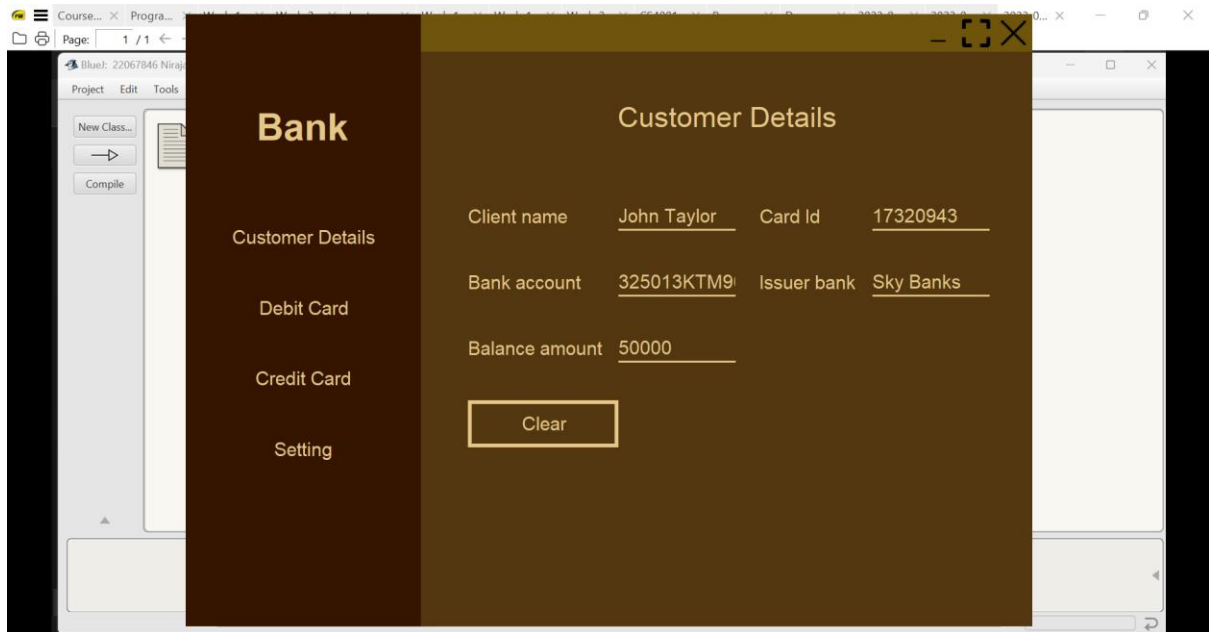


Figure 13: Screenshot displaying the entered text field values

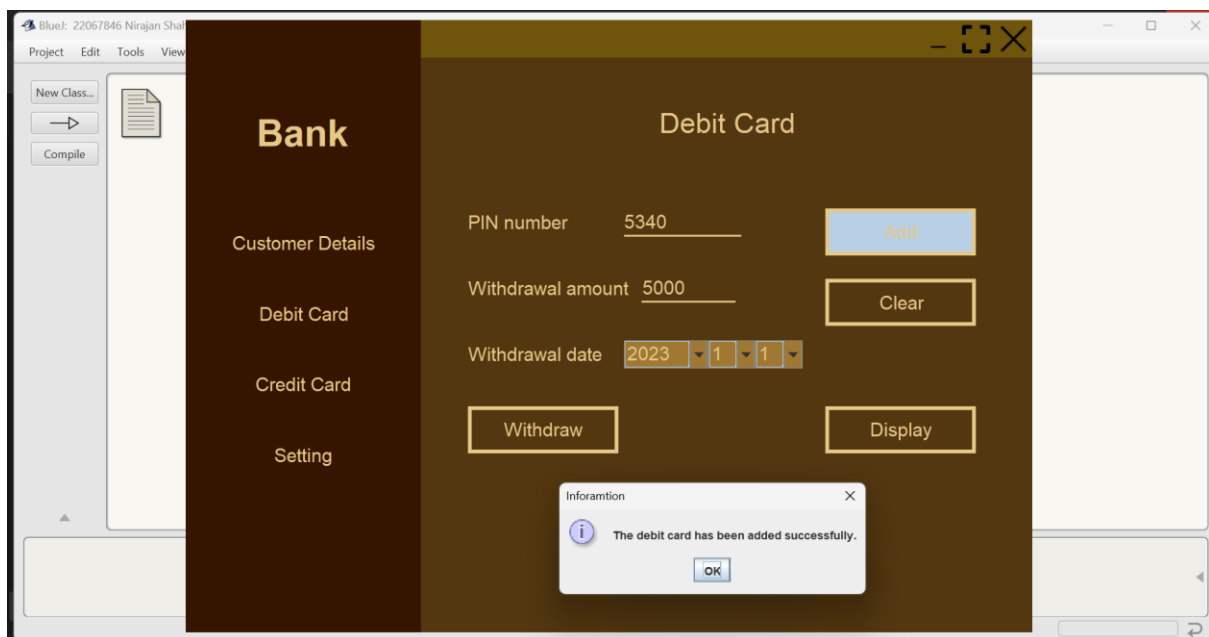


Figure 14: Screenshot after clicking on add button

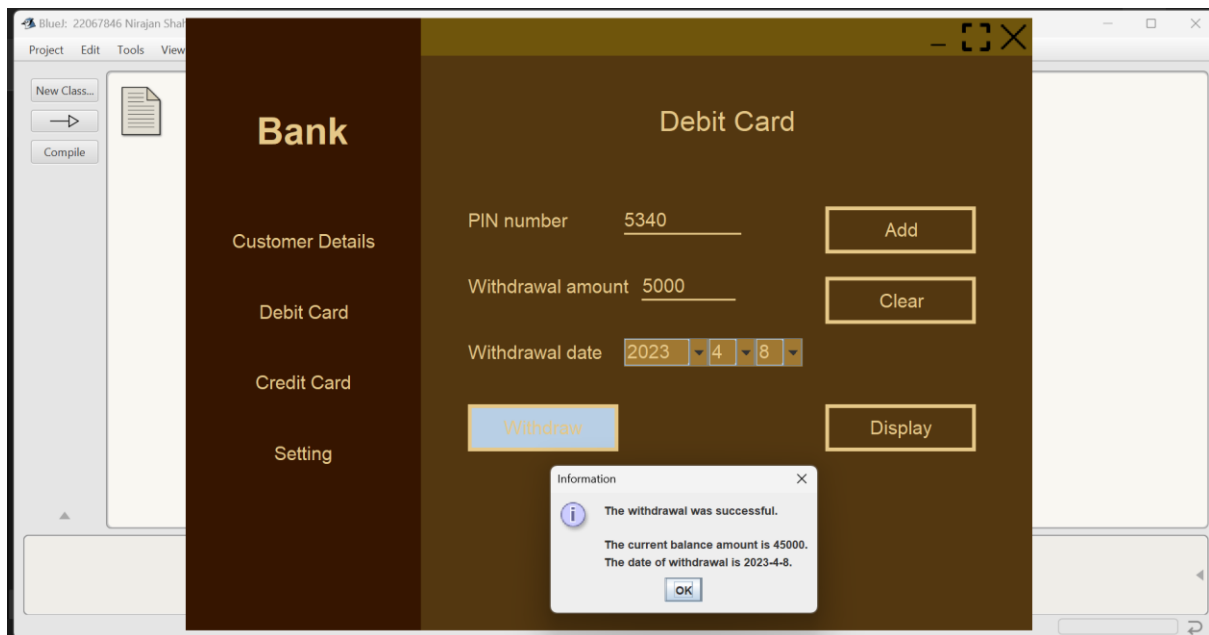


Figure 15: Screenshot after clicking on Withdraw button

5.2.4 Test case 4

Test case:	4
Objective:	To check if the entered pin number and withdrawal amount for a given card Id would be displayed in the JDialogBox from the key listener if valid values are entered for pin number, card Id and withdrawal amount.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of debit card Balance amount = 50000 Card Id = 17320943 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Pin number = 5340

	b. Add button is clicked. c. Text field values of card id and pin number is erased. d. The card id that was entered into the text field before is entered again. e. The pin number that was entered into the text field before is entered again. f. An amount is entered for withdrawal amount which is less than or equal to balance amount. g. Enter is pressed.
Expected Result:	Appropriate dialog box informing the user that valid values were entered for each individual text fields would be displayed and if valid card id, valid pin number and valid withdrawal amount were entered, a JDialog box would appear displaying all those values.
Actual Result:	Appropriate dialog box informing the user that valid values were entered for each individual text fields is displayed and if valid card id, valid pin number and valid withdrawal amount were entered, a JDialog box appeared displaying all those values.
Conclusion:	The test case is successful.

Table 6: To check if dialog messages are displayed after entering valid debit card attribute values

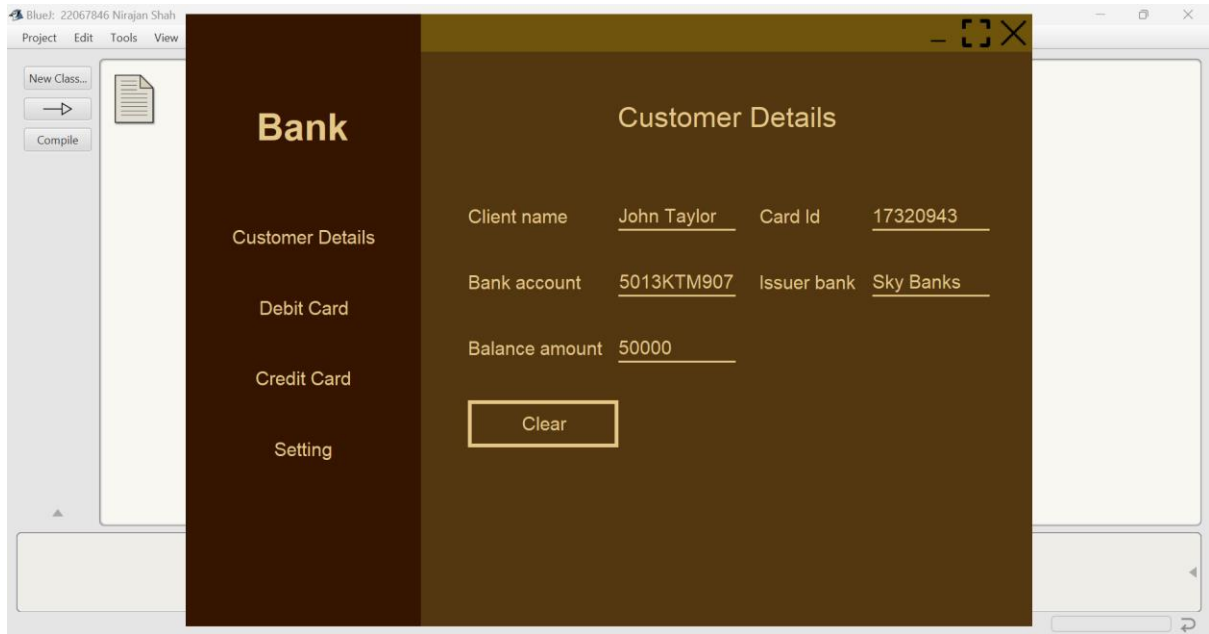


Figure 16: Screenshot displaying the entered text field values

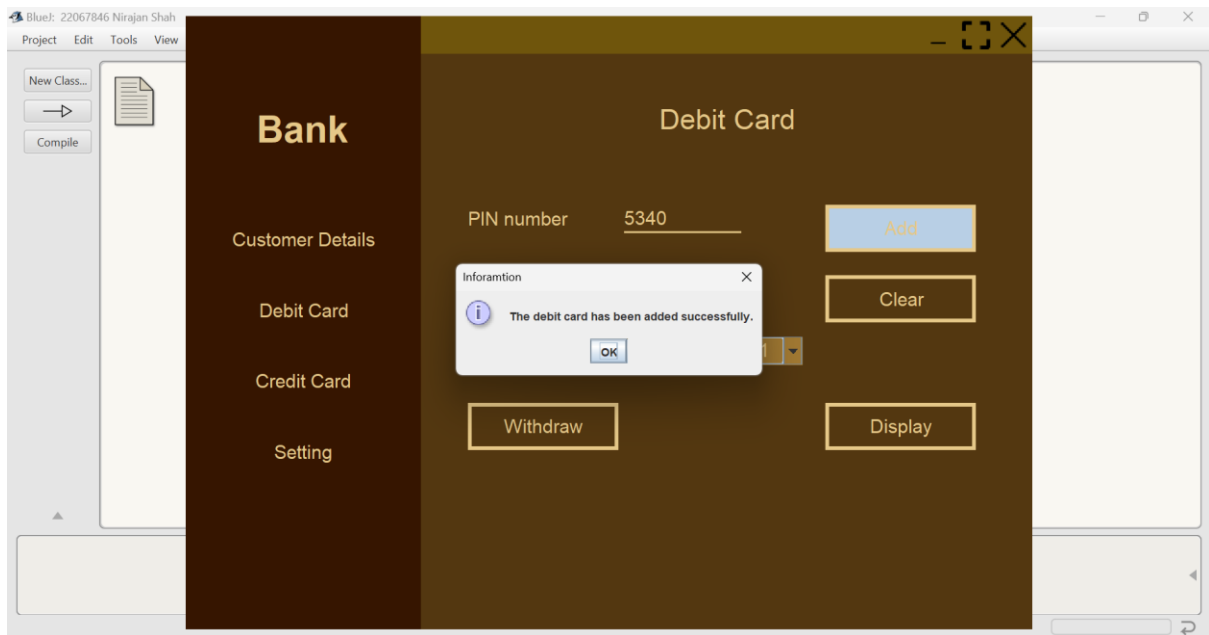


Figure 17: Screenshot after clicking on add button

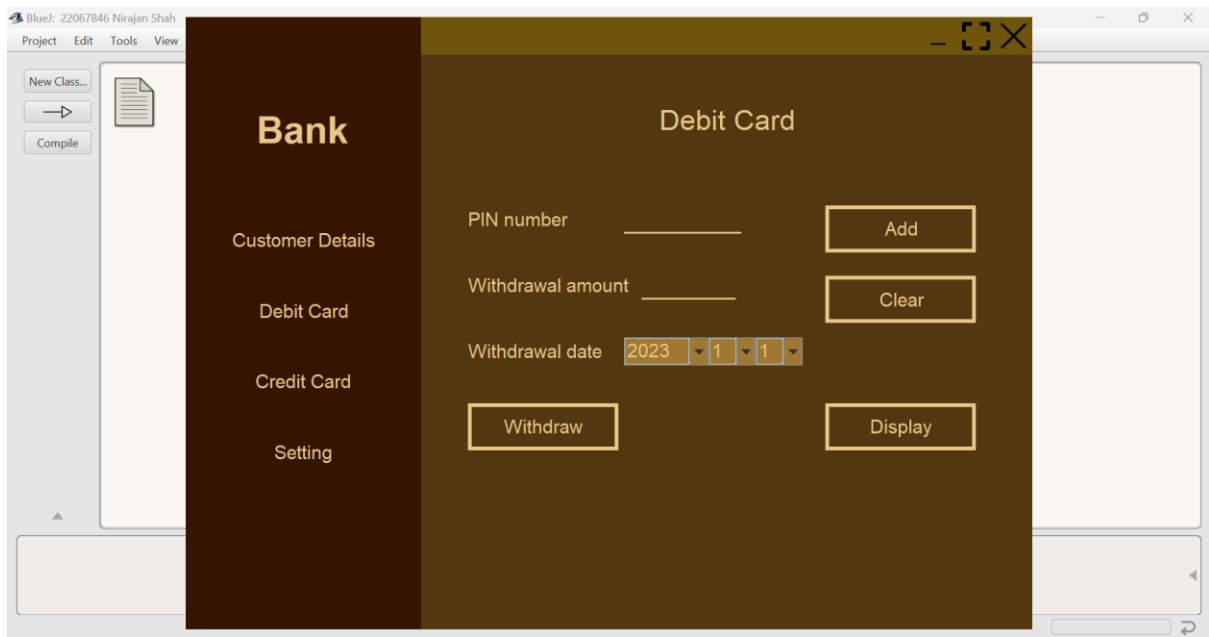


Figure 18: Screenshot after clearing out the PIN number text field value

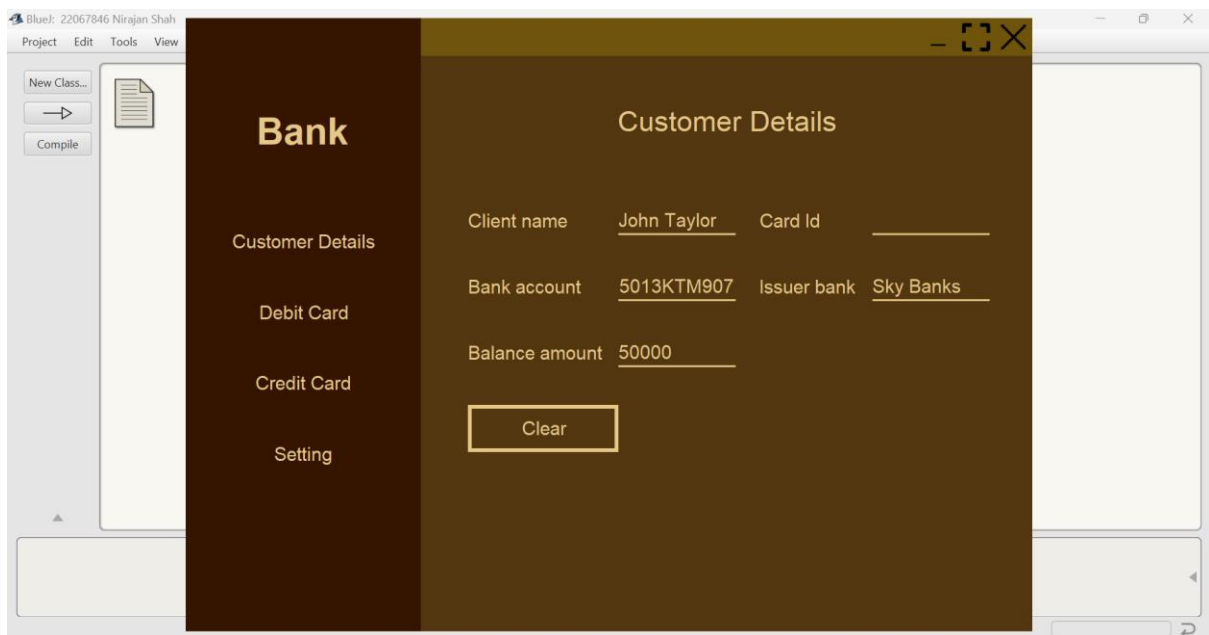


Figure 19: Screenshot after clearing out the Card Id text field value

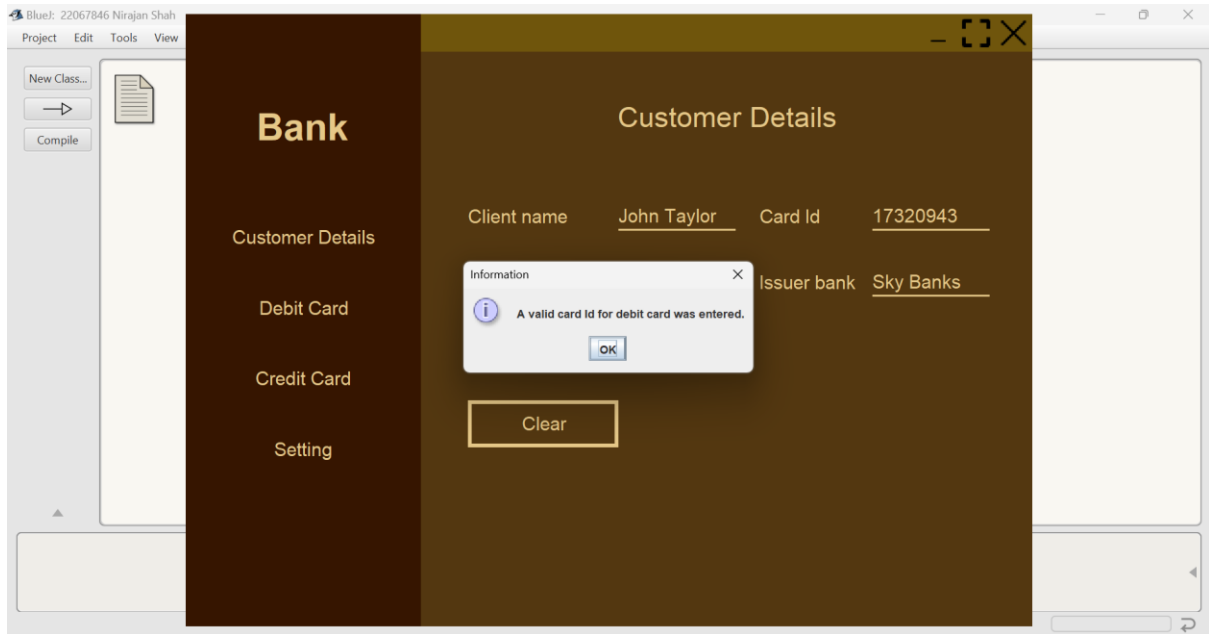


Figure 20: Screenshot after re-entering the value of card id

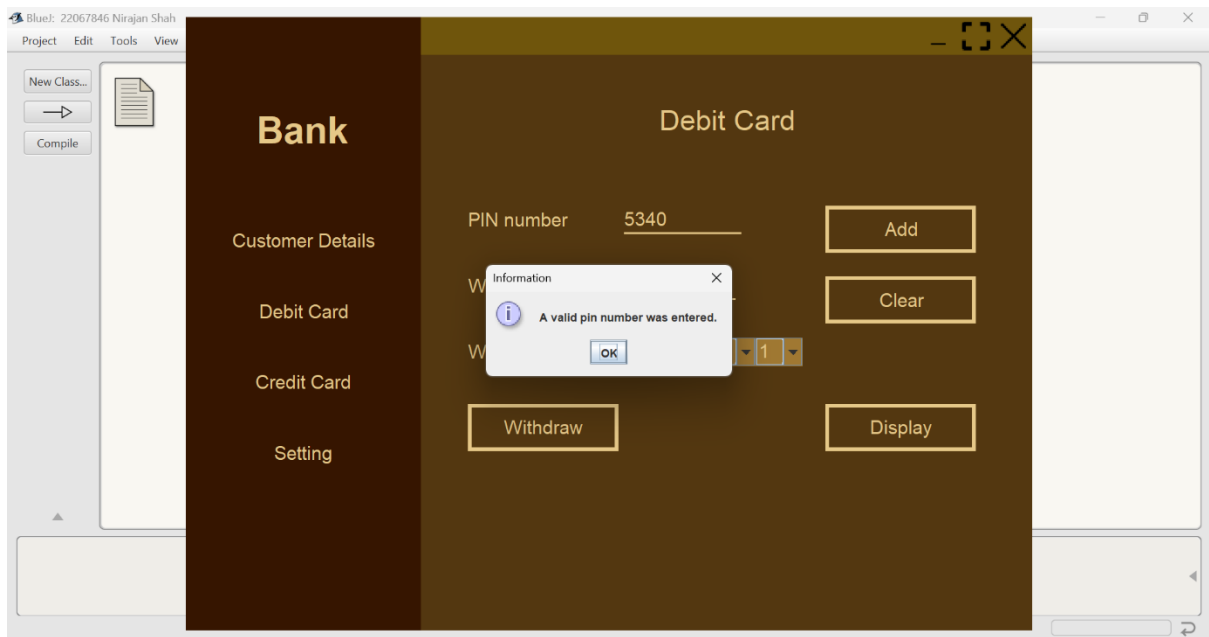


Figure 21: Screenshot after re-entering the value of pin number

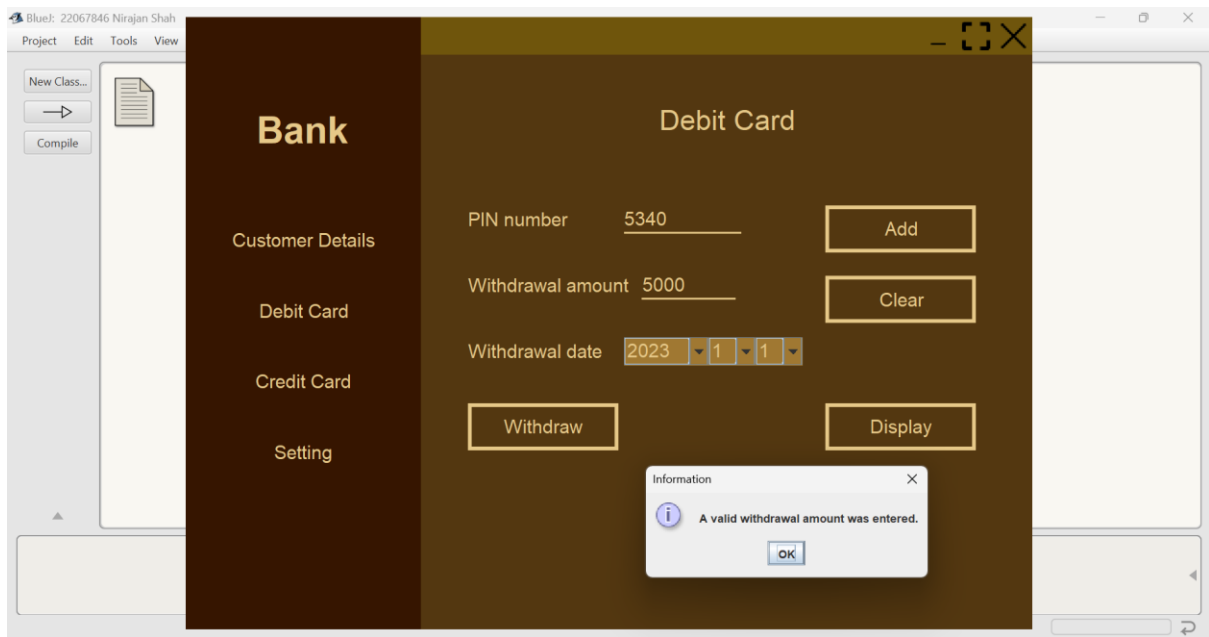


Figure 22: Screenshot after entering a value for withdrawal amount and pressing enter

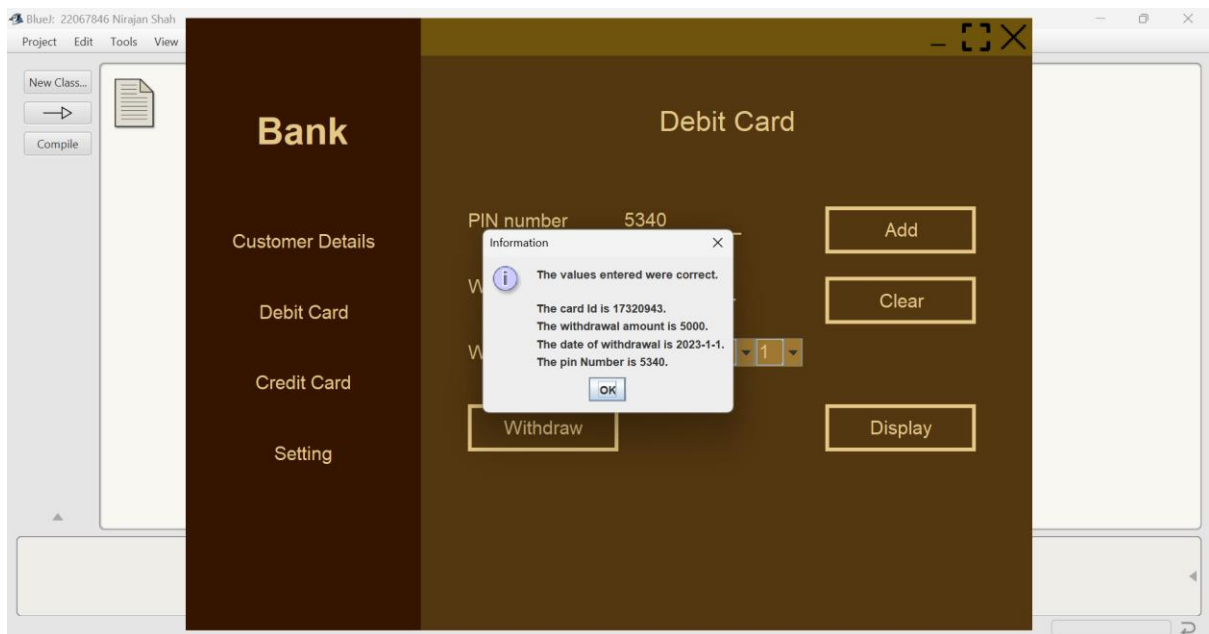


Figure 23: Screenshot after clicking on OK on the dialog box

5.2.5 Test case 5

Test case:	5
Objective:	To check if a certain amount of credit limit would be set for an instance of credit card from the GUI
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of debit card Balance amount = 50000 Card id = 54432934 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Cvc number = 890 interestRate= 16.0 b. The Add button is clicked c. A certain amount of credit limit is entered into the text field along with date of withdrawal. Credit Limit = 100000 Date of withdrawal = 2023/08/02 d. The set limit button is clicked.
Expected Result:	A certain amount of credit limit would be set for an added instance of credit card.
Actual Result:	A certain amount of credit limit would was set for an added instance of credit card.
Conclusion:	The test case is successful.

Table 7: To check if a certain amount of credit limit would be set for an instance of credit card from the GUI

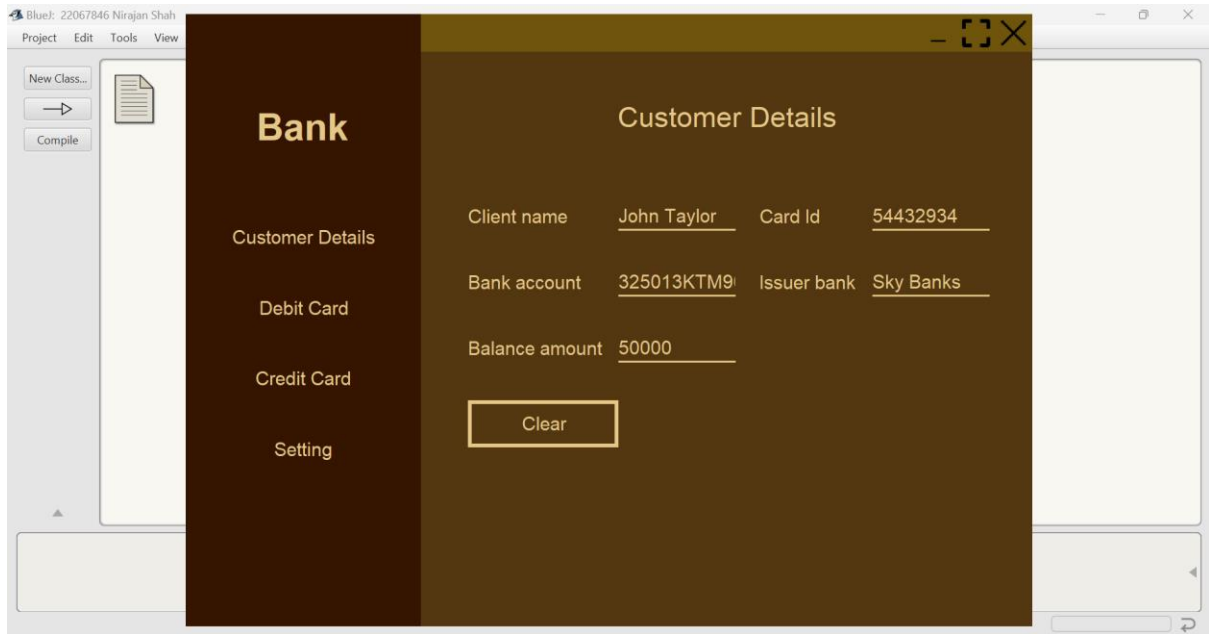


Figure 24: Screenshot displaying the entered text field values

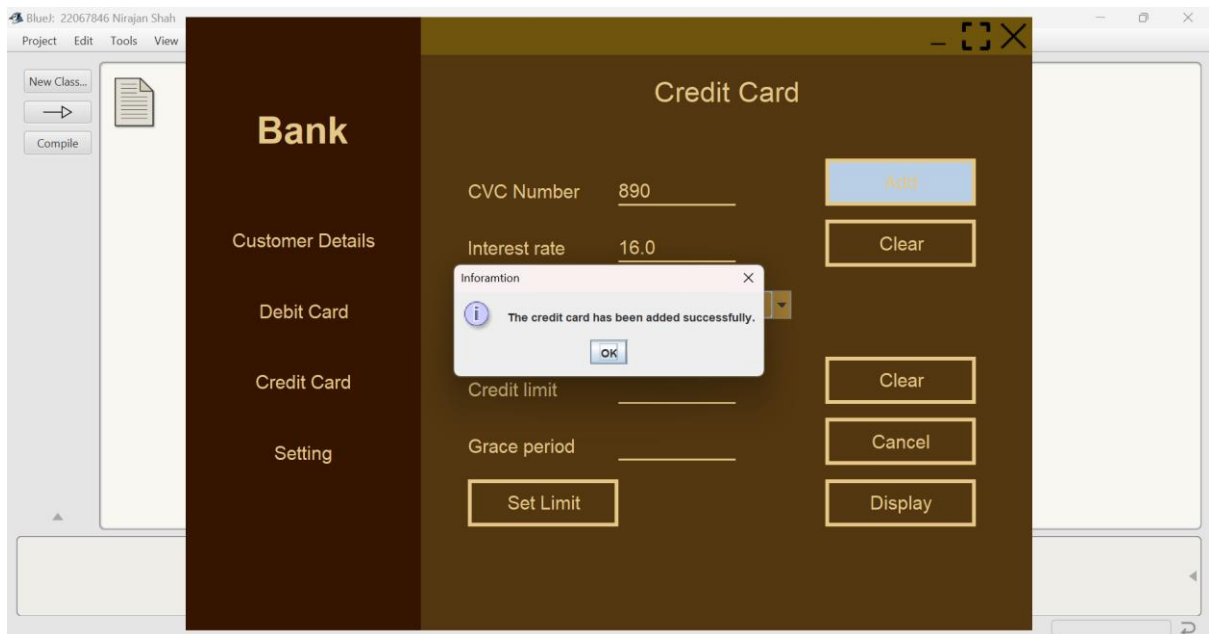


Figure 25: Screenshot after clicking on add button

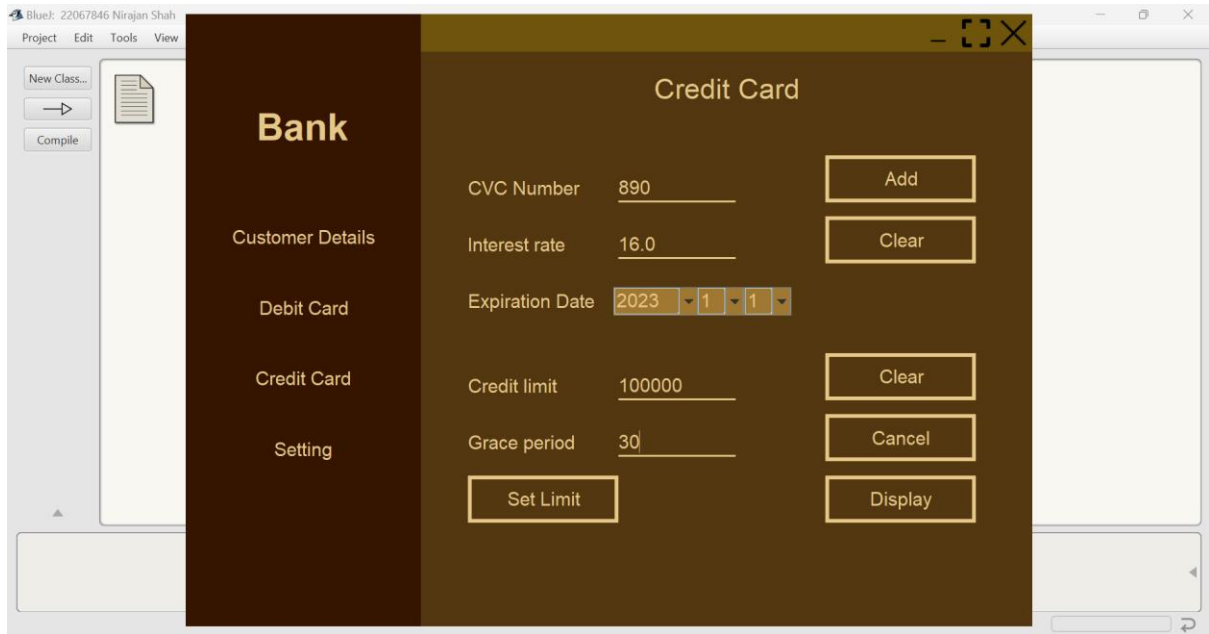


Figure 26: Screenshot after entering the values for credit limit and grace period

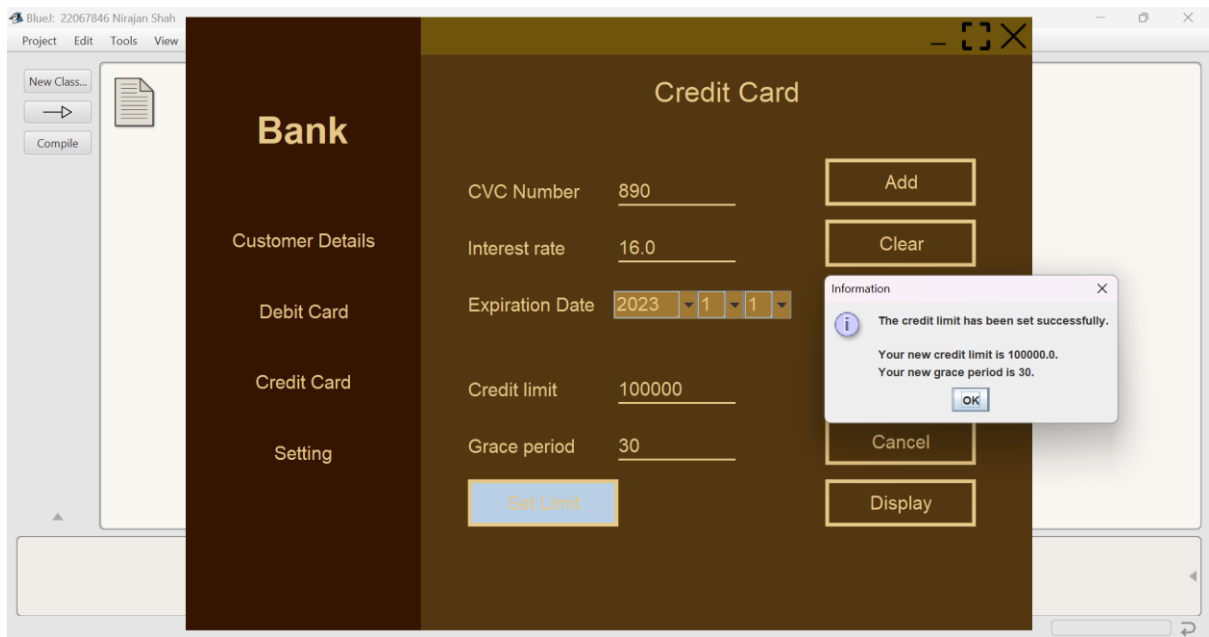


Figure 27: Screenshot after clicking on Set Limit button

5.2.5 Test case 5

Test case:	5
Objective:	To check if credit limit would be cancelled for an instance of credit card from the GUI
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of credit card Balance amount = 55000 Card id = 39574394 Bank account = 203040KTM907 Issuer bank = Hill Banks Client name = Thomas Anderson Cvc number = 900 interestRate= 18.0 b. The add button is clicked. c. A certain amount of credit limit is entered into the text field. Credit Limit = 110000 Date of withdrawal = 2023/08/07 d. The Set Limit button is clicked. e. The cancel button is clicked.
Expected Result:	The credit limit would be cancelled.
Actual Result:	The credit limit was cancelled.
Conclusion:	The test case is successful.

Table 8: To check if credit limit would be cancelled for an instance of credit card from the GUI

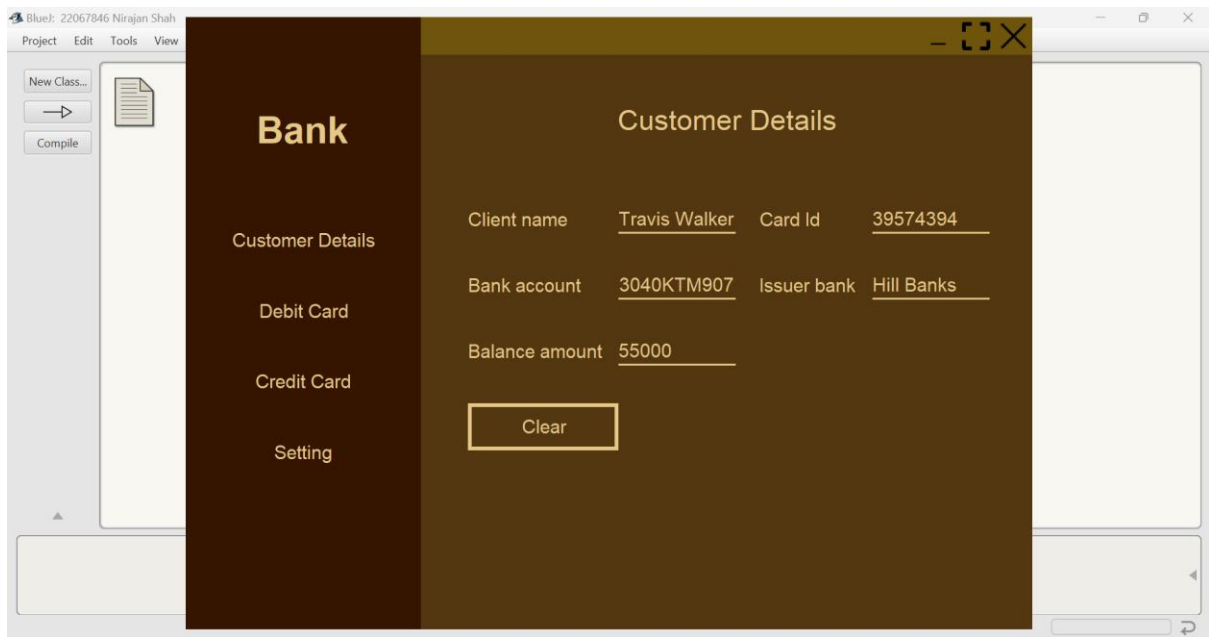


Figure 28: Screenshot displaying the entered text field values

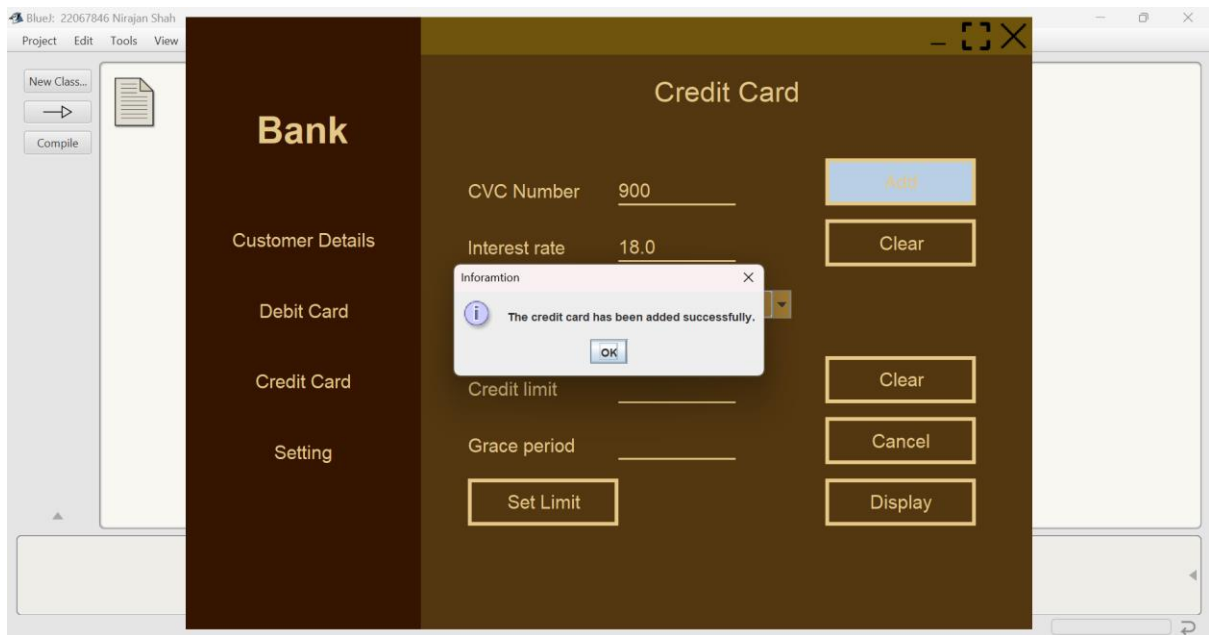


Figure 29: Screenshot after clicking on the add button

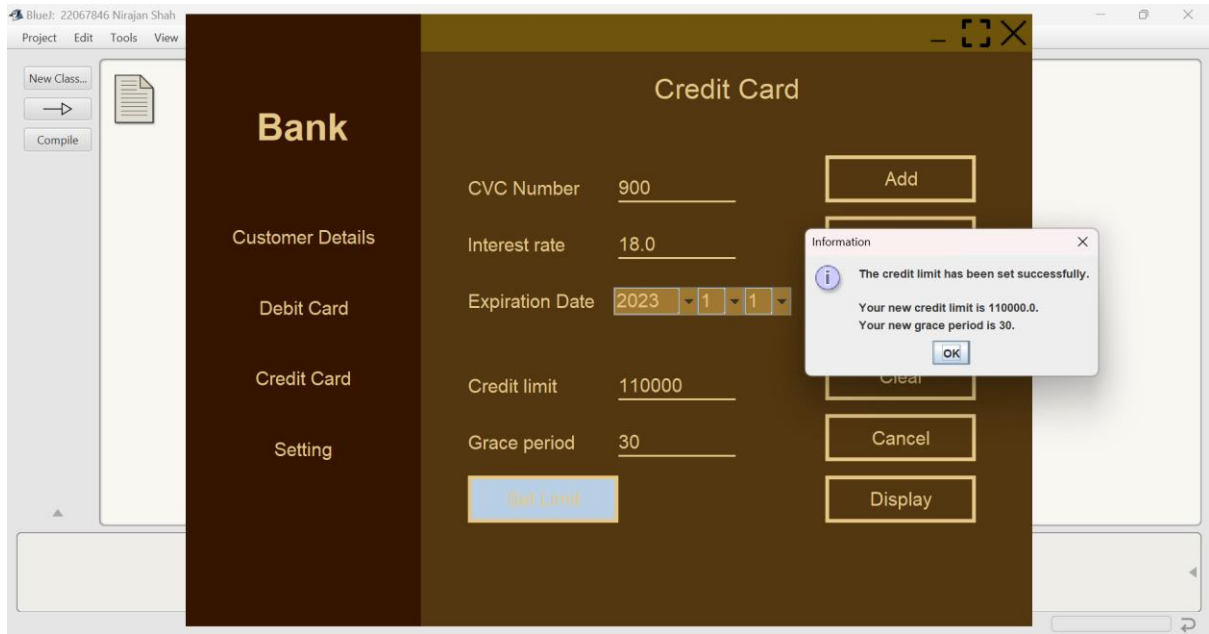


Figure 30: Screenshot after clicking on the Set Limit button after entering values for credit limit and grace period

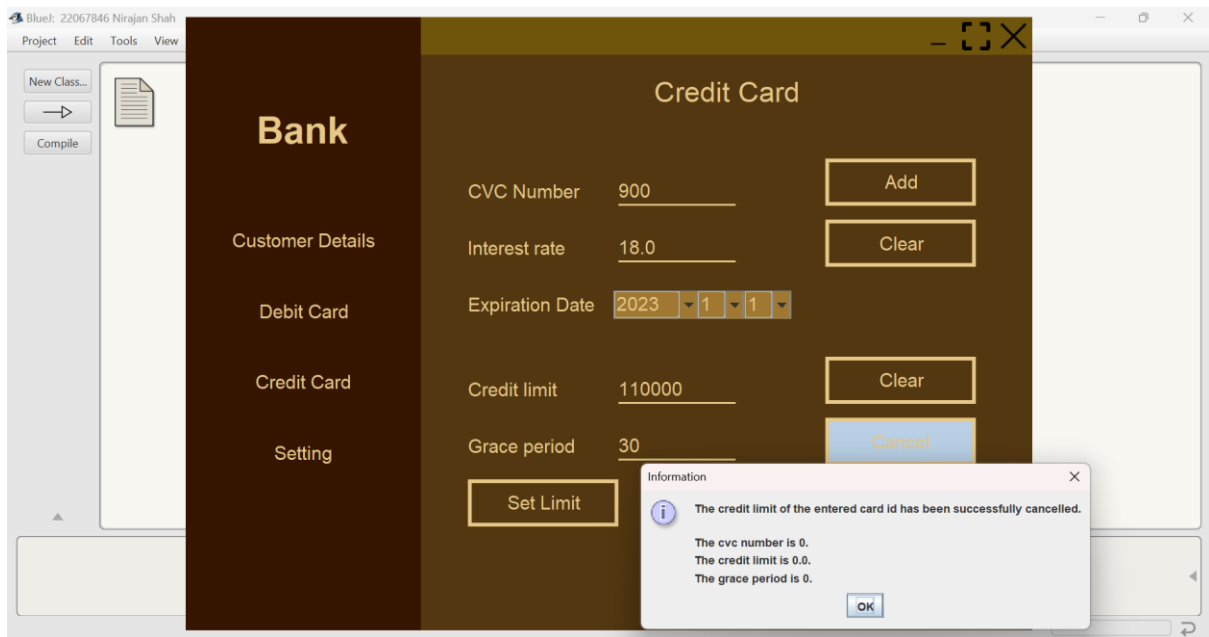


Figure 31: Screenshot after clicking on the Cancel button

5.3 Test 3

5.3.1 Test case 1

Test case:	1
Objective:	To check if a dialog box informing that duplicate entry cannot be added upon adding a credit card entry twice.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of credit card Balance amount = 50000 Card id = 54432934 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Cvc number = 890 interestRate= 16.0 b. The add button is clicked. c. The add button is clicked.
Expected Result:	A dialog box informing that an entry cannot be added twice would appear.
Actual Result:	A dialog box informing that an entry cannot be added twice appeared.
Conclusion:	The test case is successful.

Table 9: To check if a dialog box informing about a duplicate would appear upon adding a credit card entry twice.

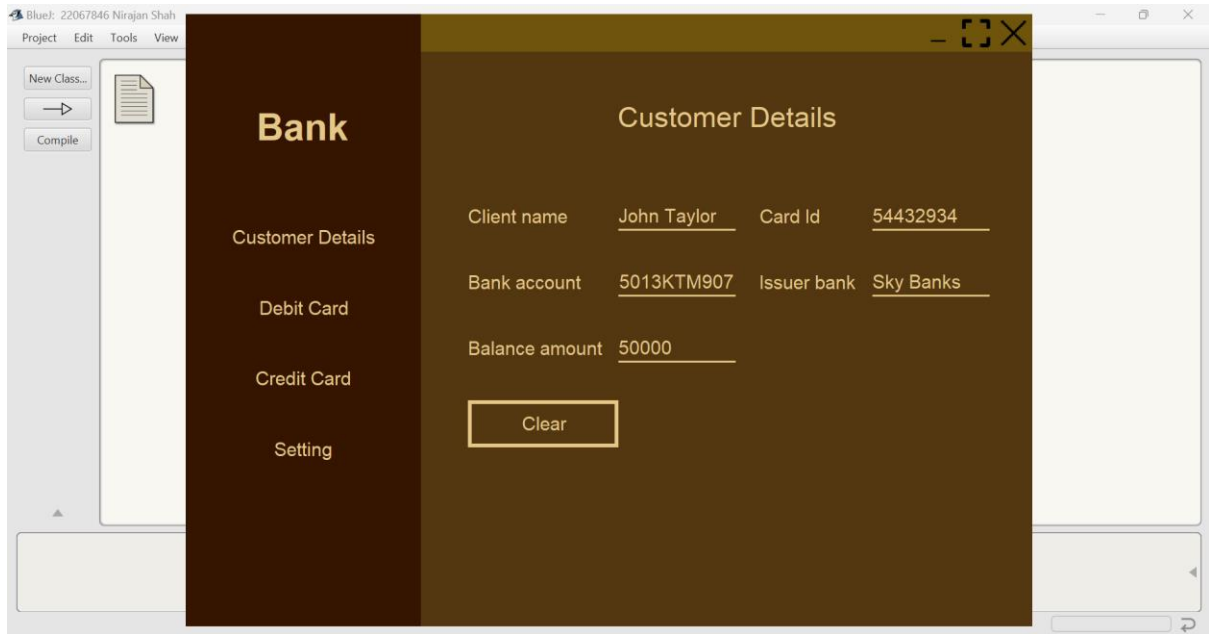


Figure 32: Screenshot displaying the entered text field values

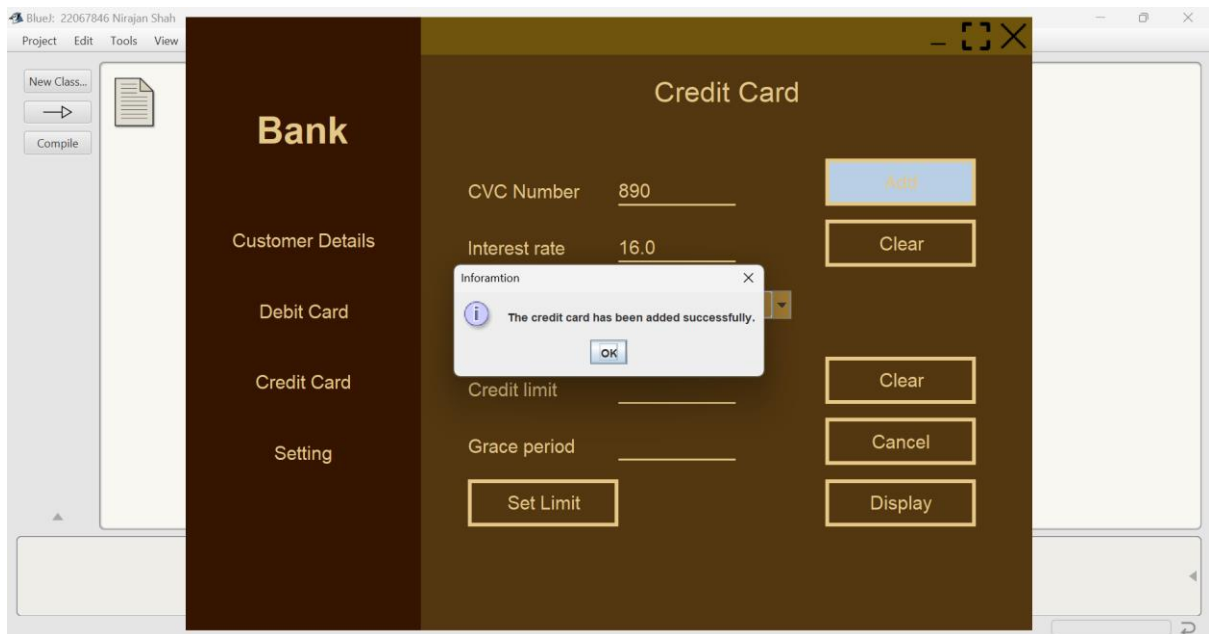


Figure 33: Screenshot after clicking on the add button

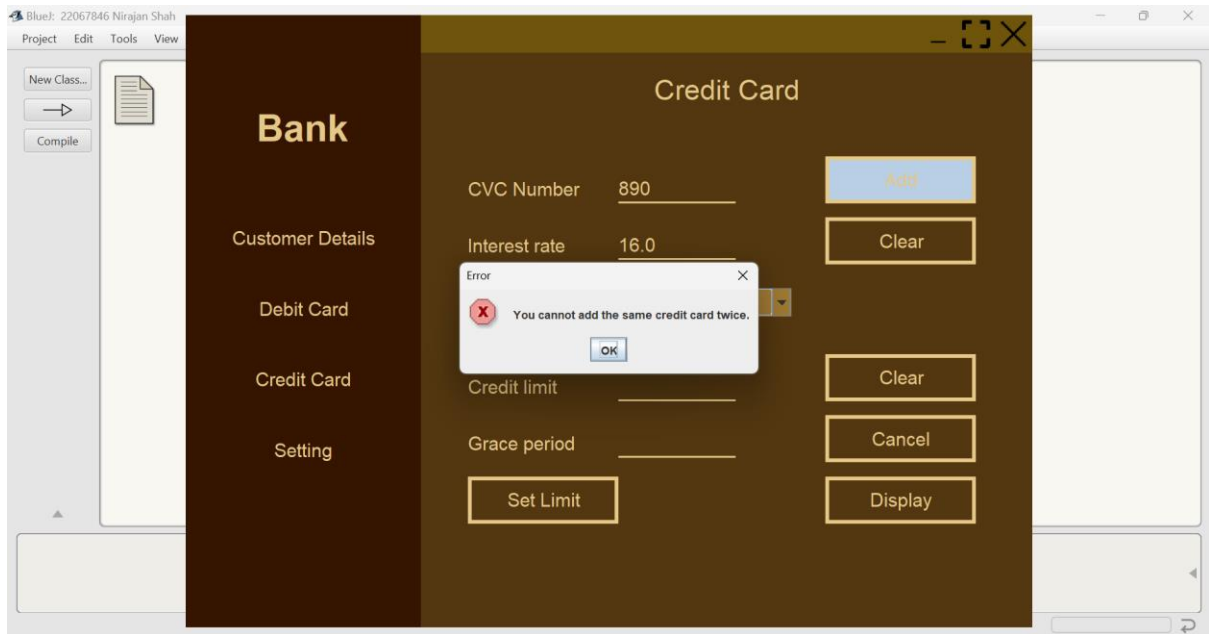


Figure 34: Screenshot after clicking on add button

5.3.2 Test case 2

Test case:	2
Objective:	To check if a dialog box informing that an invalid value for card Id was entered.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of debit card except for card id for which an incorrect value is entered. Balance amount = 50000 Card id = Test Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Pin number = 5340 b. The add button is clicked.
Expected Result:	A dialog box informing that an invalid input was entered would appear.
Actual Result:	A dialog box informing that an invalid input was entered appeared.
Conclusion:	The test case is successful.

Table 10: To check if a dialog box informing that an invalid value for card Id was entered.

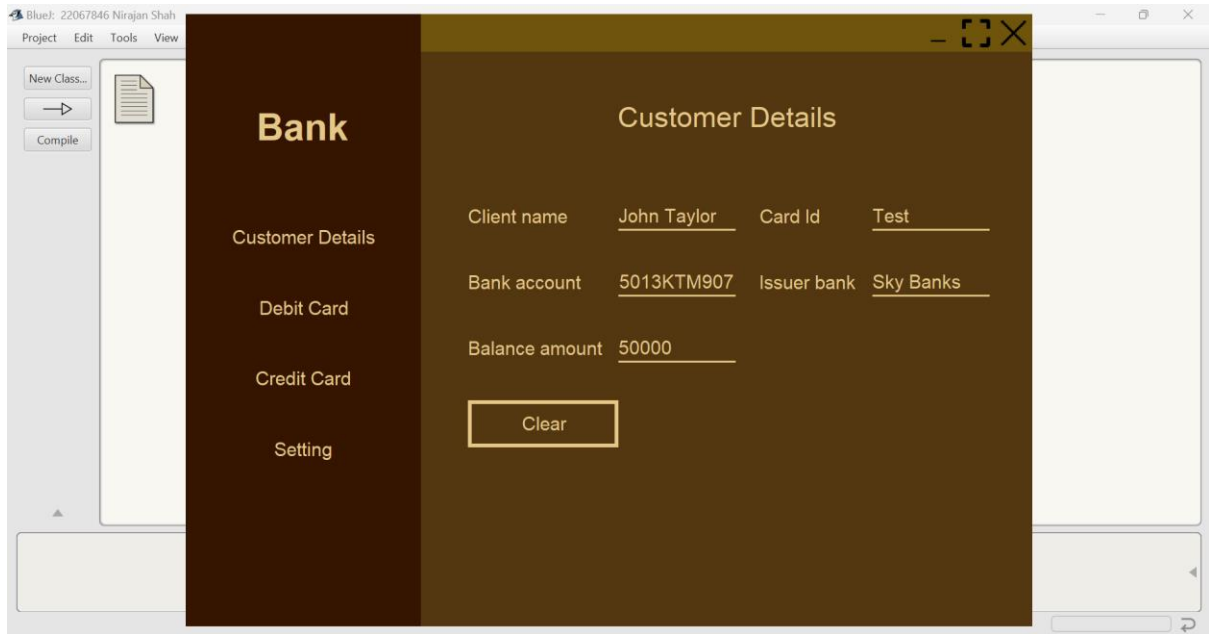


Figure 35: Screenshot displaying the entered text field values

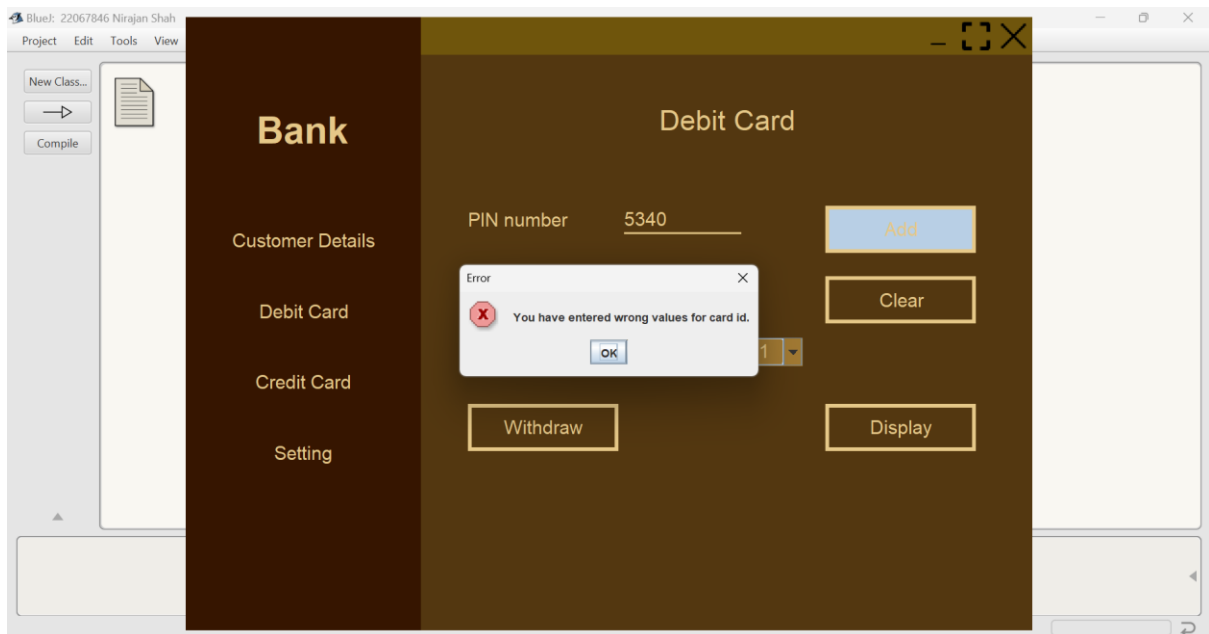


Figure 36: Screenshot after clicking on the add button

5.3.3 Test case 3

Test case:	3
Objective:	To check if a dialog box informing that an empty value for card Id was entered would appear.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of debit card except for card id for which the value is an empty string. Balance amount = 50000 Card id = Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Pin number = 5340 b. The add button is clicked.
Expected Result:	A dialog box informing that the value for card id was empty would appear.
Actual Result:	A dialog box informing that the value for card id was empty appeared.
Conclusion:	The test case is successful.

Table 11: To check if a dialog box informing that an empty value for card Id was entered would appear

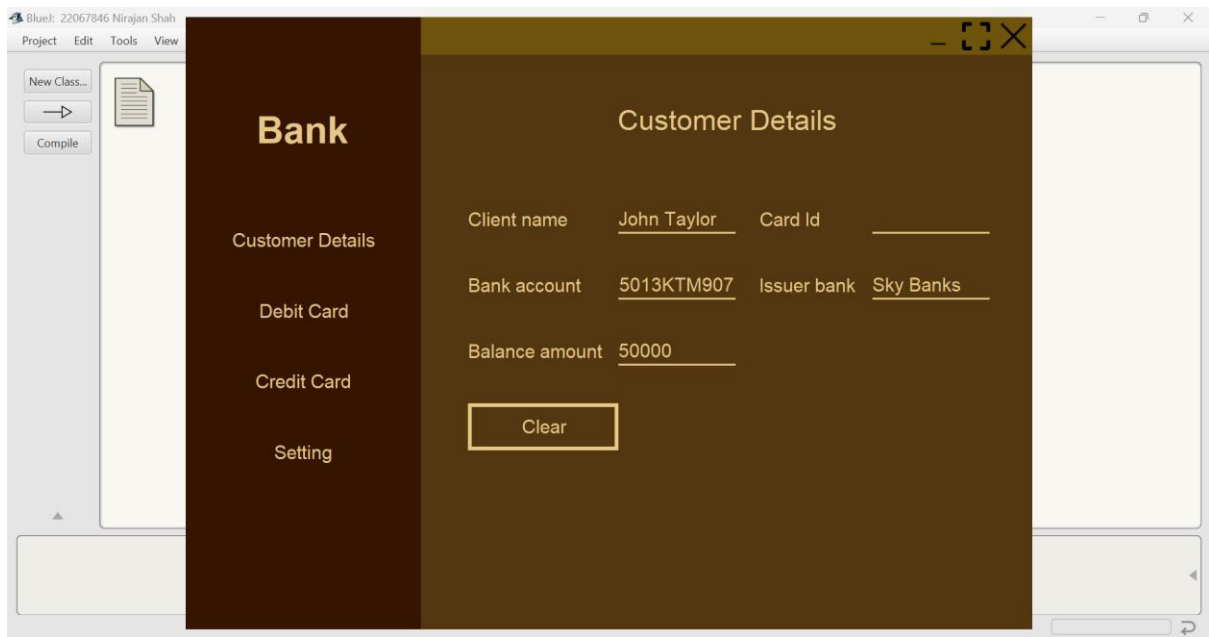


Figure 37: Screenshot displaying the entered text field values

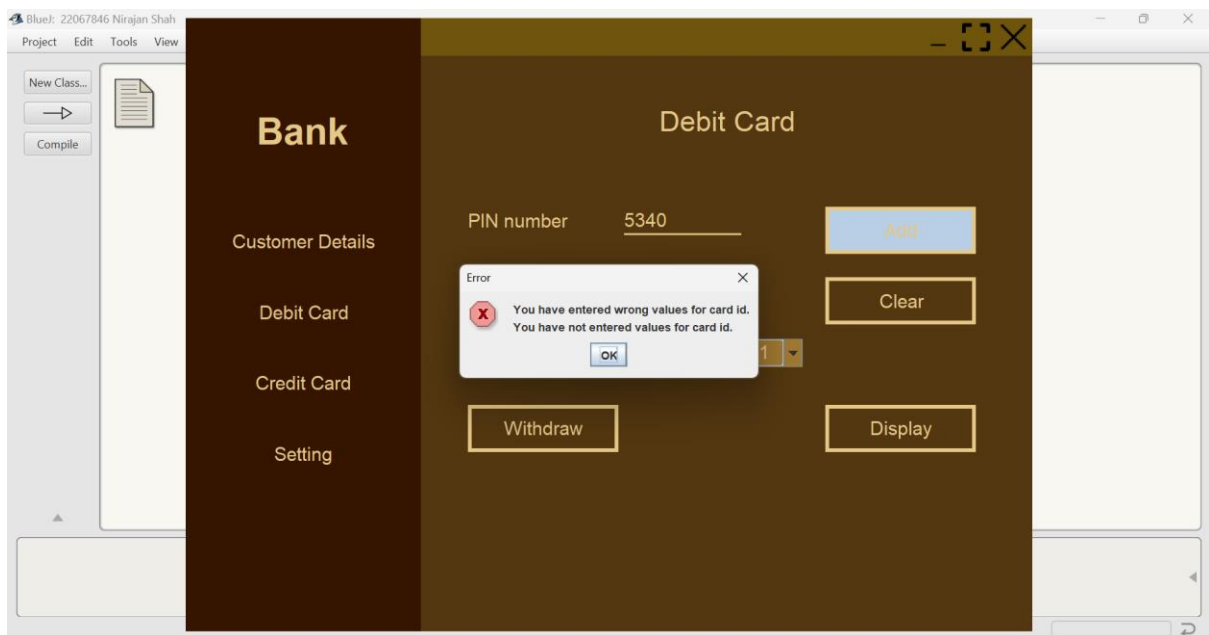


Figure 38: Screenshot after clicking on the add button

5.3.4 Test case 4

Test case:	4
Objective:	To check if a dialog box informing that the database is empty when withdraw button is clicked before adding any objects to array list.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of debit card. Balance amount = 50000 Card id = 54432934 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Pin number = 5340 b. A value for withdrawal amount is entered. Withdrawal amount = 5000 c. The withdraw button is clicked.
Expected Result:	A dialog box informing that there are no entries in the database would appear.
Actual Result:	A dialog box informing that there are no entries in the database appeared.
Conclusion:	The test case is successful.

Table 12: To check if a dialog box would appear informing about empty database when withdraw button is clicked before adding any entries into the array list

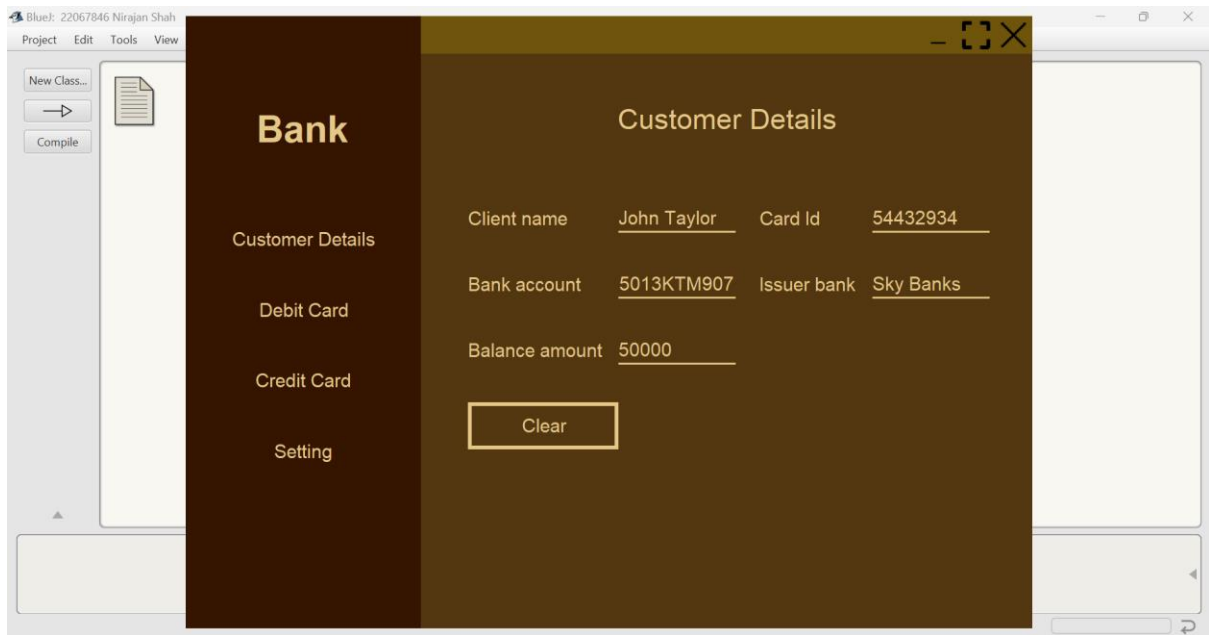


Figure 39: Screenshot displaying the entered text field values.

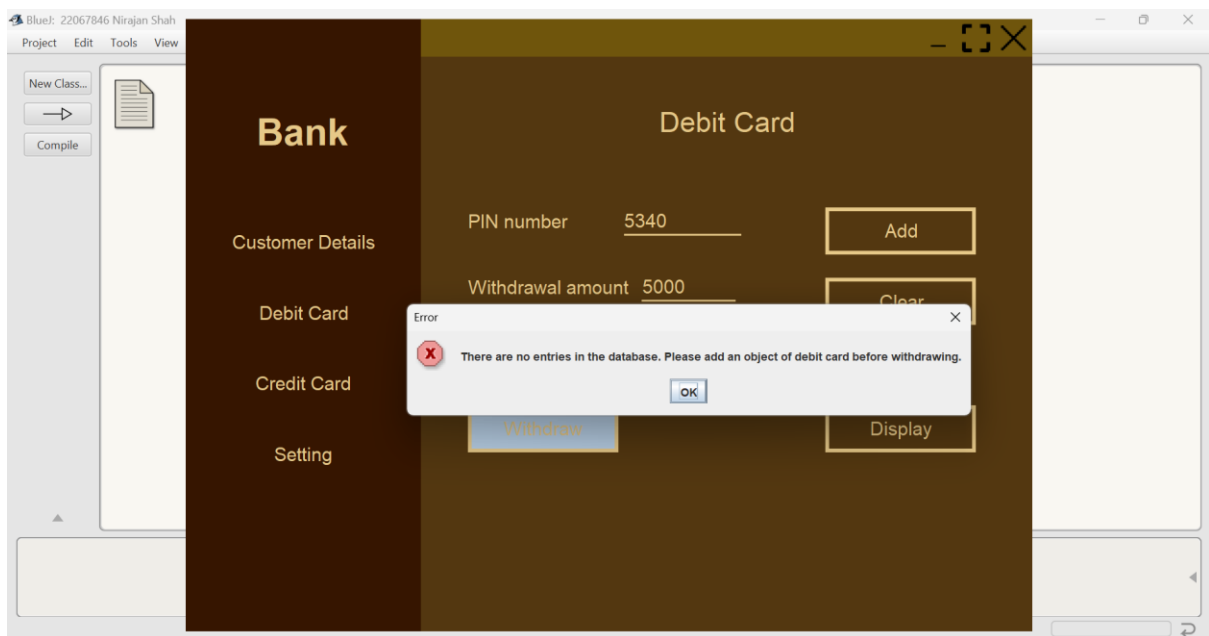


Figure 40: Screenshot after clicking on the Withdraw button

5.3.5 Test case 5

Test case:	5
Objective:	To check if a dialog box informing that withdrawal was not successful would appear when the withdrawal amount entered is more than the balance amount of a given card id.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of debit card. Balance amount = 50000 Card id = 54432934 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Pin number = 5340 b. The add button is clicked. c. A value greater than balance amount for withdrawal amount is entered. Withdrawal amount = 100000 d. The withdraw button is clicked.
Expected Result:	A dialog box informing that withdrawal was not successful would appear.
Actual Result:	A dialog box informing that withdrawal was not successful appeared.
Conclusion:	The test case is successful.

Table 13: To check if a dialog box informing that withdrawal was not successful would appear

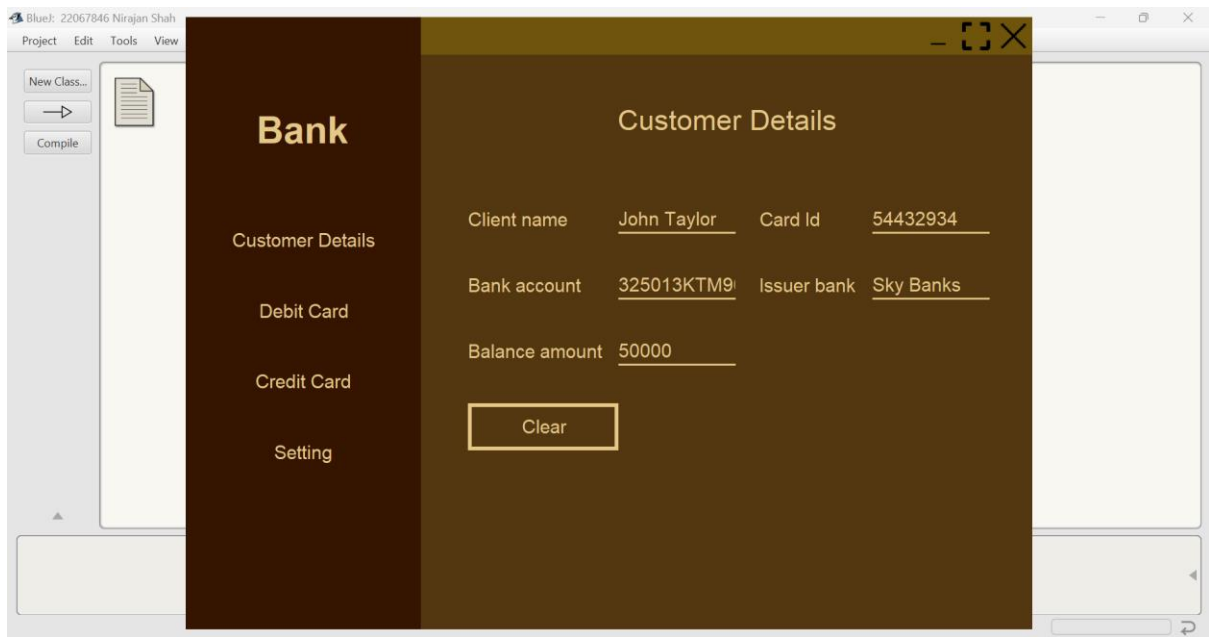


Figure 41: Screenshot displaying the entered text field values

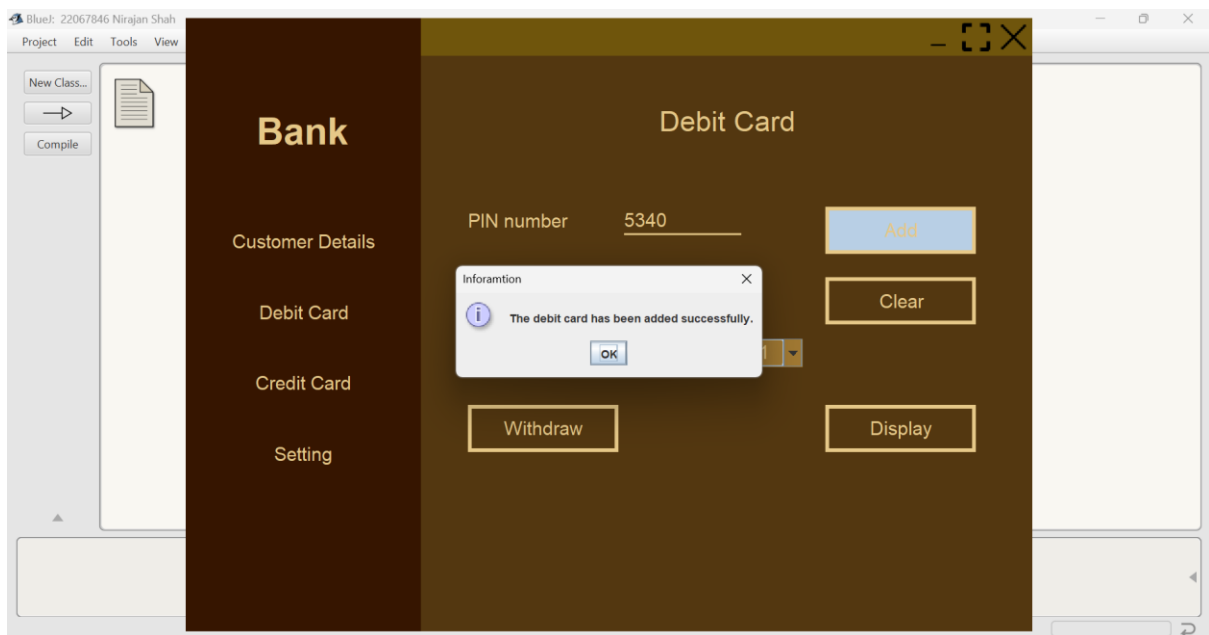


Figure 42: Screenshot after clicking on the add button

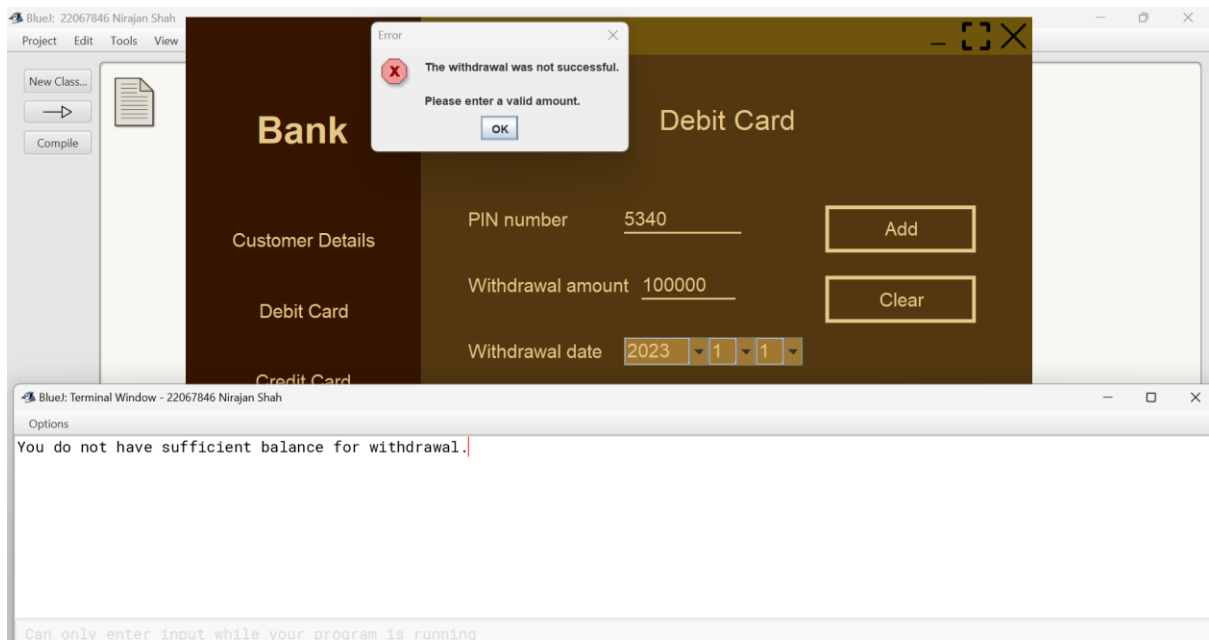


Figure 43: Screenshot after clicking on the withdraw button

5.3.5 Test case 5

Test case:	5
Objective:	To check if a dialog box informing that the credit limit could not be set when entering a balance amount which is 2.5 times greater than the balance amount would appear.
Action:	a. Appropriate values are entered into the text fields required for initializing an instance of credit card. Balance amount = 50000 Card id = 54432934 Bank account = 325013KTM907 Issuer bank = Sky Banks Client name = John Taylor Cvc number = 890 interestRate= 16.0 b. The add button is clicked. c. A value that is 2.5 times greater than balance amount is entered for credit limit and an appropriate value is entered for grace period. Credit limit = 300000 Grace period = 30 d. The Set Limit button is clicked.
Expected Result:	A dialog box informing that credit limit could not be set would appear.
Actual Result:	A dialog box informing that credit limit could not be set appeared.
Conclusion:	The test case is successful.

Table 14: To check if a dialog box would appear when an invalid amount of credit limit is attempted to be set for a given card id

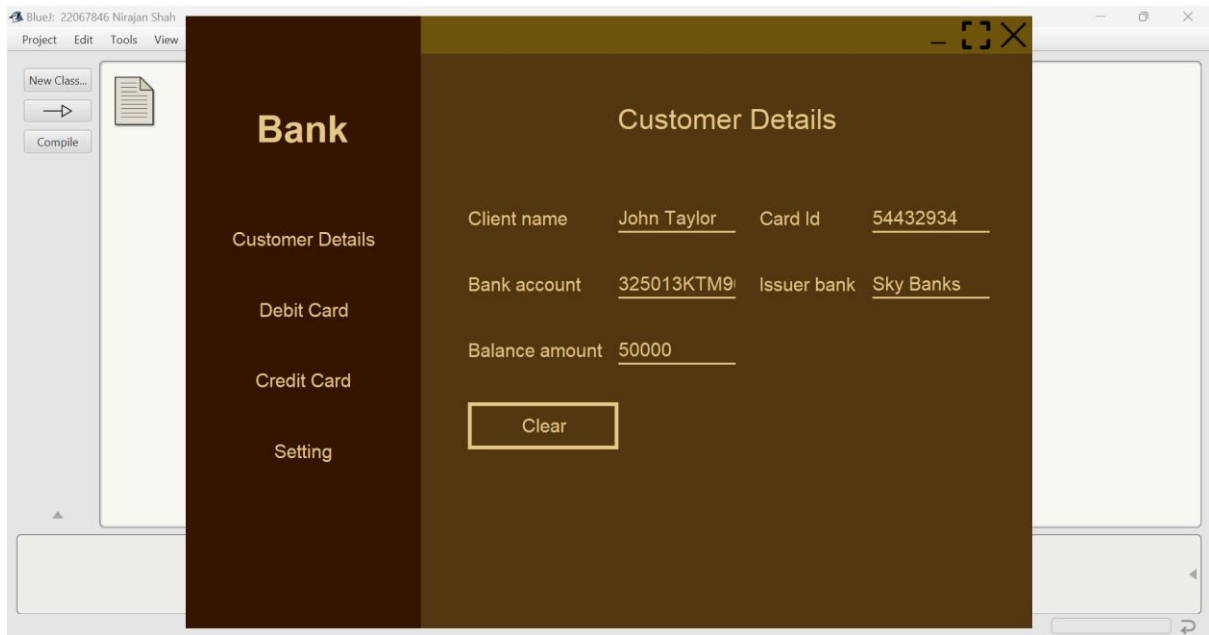


Figure 44: Screenshot displaying the entered text field values

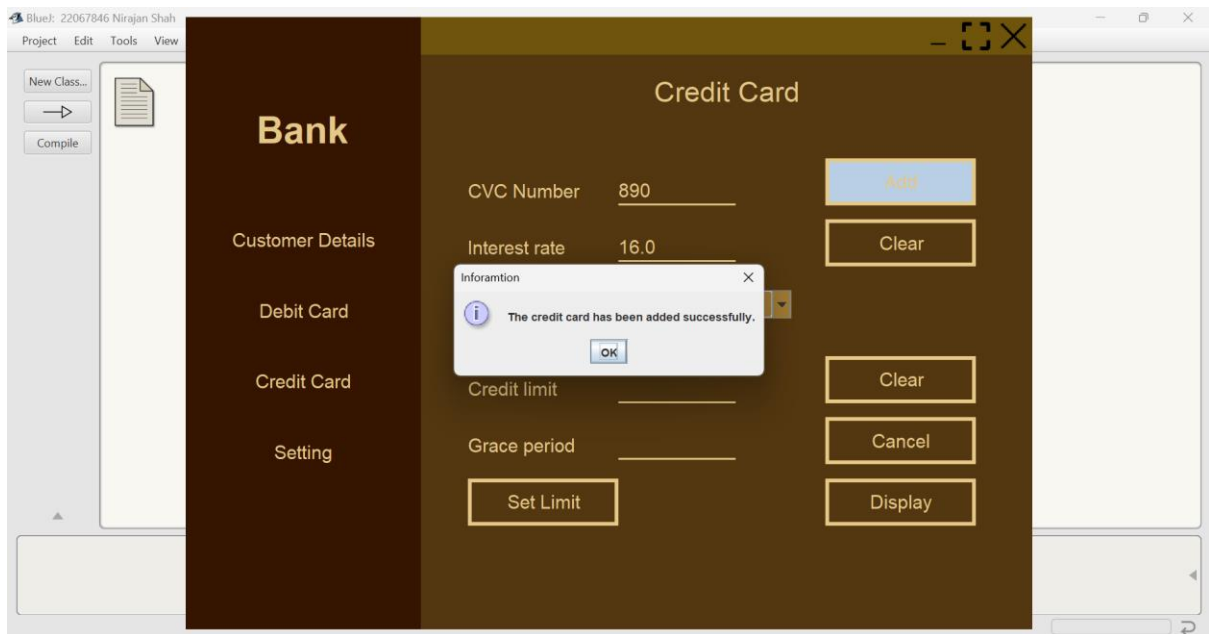


Figure 45: Screenshot after clicking on the add button

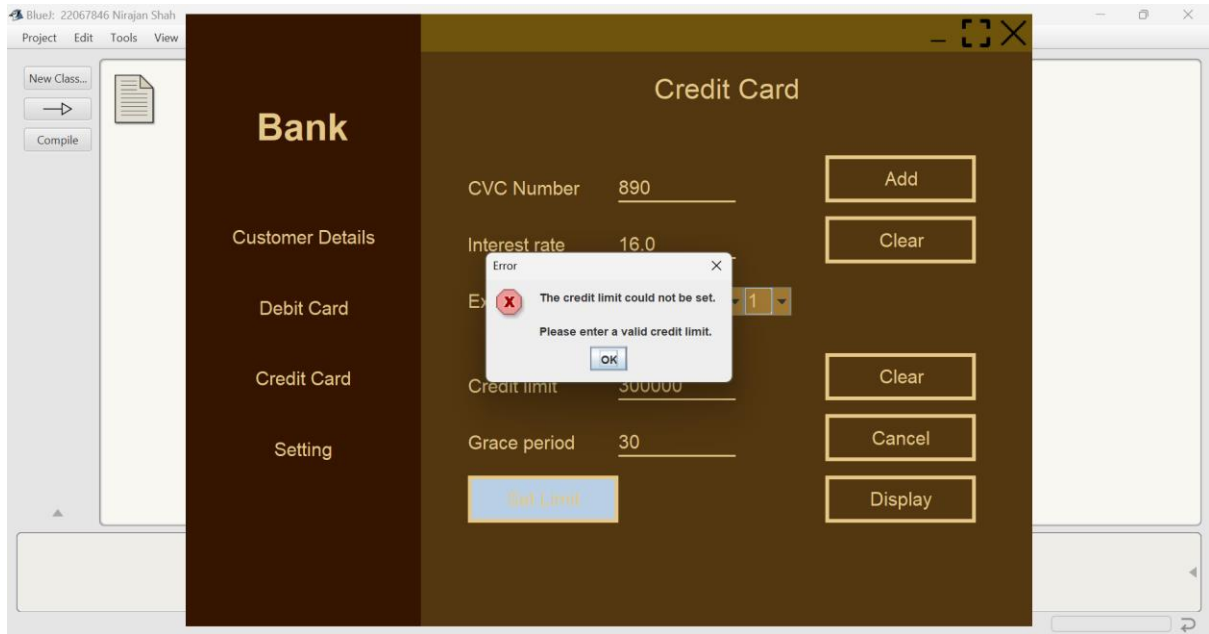


Figure 46: Screenshot after clicking on the Set Limit button

Error detection and correction

Any compilation error encountered during the coding phase of the project was known through compilation error messages which is shown on the line of code itself where the error is present after compiling. Compilation error messages give a detailed report of the error made by the programmer (Becker, 2016).

6.1 Syntax error

Syntax error is a type of error which arises when the keywords are not typed with correct spelling or when certain punctuations are missed. For example, missing ';' (semicolon) at the end of the line or missing quotation marks. Detecting and correcting syntax errors is comparatively easy since most of them are easily identified (Hristova, et al., 2003).

Name of error	Missing ; in the BankGUI class Compiler error message: ';' expected
Type	Syntax error
Information	There was a missing semicolon at the end of the line.
Correction	A semicolon was added at the end of the line.

Table 15: Syntax error

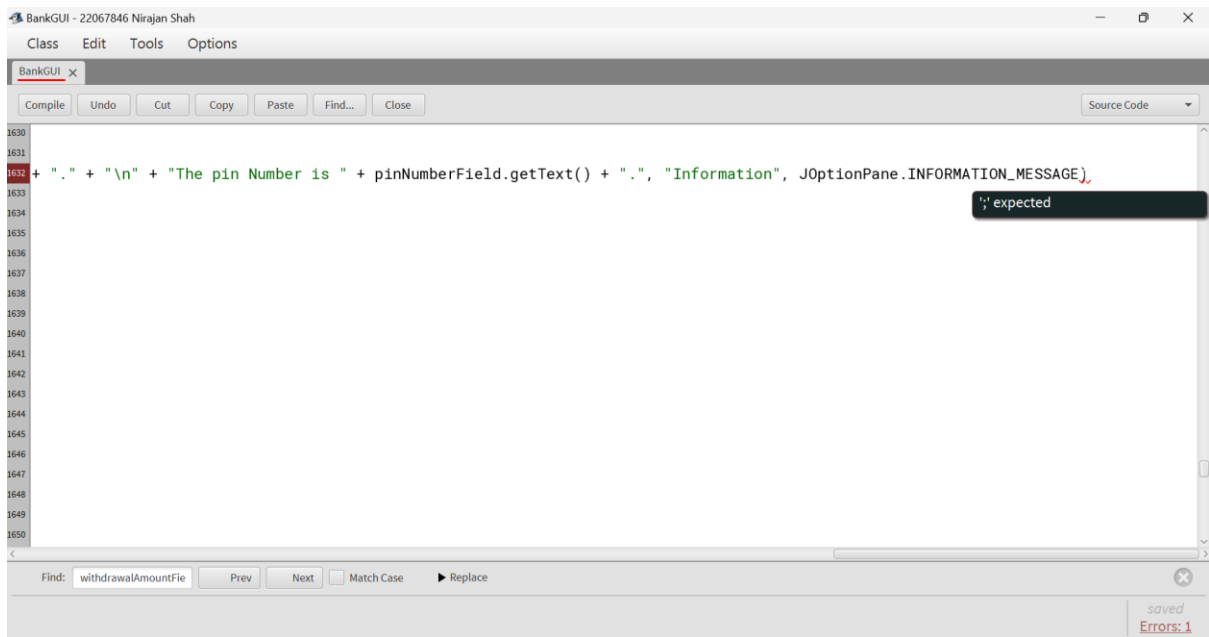


Figure 47: Screenshot of the syntax error

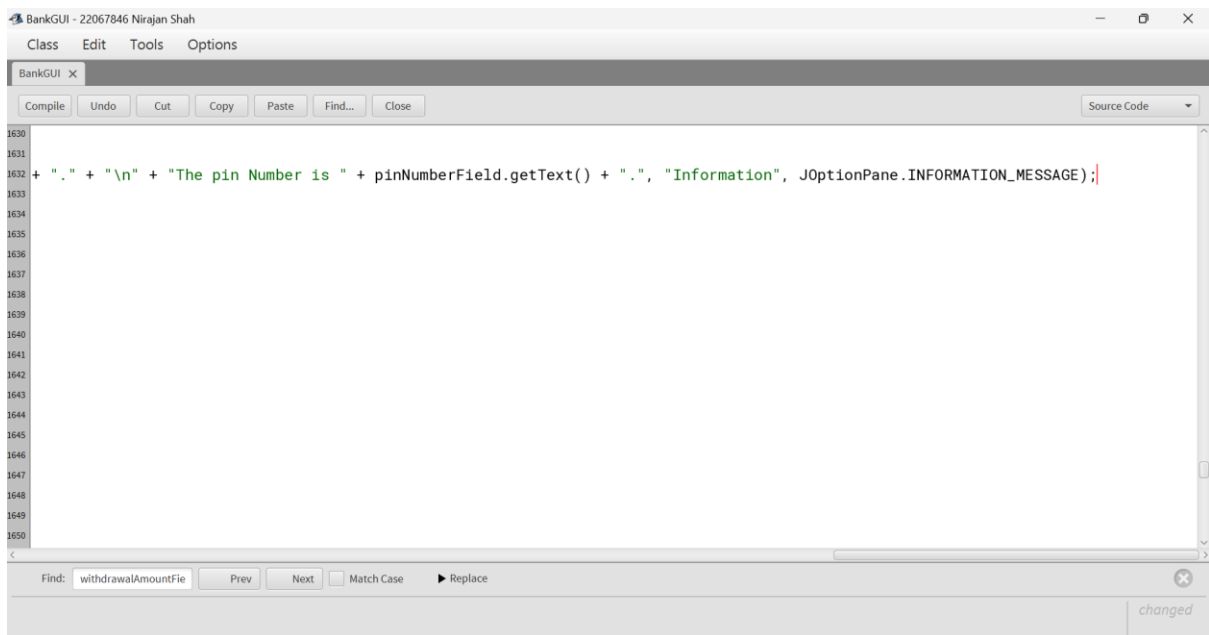


Figure 48: Screenshot of the fix

6.2 Semantic error

Semantic errors are those errors which arise from the incorrect understanding of the programmer on how a compiler would process the said programmer's set of instructions (Hristova, et al., 2003).

Name of error	Trying to store values of text field without the getText() method Compiler error message: "incompatible types: javax.swing.JTextField cannot be converted to java.lang.String"
Type	Semantic error
Information	An attempt to get the text field value was made without using the getText() method on cardIdField i.e., a JTextField inside withdrawal amount field button.
Correction	getText() method was used to get the text field value from the card id.

Table 16: Semantic error

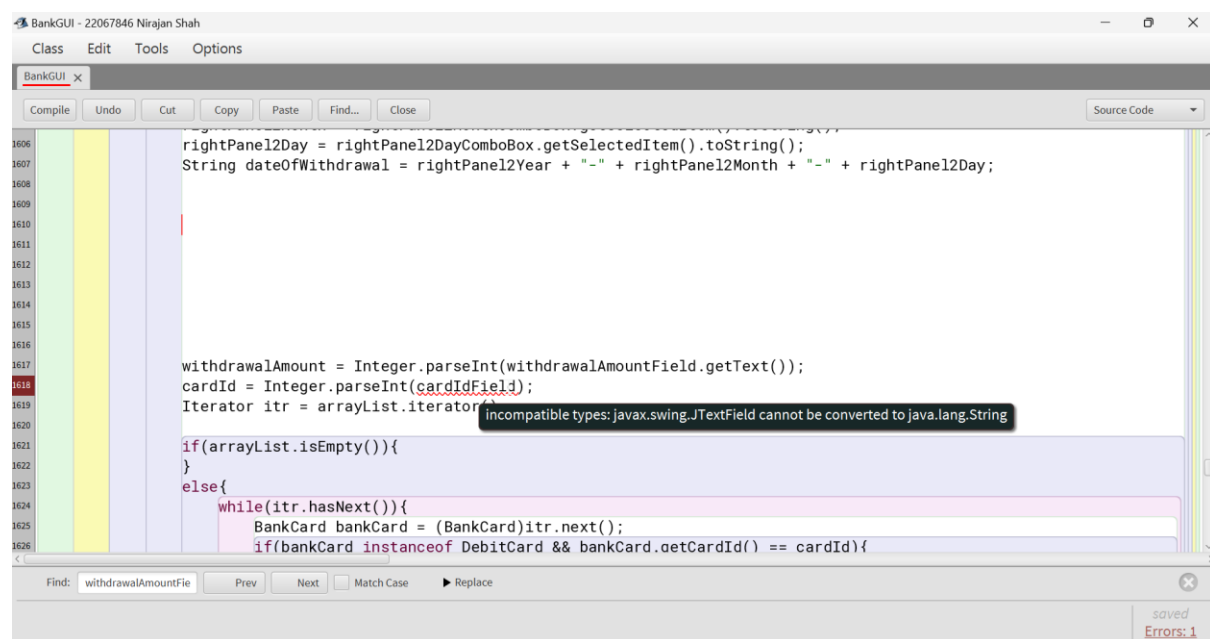


Figure 49: Screenshot of the semantic error

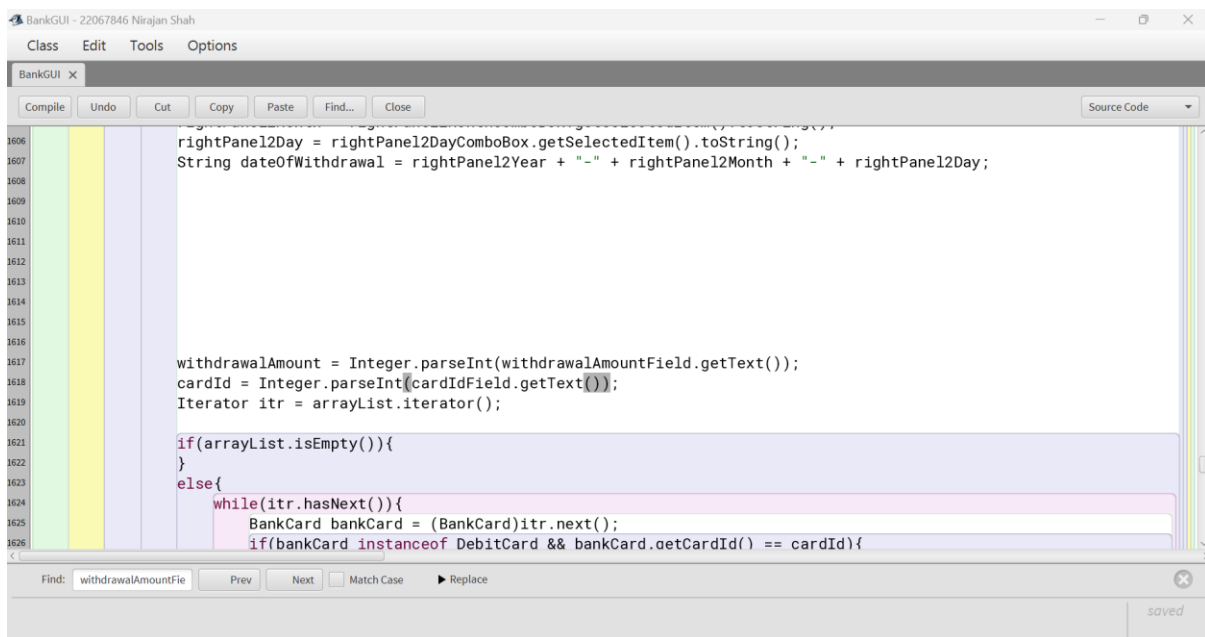


Figure 50: Screenshot of the fix

6.3 Logical error

Some of the logical errors is not actually detected by compiler as these types of error arise generally out of erroneous logical thought process of the programmer translating into actual working source code (Hristova, et al., 2003). The program can be executed but the result of the test case is always false with these types of errors.

Name of error	Opposite initialization of the variable catchCardId Compiler error message: No compiler error message
Type	Logical error
Information	The boolean value catchCardId was being initialized to false even when exception had occurred when trying to cast cardId as integer.

Correction	The catchCardId was initialized to true instead of false.

Table 17: Logical error

```

779 catch(NumberFormatException exception1){
780     catchBalanceAmount = true;
781     if (balanceAmountField.getText().equals("")){
782         emptyBalanceAmount = true;
783     }
784 }
785 try{
786     cardId = Integer.parseInt(cardIdField.getText());
787 }
788 catch(NumberFormatException exception1){
789     catchCardId = false;
790     if (cardIdField.getText().equals("")){
791         emptyCardId = true;
792     }
793 }
794 try{
795     pinNumber = Integer.parseInt(pinNumberField.getText());
796 }
797 catch(NumberFormatException exception1){
798     catchPinNumber = true;
799 }

```

Figure 51: Screenshot of the logical error

```

779 catch(NumberFormatException exception1){
780     catchBalanceAmount = true;
781     if (balanceAmountField.getText().equals("")){
782         emptyBalanceAmount = true;
783     }
784 }
785 try{
786     cardId = Integer.parseInt(cardIdField.getText());
787 }
788 catch(NumberFormatException exception1){
789     catchCardId = true;
790     if (cardIdField.getText().equals("")){
791         emptyCardId = true;
792     }
793 }
794 try{
795     pinNumber = Integer.parseInt(pinNumberField.getText());
796 }
797 catch(NumberFormatException exception1){
798     catchPinNumber = true;
799 }

```

Figure 52: Screenshot of the fix

Conclusion

7.1 Evaluation

A GUI was developed using the swing library of the java. Some awt classes were imported. However, the foundation of the whole of the project lies on the Swing framework of the Java. The total lines of code for developing this project is around one thousand nine hundred and fifty including comments. Only select few components of the awt package were imported instead of importation of whole of the toolkit so as to follow good programming practise and to reduce wastage of system resources. Although, the difference would be minimal for now, the same may not be said for future big projects. This GUI was made to be as much as attractive as possible so as to keep the user engaged with it. A single panel of BankCard class served as the entry of input values for the first five parameter values of both the DebitCard class and CreditCard class. All the buttons are functional and the program was tested moderately.

7.2 Reflection on acquired knowledge

I was made known about many new knowledge regarding programming such as exception handling and event handling. The program crashing on entering any invalid input was a given to me before. However, using exception handling to direct the flow of program instead of completely stopping it altogether was a new concept to me.

Learning about event handling also made me realize about how frequently this concept is being in our daily lives, yet, we remain oblivious to it. For example, even as I type this sentence, there must be some sort of keyPressed, keyTyped and keyReleased method being implemented somewhere in the OS or a similar concept must have been implemented. Everytime, I compiled classes in Java or called the main method, an ActionEvent must have been generated which may be handled by actionPerformed method somewhere.

One other new knowledge that I may have gained is finally understanding what the main method actually means. I learned about arbitrary number of arguments when I wrote a function that took a dictionary as a parameter with `**kwargs` in python. After learning to split a string on the basis of commas, I believe that `String[] args` means to send an arbitrary number of arguments of String datatype. Of course, this presumption may be wrong but the feeling of delving deeper and understanding more about programming elates me the most.

7.3 Difficulties encountered

If there is one thing that us students can write about the construction of this project without stop for hours are the errors and difficulties encountered. Numerous errors were encountered during the construction of this project.

There were errors that did not have any compile time messages. BlueJ only displayed that there were errors but no line number or link to errors were specified. I had forgotten an equals to mark in the if clause just after the end of the while loop. Strangely, blueJ did not point to the error at all. It did not let me compile either.

Another difficulty that was encountered was to initialize values for variables like `catchCardId`, `emptyCardId` etc inside the `actionPerformed` method but outside the if else clause. I had assumed that `getText()` would run on the text fields in that area each time `actionPerformed` method would run. However, I mistaken and found a workaround by adding the `getText()` inside the if clause sourcing event from either `addDebit` or `addCredit`. I had also assumed that if this if clause were to run even a millisecond after the individual event sourcing clause of either `addDebit` or `addCredit`, the program would crash. However, to my wonder, everything worked out perfectly.

Another difficulty that was difficult to surmount was to display values in the text fields only if a valid card Id(card Id already in the array list) or if a valid pin number(a pin number that is linked to the card id value present in the text field) or if a valid withdrawal amount was entered. Typing in 100 for a card Id with a balance amount of 1000 would prompt a dialog box after each release of the key.

A wall that was difficult to break was to display the values of object of different type but same card id one after the other in the case that their card id matches. Only during the days nearing the deadline was I finally able to solve this problem.

7.4 How I overcame the said difficulties

For the first problem, I added an equals to mark after the if clause just after the while loop. Such a simple solution took a lot of time before implementation.

For the second problem, I added all the lines of code with getText() inside the if else clause with the source being either addDebit or addCredit.

For the third problem, I removed the dialog box and only made the program update the instance variable denoting whether the entered withdrawal amount for that card id was valid or not.

For the final problem, I initialized two different iterators to iterate for each different type.

I have now come to cherish the errors and the hours of debugging that I had to do because, this has made me understand deeper and has made me more learned in regards to programming.

References

- Alexander, R., Bieman, J., Ghosh, S. & Ji, B., 2002. Mutation of Java objects. In: *13th International Symposium on Software Reliability Engineering, 2002. Proceedings..* Annapolis: IEEE, pp. 341-351.
- Arnold, K., Gosling, J. & Holmes, D., 2005. java.security and Related Packages Security Tools. In: K. Coyle, ed. *THE Java™ Programming Language, Fourth Edition*. s.l.:Addison Wesley Professional, p. 585.
- Becker, B. A., 2016. An effective approach to enhancing compiler error messages. In: *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*. s.l.:s.n., pp. 126-131.
- Dale, N. B., Weems, . C. & Headington, M. R., 2003. *Introduction to java and software design: swing update*. s.l.:Jones & Bartlett Learning.
- Hristova, M., Misra, A., Rutter, M. & Mercuri, R., 2003. Identifying and correcting Java programming errors for introductory computer science students. *ACM SIGCSE Bulletin*, 35(1), pp. 153-156.
- Jacobson, I., Booch, G. & Rumbaugh, J., 1999. *The unified software development process..* New Delhi: Pearson Education India.
- JGraph, 2021. *Diagram Software and Flowchart Maker*. [Online] Available at: <https://www.diagrams.net/> [Accessed 17 January 2023].
- Kölling, M., Qui, . B., Patterso, A. & Rosenberg, J., 2003. The BlueJ System and its Pedagogy. *Computer Science Education* , 13(4), pp. 249-268.
- Kulkarni, R. N. & Prasad, P. P. R., 2021. Abstraction of UML Class Diagram from the Input Java Program. *International Journal Of Advanced Networking And Applications*, 12(4), pp. 4644-4649.
- O'Regan, G., 2018. Java Programming Language. In: *The Innovation in Computing Companion*. Cork: Springer, Cham, pp. 171-174.
- Oda, Y. et al., 2015. Learning to generate pseudo-code from source code using statistical machine translation.. In: *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Lincoln: IEEE, pp. 574-584.
- OMG, 2017. *About the Unified Modeling Language Specification Version 2.5.1*. [Online] Available at: <https://www.omg.org/spec/UML#document-metadata> [Accessed 18 January 2023].
- Rumbaugh, J., 2006. ER is UML. In: *Journal of Information Systems Education*. Cupertino: Information Systems and Computing Academic Professionals, pp. 21-26.

Sobocik, S., 2020. *Fintech application development using Java*. [Online]
Available at: <https://www.future-processing.com/blog/fintech-applications-development-using-java/>
[Accessed 17 January 2023].

Sridhara, G. et al., 2010. Towards automatically generating summary comments for java methods. In: *Proceedings of the IEEE/ACM international conference on Automated software engineering*. s.l.:s.n., pp. 43-52.

Stack Overflow, 2019. *Stack Overflow Developer Survey 2019*. [Online]
Available at: <https://insights.stackoverflow.com/survey/2019#technology>
[Accessed 18 January 2023].

Appendix

Appendix A: Code

Appendix A.1: BankGUI

```
import javax.swing.*;
import java.awt.Font;
import java.awt.Color;
import java.awt.event.*;
import java.awt.FlowLayout;
import java.awt.Image;
import java.awt.Dimension;
import java.awt.GridLayout;
import java.util.ArrayList;
import java.util.Iterator;

public class BankGUI implements ActionListener, KeyListener{

    private int i, k;

    private ArrayList<BankCard> arrayList = new ArrayList<BankCard>();

    private String rightPanel2Year, rightPanel2Month, rightPanel2Day,
    rightPanel3Year, rightPanel3Month, rightPanel3Day;

    private Timer timer1;

    private JButton minimizeButton, maximizeButton, closeButton, detailsButton,
    debitCardButton, creditCardButton, settingButton, rightPanel1ClearButton;
```

```

private JButton displayDebit, clearDebit, addDebit, withdraw;

private JButton addCredit, clearCredit1, clearCredit2, setCreditLimit, displayCredit,
cancelCredit;

private JFrame loadingFrame, mainFrame ;

private JLabel loadingFrameLabel1, loadingFrameLabel2, leftPanelLabel1,
rightPanel1Header;

private JLabel cardIdLabel, clientNameLabel, issuerBankLabel,
bankAccountLabel, balanceAmountLabel;

private JTextField cardIdField, clientNameField, issuerBankField,
bankAccountField, balanceAmountField;

private JLabel rightPanel2Header, pinNumberLabel, withdrawalAmountLabel,
withdrawalDateLabel;

private JTextField pinNumberField, withdrawalAmountField;

private JLabel rightPanel3Header, cvcNumber, interestRate, expirationDate,
creditLimit, gracePeriod;

private JTextField cvcNumberField, interestRateField, creditLimitField,
gracePeriodField;

private JLabel rightPanel4Header, theme;

private JComboBox rightPanel2YearComboBox, rightPanel2MonthComboBox,
rightPanel2DayComboBox;

private JComboBox rightPanel3YearComboBox, rightPanel3MonthComboBox,
rightPanel3DayComboBox;

private JProgressBar progressBar;

private JPanel topPanel, leftPanel, leftPanelPanel, rightPanel1, rightPanel2,
rightPanel3, rightPanel4, currentPanel;

```

```

private JRadioButton darkMode, lightMode;

private ButtonGroup buttonGroup;

private boolean validCardIdDebit = false;
private boolean validCardIdCredit = false;

private boolean validWithdrawalAmount = false;
private boolean validPinNumber = false;

public BankGUI() {

    loadingFrameLabel2 = new JLabel();
    ImageIcon image = new ImageIcon("Background.jpg");
    Image image1 = image.getImage().getScaledInstance(500, 300,
Image.SCALE_SMOOTH);
    image = new ImageIcon(image1);
    loadingFrameLabel2.setIcon(image);
    loadingFrameLabel2.setBounds(0,0, 500, 300);

    loadingFrame = new JFrame();
    mainFrame = new JFrame();

    progressBar = new JProgressBar(0, 100);
    progressBar.setForeground(new Color(83, 145, 101));
    progressBar.setBounds(0, 288, 500, 13);
    progressBar.setStringPainted(true);

    loadingFrameLabel1 = new JLabel("Loading.");
    loadingFrameLabel1.setBounds(160, 100, 200, 50);

```

```

loadingFrameLabel1.setFont(new Font("Arial", Font.BOLD, 40));
loadingFrameLabel1.setForeground(new Color(83, 145, 101));

this.timer1 = new Timer(40, this);
timer1.start();

loadingFrame.add(progressBar);
loadingFrame.add(loadingFrameLabel1);
loadingFrame.add(loadingFrameLabel2);
loadingFrame.setResizable(false);
loadingFrame.getContentPane().setBackground(new Color(38,58, 41));
loadingFrame.setUndecorated(true);
loadingFrame.setLayout(null);
loadingFrame.setSize(500, 300);
loadingFrame.setVisible(true);
loadingFrame.setDefaultCloseOperation(loadingFrame.EXIT_ON_CLOSE);
loadingFrame.setLocationRelativeTo(null);

String yearArray[] = {"2023", "2024", "2025", "2026", "2027", "2028", "2029",
"2030"};

String monthArray[] = new String[12];
String dayArray[] = new String[31];

for(int i = 1; i <= 12 ; i++){
    monthArray[i-1] = String.valueOf(i); //First iteration: month[0] = 1. Final
iteration: month[11] = 12
}

for(int i = 1; i < 32 ; i++){
    dayArray[i-1] = String.valueOf(i); //First iteration: month[0] = 1. Final
iteration: month[30] = 31

```

```

}

topPanel = new JPanel();
topPanel.setLayout(new FlowLayout(FlowLayout.RIGHT, 0, 0));

ImageIcon minimizeImage = new ImageIcon("Minimize.png");
Image scaledMinimizeImage =
minimizeImage.getImage().getScaledInstance(40, 40, Image.SCALE_SMOOTH);
minimizeImage = new ImageIcon(scaledMinimizeImage);

topPanel.setBounds(250,0,650,40);
topPanel.setBackground(new Color(111, 85, 12));
mainFrame.add(topPanel);

minimizeButton = new JButton();
minimizeButton.setBackground(new Color(111, 85, 12));
minimizeButton.setBorderPainted(false);
minimizeButton.setFocusable(false);
minimizeButton.setIcon(minimizeImage);
minimizeButton.setPreferredSize(new Dimension(40,40));
topPanel.add(minimizeButton);

minimizeButton.addActionListener(this);

ImageIcon maximizeImage = new ImageIcon("Maximize.png");
Image scaledMaximizeImage =
maximizeImage.getImage().getScaledInstance(50, 50, Image.SCALE_SMOOTH);
maximizeImage = new ImageIcon(scaledMaximizeImage);

maximizeButton = new JButton();
maximizeButton.setBackground(new Color(111, 85, 12));

```

```

maximizeButton.setBorderPainted(false);
maximizeButton.setIcon(maximizeImage);
maximizeButton.setPreferredSize(new Dimension(40, 40));
maximizeButton.addActionListener(this);
topPanel.add(maximizeButton);

ImageIcon closeImage = new ImageIcon("Close.png");
Image scaledCloseImage = closeImage.getImage().getScaledInstance(26, 26,
Image.SCALE_SMOOTH);
closeImage = new ImageIcon(scaledCloseImage);

closeButton = new JButton();
closeButton.setBackground(new Color(111, 85, 12));
closeButton.setFocusable(false);
closeButton.setBorderPainted(false);
closeButton.setIcon(closeImage);
closeButton.setPreferredSize(new Dimension(40, 40));
topPanel.add(closeButton);

closeButton.addActionListener(this); \

leftPanel = new JPanel();
leftPanel.setLayout(null);
leftPanel.setBounds(0,0, 250, 650);
leftPanel.setBackground(new Color(54,21, 0));
mainFrame.add(leftPanel);

leftPanelLabel1 = new JLabel("Bank");
leftPanelLabel1.setBounds(75,100, 100, 40);
leftPanelLabel1.setFont(new Font("Arial", Font.BOLD, 40));
leftPanelLabel1.setForeground(new Color(230, 200, 136));

```

```
leftPanel.add(leftPanelLabel1);
```

```
//Using grid layout for components with equal importance.
```

```
leftPanelPanel = new JPanel();
```

```
leftPanelPanel.setLayout(new GridLayout(4,1));
```

```
detailsButton = new JButton("Customer Details");
```

```
detailsButton.setForeground(new Color(230, 200, 136));
```

```
detailsButton.setBackground(new Color(54,21, 0));
```

```
detailsButton.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
detailsButton.setFocusable(false);
```

```
detailsButton.setBorderPainted(false);
```

```
detailsButton.addActionListener(this);
```

```
leftPanelPanel.add(detailsButton);
```

```
debitCardButton = new JButton("Debit Card");
```

```
debitCardButton.setBackground(new Color(54,21, 0));
```

```
debitCardButton.setForeground(new Color(230, 200, 136));
```

```
debitCardButton.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
debitCardButton.setFocusable(false);
```

```
debitCardButton.setBorderPainted(false);
```

```
debitCardButton.addActionListener(this);
```

```
leftPanelPanel.add(debitCardButton);
```

```
creditCardButton = new JButton("Credit Card");
```

```
creditCardButton.setBackground(new Color(54,21, 0));
```

```
creditCardButton.setForeground(new Color(230, 200, 136));
```

```
creditCardButton.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
creditCardButton.setFocusable(false);
```

```
creditCardButton.setBorderPainted(false);  
creditCardButton.addActionListener(this);  
leftPanelPanel.add(creditCardButton);
```

```
settingButton = new JButton("Setting");  
settingButton.setBackground(new Color(54,21, 0));  
settingButton.setForeground(new Color(230, 200, 136));  
settingButton.setFont(new Font("Arial", Font.PLAIN, 20));  
settingButton.setFocusable(false);  
settingButton.setBorderPainted(false);  
settingButton.addActionListener(this);  
leftPanelPanel.add(settingButton);
```

```
leftPanel.add(leftPanelPanel);  
leftPanelPanel.setBounds(0,200,250,300);
```

```
rightPanel1 = new JPanel();  
rightPanel1.setLayout(null);  
rightPanel1.setBounds(250,40,650,610);  
rightPanel1.setBackground(new Color(83, 55, 16));  
rightPanel1.setVisible(true);  
mainFrame.add(rightPanel1);
```

```
rightPanel1Header = new JLabel("Customer Details", JLabel.CENTER);  
rightPanel1Header.setForeground(new Color(230, 200, 136));  
rightPanel1Header.setFont(new Font("Arial", Font.PLAIN, 30));  
rightPanel1Header.setBounds(0, 50, 650, 40);  
rightPanel1.add(rightPanel1Header);
```

```
clientNameLabel = new JLabel("Client name");
```

```

clientNameLabel.setForeground(new Color(230, 200, 136));
clientNameLabel.setFont(new Font("Arial", Font.PLAIN, 20));
clientNameLabel.setBounds(50,160, 160, 30);
rightPanel1.add(clientNameLabel);

clientNameField = new JTextField();
clientNameField.setBackground(new Color(83, 55, 16));
clientNameField.setForeground(new Color(230, 200, 136));
clientNameField.setFont(new Font("Arial", Font.PLAIN, 20));
clientNameField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new
Color(230, 200, 136)));
clientNameField.setCaretColor(new Color(230, 200, 136));
clientNameField.setBounds(210,160, 125, 30);
rightPanel1.add(clientNameField);

cardIdLabel = new JLabel("Card Id");
cardIdLabel.setForeground(new Color(230, 200, 136));
cardIdLabel.setFont(new Font("Arial", Font.PLAIN, 20));
cardIdLabel.setBounds(360,160, 120, 30);
rightPanel1.add(cardIdLabel);

cardIdField = new JTextField();
cardIdField.setBackground(new Color(83, 55, 16));
cardIdField.setForeground(new Color(230, 200, 136));
cardIdField.setFont(new Font("Arial", Font.PLAIN, 20));
cardIdField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new
Color(230, 200, 136)));
cardIdField.setCaretColor(new Color(230, 200, 136));
cardIdField.setBounds(480,160, 125, 30);
cardIdField.addKeyListener(this);
rightPanel1.add(cardIdField);

```

```

bankAccountLabel = new JLabel("Bank account");
bankAccountLabel.setForeground(new Color(230, 200, 136));
bankAccountLabel.setFont(new Font("Arial", Font.PLAIN, 20));
bankAccountLabel.setBounds(50,230, 160, 30);
rightPanel1.add(bankAccountLabel);

bankAccountField = new JTextField();
bankAccountField.setBackground(new Color(83, 55, 16));
bankAccountField.setForeground(new Color(230, 200, 136));
bankAccountField.setFont(new Font("Arial", Font.PLAIN, 20));
bankAccountField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new
Color(230, 200, 136)));
bankAccountField.setCaretColor(new Color(230, 200, 136));
bankAccountField.setBounds(210,230, 125, 30);
rightPanel1.add(bankAccountField);

issuerBankLabel = new JLabel("Issuer bank");
issuerBankLabel.setForeground(new Color(230, 200, 136));
issuerBankLabel.setFont(new Font("Arial", Font.PLAIN, 20));
issuerBankLabel.setBounds(360,230, 120, 30);
rightPanel1.add(issuerBankLabel);

issuerBankField = new JTextField();
issuerBankField.setBackground(new Color(83, 55, 16));
issuerBankField.setForeground(new Color(230, 200, 136));
issuerBankField.setFont(new Font("Arial", Font.PLAIN, 20));
issuerBankField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new
Color(230, 200, 136)));
issuerBankField.setCaretColor(new Color(230, 200, 136));
issuerBankField.setBounds(480,230, 125, 30);

```

```

rightPanel1.add(issuerBankField);

balanceAmountLabel = new JLabel("Balance amount");
balanceAmountLabel.setForeground(new Color(230, 200, 136));
balanceAmountLabel.setFont(new Font("Arial", Font.PLAIN, 20));
balanceAmountLabel.setBounds(50,300, 160, 30);
rightPanel1.add(balanceAmountLabel);

balanceAmountField = new JTextField();
balanceAmountField.setBackground(new Color(83, 55, 16));
balanceAmountField.setForeground(new Color(230, 200, 136));
balanceAmountField.setFont(new Font("Arial", Font.PLAIN, 20));
balanceAmountField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0,
new Color(230, 200, 136)));
balanceAmountField.setCaretColor(new Color(230, 200, 136));
balanceAmountField.setBounds(210,300, 125, 30);
rightPanel1.add(balanceAmountField);

rightPanel1ClearButton = new JButton("Clear");
rightPanel1ClearButton.setForeground(new Color(230, 200, 136));
rightPanel1ClearButton.setBackground(new Color(83, 55, 16));
rightPanel1ClearButton.setBorder(BorderFactory.createLineBorder(new
Color(230, 200, 136), 4));
rightPanel1ClearButton.setFont(new Font("Arial", Font.PLAIN, 20));
rightPanel1ClearButton.setBounds(50, 370, 160, 50);
rightPanel1ClearButton.setFocusable(false);
rightPanel1ClearButton.addActionListener(this);
rightPanel1.add(rightPanel1ClearButton);

currentPanel = rightPanel1;

```

```

rightPanel2 = new JPanel();
rightPanel2.setLayout(null);
rightPanel2.setBounds(250,40,650,610);
rightPanel2.setBackground(new Color(83, 55, 16));
rightPanel2.setVisible(false);
mainFrame.add(rightPanel2);

rightPanel2Header = new JLabel("Debit Card", JLabel.CENTER);
rightPanel2Header.setForeground(new Color(230, 200, 136));
rightPanel2Header.setFont(new Font("Arial", Font.PLAIN, 30));
rightPanel2Header.setBounds(0, 50, 650, 40);
rightPanel2.add(rightPanel2Header);

pinNumberLabel = new JLabel("PIN number");
pinNumberLabel.setForeground(new Color(230, 200, 136));
pinNumberLabel.setFont(new Font("Arial", Font.PLAIN, 20));
pinNumberLabel.setBounds(50,160, 160, 30);
rightPanel2.add(pinNumberLabel);

pinNumberField = new JTextField();
pinNumberField.setBackground(new Color(83, 55, 16));
pinNumberField.setForeground(new Color(230, 200, 136));
pinNumberField.setFont(new Font("Arial", Font.PLAIN, 20));
pinNumberField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new
Color(230, 200, 136)));
pinNumberField.setCaretColor(new Color(230, 200, 136));
pinNumberField.setBounds(216,160, 125, 30);
pinNumberField.addKeyListener(this);
rightPanel2.add(pinNumberField);

withdrawalAmountLabel = new JLabel("Withdrawal amount");

```

```

withdrawalAmountLabel.setForeground(new Color(230, 200, 136));
withdrawalAmountLabel.setFont(new Font("Arial", Font.PLAIN, 20));
withdrawalAmountLabel.setBounds(50,230, 180, 30);
rightPanel2.add(withdrawalAmountLabel);

withdrawalAmountField = new JTextField();
withdrawalAmountField.setBackground(new Color(83, 55, 16));
withdrawalAmountField.setForeground(new Color(230, 200, 136));
withdrawalAmountField.setFont(new Font("Arial", Font.PLAIN, 20));
withdrawalAmountField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0,
new Color(230, 200, 136)));
withdrawalAmountField.setCaretColor(new Color(230, 200, 136));
withdrawalAmountField.setBounds(235,230, 100, 30);
withdrawalAmountField.addActionListener(this);
withdrawalAmountField.addKeyListener(this);
rightPanel2.add(withdrawalAmountField);

withdrawalDateLabel = new JLabel("Withdrawal date");
withdrawalDateLabel.setForeground(new Color(230, 200, 136));
withdrawalDateLabel.setFont(new Font("Arial", Font.PLAIN, 20));
withdrawalDateLabel.setBounds(50,300, 170, 30);
rightPanel2.add(withdrawalDateLabel);

rightPanel2YearComboBox = new JComboBox<String>(yearArray);
rightPanel2YearComboBox.setBackground(new Color(160, 120, 50));
rightPanel2YearComboBox.setForeground(new Color(230, 200, 136));
rightPanel2YearComboBox.setFont(new Font("Arial", Font.PLAIN, 20));
rightPanel2YearComboBox.setFocusable(false);
rightPanel2YearComboBox.setBounds(216, 300, 90, 30);
rightPanel2YearComboBox.addActionListener(this);
rightPanel2.add(rightPanel2YearComboBox);

```

```
rightPanel2MonthComboBox = new JComboBox<String>(monthArray);
rightPanel2MonthComboBox.setBackground(new Color(160, 120, 50));
rightPanel2MonthComboBox.setForeground(new Color(230, 200, 136));
rightPanel2MonthComboBox.setFont(new Font("Arial", Font.PLAIN, 20));
rightPanel2MonthComboBox.setFocusable(false);
rightPanel2MonthComboBox.setBounds(306, 300, 50, 30);
rightPanel2MonthComboBox.addActionListener(this);
rightPanel2.add(rightPanel2MonthComboBox);
```

```
rightPanel2DayComboBox = new JComboBox<String>(dayArray);
rightPanel2DayComboBox.setBackground(new Color(160, 120, 50));
rightPanel2DayComboBox.setForeground(new Color(230, 200, 136));
rightPanel2DayComboBox.setFont(new Font("Arial", Font.PLAIN, 20));
rightPanel2DayComboBox.setFocusable(false);
rightPanel2DayComboBox.setBounds(356, 300, 50, 30);
rightPanel2DayComboBox.addActionListener(this);
rightPanel2.add(rightPanel2DayComboBox);
```

```
addDebit = new JButton("Add");
addDebit.setForeground(new Color(230, 200, 136));
addDebit.setBackground(new Color(83, 55, 16));
addDebit.setBorder(BorderFactory.createLineBorder(new Color(230, 200, 136),
4));
addDebit.setFont(new Font("Arial", Font.PLAIN, 20));
addDebit.setBounds(430, 160, 160, 50);
addDebit.setFocusable(false);
addDebit.addActionListener(this);
rightPanel2.add(addDebit);
```

```
withdraw = new JButton("Withdraw");
```

```
withdraw.setForeground(new Color(230, 200, 136));
withdraw.setBackground(new Color(83, 55, 16));
withdraw.setBorder(BorderFactory.createLineBorder(new Color(230, 200, 136),
4));
withdraw.setFont(new Font("Arial", Font.PLAIN, 20));
withdraw.setBounds(50, 370, 160, 50);
withdraw.setFocusable(false);
withdraw.addActionListener(this);
rightPanel2.add(withdraw);
```

```
displayDebit = new JButton("Display");
displayDebit.setForeground(new Color(230, 200, 136));
displayDebit.setBackground(new Color(83, 55, 16));
displayDebit.setBorder(BorderFactory.createLineBorder(new Color(230, 200,
136), 4));
displayDebit.setFont(new Font("Arial", Font.PLAIN, 20));
displayDebit.setBounds(430, 370, 160, 50);
displayDebit.setFocusable(false);
displayDebit.addActionListener(this);
rightPanel2.add(displayDebit);
```

```
clearDebit = new JButton("Clear");
clearDebit.setForeground(new Color(230, 200, 136));
clearDebit.setBackground(new Color(83, 55, 16));
clearDebit.setBorder(BorderFactory.createLineBorder(new Color(230, 200,
136), 4));
clearDebit.setFont(new Font("Arial", Font.PLAIN, 20));
clearDebit.setBounds(430, 235, 160, 50);
clearDebit.setFocusable(false);
clearDebit.addActionListener(this);
rightPanel2.add(clearDebit);
```

```

rightPanel3 = new JPanel();
rightPanel3.setLayout(null);
rightPanel3.setBounds(250,40,650,610);
rightPanel3.setBackground(new Color(83, 55, 16));
rightPanel3.setVisible(false);
mainFrame.add(rightPanel3);

rightPanel3Header = new JLabel("Credit Card", JLabel.CENTER);
rightPanel3Header.setForeground(new Color(230, 200, 136));
rightPanel3Header.setFont(new Font("Arial", Font.PLAIN, 30));
rightPanel3Header.setBounds(0, 20, 650, 40);
rightPanel3.add(rightPanel3Header);

cvcNumber = new JLabel("CVC Number");
cvcNumber.setForeground(new Color(230, 200, 136));
cvcNumber.setFont(new Font("Arial", Font.PLAIN, 20));
cvcNumber.setBounds(50,130, 160, 30);
rightPanel3.add(cvcNumber);

cvcNumberField = new JTextField();
cvcNumberField.setBackground(new Color(83, 55, 16));
cvcNumberField.setForeground(new Color(230, 200, 136));
cvcNumberField.setFont(new Font("Arial", Font.PLAIN, 20));
cvcNumberField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new
Color(230, 200, 136)));
cvcNumberField.setCaretColor(new Color(230, 200, 136));
cvcNumberField.setBounds(210,130, 125, 30);
rightPanel3.add(cvcNumberField);

interestRate = new JLabel("Interest rate");
interestRate.setForeground(new Color(230, 200, 136));

```

```

interestRate.setFont(new Font("Arial", Font.PLAIN, 20));
interestRate.setBounds(50,190, 120, 30);
rightPanel3.add(interestRate);

interestRateField = new JTextField();
interestRateField.setBackground(new Color(83, 55, 16));
interestRateField.setForeground(new Color(230, 200, 136));
interestRateField.setFont(new Font("Arial", Font.PLAIN, 20));
interestRateField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new
Color(230, 200, 136)));
interestRateField.setCaretColor(new Color(230, 200, 136));
interestRateField.setBounds(210,190, 125, 30);
rightPanel3.add(interestRateField);

expirationDate = new JLabel("Expiration Date");
expirationDate.setForeground(new Color(230, 200, 136));
expirationDate.setFont(new Font("Arial", Font.PLAIN, 20));
expirationDate.setBounds(50,250, 160, 30);
rightPanel3.add(expirationDate);

rightPanel3YearComboBox = new JComboBox<String>(yearArray);
rightPanel3YearComboBox.setBackground(new Color(160, 120, 50));
rightPanel3YearComboBox.setForeground(new Color(230, 200, 136));
rightPanel3YearComboBox.setFont(new Font("Arial", Font.PLAIN, 20));
rightPanel3YearComboBox.setFocusable(false);
rightPanel3YearComboBox.setBounds(205, 250, 90, 30);
rightPanel3YearComboBox.addActionListener(this);
rightPanel3.add(rightPanel3YearComboBox);

rightPanel3MonthComboBox = new JComboBox<String>(monthArray);
rightPanel3MonthComboBox.setBackground(new Color(160, 120, 50));

```

```

rightPanel3MonthComboBox.setForeground(new Color(230, 200, 136));
rightPanel3MonthComboBox.setFont(new Font("Arial", Font.PLAIN, 20));
rightPanel3MonthComboBox.setFocusable(false);
rightPanel3MonthComboBox.setBounds(294, 250, 50, 30);
rightPanel3MonthComboBox.addActionListener(this);
rightPanel3.add(rightPanel3MonthComboBox);

rightPanel3DayComboBox = new JComboBox<String>(dayArray);
rightPanel3DayComboBox.setBackground(new Color(160, 120, 50));
rightPanel3DayComboBox.setForeground(new Color(230, 200, 136));
rightPanel3DayComboBox.setFont(new Font("Arial", Font.PLAIN, 20));
rightPanel3DayComboBox.setFocusable(false);
rightPanel3DayComboBox.setBounds(344, 250, 50, 30);
rightPanel3DayComboBox.addActionListener(this);
rightPanel3.add(rightPanel3DayComboBox);

addCredit = new JButton("Add");
addCredit.setForeground(new Color(230, 200, 136));
addCredit.setBackground(new Color(83, 55, 16));
addCredit.setBorder(BorderFactory.createLineBorder(new Color(230, 200,
136), 4));
addCredit.setFont(new Font("Arial", Font.PLAIN, 20));
addCredit.setBounds(430, 110, 160, 50);
addCredit.setFocusable(false);
addCredit.addActionListener(this);
rightPanel3.add(addCredit);

clearCredit1 = new JButton("Clear");
clearCredit1.setForeground(new Color(230, 200, 136));
clearCredit1.setBackground(new Color(83, 55, 16));

```

```
clearCredit1.setBorder(BorderFactory.createLineBorder(new Color(230, 200, 136), 4));
```

```
clearCredit1.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
clearCredit1.setBounds(430, 175, 160, 50);
```

```
clearCredit1.setFocusable(false);
```

```
clearCredit1.addActionListener(this);
```

```
rightPanel3.add(clearCredit1);
```

```
creditLimit = new JLabel("Credit limit");
```

```
creditLimit.setForeground(new Color(230, 200, 136));
```

```
creditLimit.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
creditLimit.setBounds(50,340, 160, 30);
```

```
rightPanel3.add(creditLimit);
```

```
creditLimitField = new JTextField();
```

```
creditLimitField.setBackground(new Color(83, 55, 16));
```

```
creditLimitField.setForeground(new Color(230, 200, 136));
```

```
creditLimitField.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
creditLimitField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new Color(230, 200, 136)));
```

```
creditLimitField.setCaretColor(new Color(230, 200, 136));
```

```
creditLimitField.setBounds(210,340, 125, 30);
```

```
creditLimitField.addKeyListener(this);
```

```
rightPanel3.add(creditLimitField);
```

```
gracePeriod = new JLabel("Grace period");
```

```
gracePeriod.setForeground(new Color(230, 200, 136));
```

```
gracePeriod.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
gracePeriod.setBounds(50,400, 160, 30);
```

```
rightPanel3.add(gracePeriod);
```

```

    gracePeriodField = new JTextField();
    gracePeriodField.setBackground(new Color(83, 55, 16));
    gracePeriodField.setForeground(new Color(230, 200, 136));
    gracePeriodField.setFont(new Font("Arial", Font.PLAIN, 20));
    gracePeriodField.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, new
Color(230, 200, 136)));
    gracePeriodField.setCaretColor(new Color(230, 200, 136));
    gracePeriodField.setBounds(210, 400, 125, 30);
    rightPanel3.add(gracePeriodField);

    clearCredit2 = new JButton("Clear");
    clearCredit2.setForeground(new Color(230, 200, 136));
    clearCredit2.setBackground(new Color(83, 55, 16));
    clearCredit2.setBorder(BorderFactory.createLineBorder(new Color(230, 200,
136), 4));
    clearCredit2.setFont(new Font("Arial", Font.PLAIN, 20));
    clearCredit2.setBounds(430, 320, 160, 50);
    clearCredit2.setFocusable(false);
    clearCredit2.addActionListener(this);
    rightPanel3.add(clearCredit2);

    cancelCredit = new JButton("Cancel");
    cancelCredit.setForeground(new Color(230, 200, 136));
    cancelCredit.setBackground(new Color(83, 55, 16));
    cancelCredit.setBorder(BorderFactory.createLineBorder(new Color(230, 200,
136), 4));
    cancelCredit.setFont(new Font("Arial", Font.PLAIN, 20));
    cancelCredit.setBounds(430, 385, 160, 50);
    cancelCredit.setFocusable(false);
    cancelCredit.addActionListener(this);
    rightPanel3.add(cancelCredit);

```

```
setCreditLimit = new JButton("Set Limit");
setCreditLimit.setForeground(new Color(230, 200, 136));
setCreditLimit.setBackground(new Color(83, 55, 16));
setCreditLimit.setBorder(BorderFactory.createLineBorder(new Color(230, 200,
136), 4));
setCreditLimit.setFont(new Font("Arial", Font.PLAIN, 20));
setCreditLimit.setBounds(50, 450, 160, 50);
setCreditLimit.setFocusable(false);
setCreditLimit.addActionListener(this);
rightPanel3.add(setCreditLimit);
```

```
displayCredit = new JButton("Display");
displayCredit.setForeground(new Color(230, 200, 136));
displayCredit.setBackground(new Color(83, 55, 16));
displayCredit.setBorder(BorderFactory.createLineBorder(new Color(230, 200,
136), 4));
displayCredit.setFont(new Font("Arial", Font.PLAIN, 20));
displayCredit.setBounds(430, 450, 160, 50);
displayCredit.setFocusable(false);
displayCredit.addActionListener(this);
rightPanel3.add(displayCredit);
```

```
rightPanel4 = new JPanel();
rightPanel4.setLayout(null);
rightPanel4.setBounds(250, 40, 650, 610);
rightPanel4.setBackground(new Color(83, 55, 16));
rightPanel4.setVisible(false);
mainFrame.add(rightPanel4);
```

```
rightPanel4Header = new JLabel("Setting", JLabel.CENTER);
```

```
rightPanel4Header.setForeground(new Color(230, 200, 136));
rightPanel4Header.setFont(new Font("Arial", Font.PLAIN, 30));
rightPanel4Header.setBounds(0, 50, 650, 40);
rightPanel4.add(rightPanel4Header);
```

```
theme = new JLabel("Theme");
theme.setForeground(new Color(230, 200, 136));
theme.setFont(new Font("Arial", Font.PLAIN, 20));
theme.setBounds(50, 160, 160, 30);
rightPanel4.add(theme);
```

```
darkMode = new JRadioButton("Dark Mode");
darkMode.addActionListener(this);
darkMode.setSelected(true);
darkMode.setBounds(180,160, 130, 30);
darkMode.setBackground(new Color(83, 55, 16));
darkMode.setFont(new Font("Arial", Font.PLAIN, 20));
darkMode.setForeground(new Color(230, 200, 136));
darkMode.setFocusable(false);
```

```
lightMode = new JRadioButton("Light Mode");
lightMode.addActionListener(this);
lightMode.setBounds(320, 160, 160, 30);
lightMode.setBackground(new Color(83, 55, 16));
lightMode.setFont(new Font("Arial", Font.PLAIN, 20));
lightMode.setForeground(new Color(230, 200, 136));
lightMode.setFocusable(false);
```

```
buttonGroup = new ButtonGroup();
buttonGroup.add(darkMode);
```

```

        buttonGroup.add(lightMode);

        rightPanel4.add(darkMode);
        rightPanel4.add(lightMode);

        mainFrame.setResizable(true);
        mainFrame.setUndecorated(true);
        mainFrame.setLayout(null);
        mainFrame.setSize(900, 650);
        mainFrame.setVisible(false);
        mainFrame.setDefaultCloseOperation(mainFrame.EXIT_ON_CLOSE);
        mainFrame.setLocationRelativeTo(null);
    }

```

@Override

```

public void actionPerformed(ActionEvent e){

```

```

    double interestRate;
    double creditLimit;

```

```

    int cvcNumber;
    int balanceAmount;
    int cardId;
    int pinNumber;
    int withdrawalAmount;
    int gracePeriod;

```

```

    boolean catchBalanceAmount = false;
    boolean catchCardId = false;

```

boolean catchPinNumber = false;
boolean catchCvcNumber = false;
boolean catchInterestRate = false;

boolean emptyBalanceAmount = false;
boolean emptyCardId = false;
boolean emptyPinNumber = false;
boolean emptyBankAccount = false;
boolean emptyIssuerBank = false;
boolean emptyClientName = false;
boolean emptyCvcNumber = false;
boolean emptyInterestRate = false;
boolean emptyWithdrawalAmount = false;
boolean emptyCreditLimit = false;
boolean emptyGracePeriod = false;

boolean isEmptyForDebit = false;
boolean isEmptyForCredit = false;
boolean isEmptyForWithdrawal = false;
boolean isEmptyForSetLimit = false;

boolean incorrectValueForDebit = false;
boolean incorrectValueForCredit = false;
boolean incorrectValueForWithdrawal = false;
boolean incorrectValueForSetLimit = false;

boolean catchBankAccount = false;
boolean catchIssuerBank = false;
boolean catchClientName = false;
boolean catchWithdrawalAmount = false;

```
boolean catchCreditLimit = false;  
boolean catchGracePeriod = false;
```

```
boolean exception = false;  
boolean isMatch = false;
```

```
if(e.getSource() == addCredit || e.getSource() == addDebit){
```

```
    try{  
        balanceAmount = Integer.parseInt(balanceAmountField.getText());  
    }  
    catch(NumberFormatException exception1){  
        catchBalanceAmount = true;  
        if (balanceAmountField.getText().equals("")){  
            emptyBalanceAmount = true;  
        }  
    }  
}
```

```
    try{  
        cardId = Integer.parseInt(cardIdField.getText());  
    }  
    catch(NumberFormatException exception1){  
        catchCardId = false;  
        if(cardIdField.getText().equals("")){  
            emptyCardId = true;  
        }  
    }  
}
```

```
    try{  
        pinNumber = Integer.parseInt(pinNumberField.getText());
```

```

    }

    catch(NumberFormatException exception1){
        catchPinNumber = true;
        if(pinNumberField.getText().equals("")){
            emptyPinNumber = true;
        }
    }

    try{
        cvcNumber = Integer.parseInt(cvcNumberField.getText());
    }
    catch(NumberFormatException exception1){
        catchCvcNumber = true;
        if(cvcNumberField.getText().equals("")){
            emptyCvcNumber = true;
        }
    }

    try{
        interestRate = Double.parseDouble(interestRateField.getText());
    }
    catch(NumberFormatException exception1){
        catchInterestRate = true;
        if(interestRateField.getText().equals("")){
            emptyInterestRate = true;
        }
    }

    try{
        withdrawalAmount = Integer.parseInt(withdrawalAmountField.getText());
    }

```

```

    }

    catch(NumberFormatException exception1){
        catchWithdrawalAmount = true;
        if(withdrawalAmountField.getText().equals("")){
            emptyWithdrawalAmount = true;
        }
    }

    try{
        creditLimit = Double.parseDouble(creditLimitField.getText());
    }
    catch(NumberFormatException exception1){
        catchCreditLimit = true;
        if(withdrawalAmountField.getText().equals("")){
            emptyCreditLimit = true;
        }
    }

    try{
        gracePeriod = Integer.parseInt(gracePeriodField.getText());
    }
    catch(NumberFormatException exception1){
        catchGracePeriod = true;
        if(withdrawalAmountField.getText().equals("")){
            emptyGracePeriod = true;
        }
    }

```

```

if(bankAccountField.getText().equals("")){
    emptyBankAccount = true;
}

if(issuerBankField.getText().equals("")){
    emptyIssuerBank = true;
}

if(clientNameField.getText().equals("")){
    emptyClientName = true;
}

if(emptyBankAccount || emptyIssuerBank || emptyClientName ||
emptyBalanceAmount || emptyCardId || emptyPinNumber){
    isEmptyForDebit = true;
}

if(catchBalanceAmount || catchCardId || catchPinNumber){
    incorrectValueForDebit = true;
}

if(emptyBankAccount || emptyIssuerBank || emptyClientName ||
emptyBalanceAmount || emptyCardId || emptyCvcNumber || emptyInterestRate){
    isEmptyForCredit = true;
}

if(catchBalanceAmount || catchCardId || catchCvcNumber ||
catchInterestRate){
    incorrectValueForCredit = true;
}
}

```

```
i++;

if(i == 10){
    loadingFrameLabel1.setText("Loading..");
}
else if(i == 20){
    loadingFrameLabel1.setText("Loading...");
}
else if(i == 30){
    loadingFrameLabel1.setText("Loading.");
}
else if(i == 40){
    loadingFrameLabel1.setText("Loading..");
}
else if(i == 50){
    loadingFrameLabel1.setText("Loading...");
}
else if(i == 55){
    loadingFrameLabel1.setText("Loading.");
}
else if(i == 70){
    loadingFrameLabel1.setText("Loading..");
}
else if(i == 80){
    loadingFrameLabel1.setText("Loading...");
}
else if(i == 90){
    loadingFrameLabel1.setText("Loading.");
}
```

```

else if(i == 100){
    loadingFrameLabel1.setText("Loading..");
    timer1.stop();
    loadingFrame.dispose();
    mainFrame.setVisible(true);
}
progressBar.setValue(i);

if(e.getSource() == closeButton){
    mainFrame.dispose();
}
else if(e.getSource() == maximizeButton){

}
else if(e.getSource() == minimizeButton){
    mainFrame.setState(JFrame.ICONIFIED);
}
else if(e.getSource() == detailsButton){

    currentPanel.setVisible(false);
    currentPanel = rightPanel1;
    rightPanel1.setVisible(true);
}
else if(e.getSource() == debitCardButton){
    currentPanel.setVisible(false);
    currentPanel = rightPanel2;
    rightPanel2.setVisible(true);
}
else if(e.getSource() == creditCardButton){
    currentPanel.setVisible(false);

```

```

        currentPanel = rightPanel3;
        rightPanel3.setVisible(true);
    }
    else if(e.getSource() == settingButton){
        currentPanel.setVisible(false);
        currentPanel = rightPanel4;
        rightPanel4.setVisible(true);
    }
    else if(e.getSource() == addDebit){
        try{

            if(emptyBankAccount || emptyIssuerBank || emptyClientName ||
            emptyBalanceAmount || emptyCardId || emptyPinNumber || catchBalanceAmount ||
            catchCardId || catchPinNumber){

                throw new NumberFormatException();
            }

            String bankAccount = bankAccountField.getText(); // Extracting the needed
            values from the text fields.

            String issuerBank = issuerBankField.getText();
            String clientName = clientNameField.getText();
            balanceAmount = Integer.parseInt(balanceAmountField.getText());
            cardId = Integer.parseInt(cardIdField.getText());
            pinNumber = Integer.parseInt(pinNumberField.getText());

            Iterator itr = arrayList.iterator();

            if(arrayList.isEmpty()){

                DebitCard obj = new DebitCard(balanceAmount, cardId, bankAccount,
                issuerBank, clientName, pinNumber);

```

```

        arrayList.add(obj);

        JOptionPane.showMessageDialog(mainFrame, "The debit card has
        been added successfully.", "Inforamtion", JOptionPane.INFORMATION_MESSAGE);
    }
    else{
        while(itr.hasNext()){

            BankCard bankCard = (BankCard)itr.next();

            if(bankCard instanceof DebitCard){
                if(bankCard.getCardId() == cardId){

                    isMatch = true;

                    JOptionPane.showMessageDialog(mainFrame, "You cannot add
                    the same debit card twice.", "Error", JOptionPane.ERROR_MESSAGE);

                    break;
                }
            }
        }
        if (isMatch == false){
            DebitCard obj = new DebitCard(balanceAmount, cardId,
            bankAccount, issuerBank, clientName, pinNumber);

            arrayList.add(obj);

            JOptionPane.showMessageDialog(mainFrame, "The debit card has
            been added successfully.", "Inforamtion", JOptionPane.INFORMATION_MESSAGE);
        }
    }
}

catch(NumberFormatException exception1){

    if(incorrectValueForDebit && isEmptyForDebit)

```

```
String messageForIncorrectValue = ((catchBalanceAmount) ? "balance amount," : "") + ((catchCardId) ? "card id," : "") + ((catchPinNumber) ? "pin number" : "");
```

```
String messageAfterSplit[] = messageForIncorrectValue.split(",");
```

```
if(messageAfterSplit.length == 1){
```

```
    messageForIncorrectValue = "You have entered wrong values for " + messageAfterSplit[0] + ".";
```

```
}
```

```
else if(messageAfterSplit.length == 2){
```

```
    messageForIncorrectValue = "You have entered wrong values for " + messageAfterSplit[0] + " and " + messageAfterSplit[1] + "."; ;
```

```
}
```

```
else if(messageAfterSplit.length == 3){
```

```
    messageForIncorrectValue = "You have entered wrong values for " + messageAfterSplit[0] + ", " + messageAfterSplit[1] + " and " + messageAfterSplit[2] + ".";
```

```
}
```

```
String messageForEmptyValue = ((emptyBankAccount) ? "bank account," : "") + ((emptyIssuerBank) ? "issuer bank," : "") + ((emptyClientName) ? "client name," : "") + ((emptyBalanceAmount) ? "balance amount," : "") + ((emptyCardId) ? "card id," : "") + ((emptyPinNumber) ? "pin number" : "");
```

```
String messageAfterSplit1[] = messageForEmptyValue.split(",");
```

```
if(messageAfterSplit1.length == 1){
```

```
    messageForEmptyValue = "You have not entered values for " + messageAfterSplit1[0] + ".";
```

```
}
```

```
else if(messageAfterSplit1.length == 2){
```

```
    messageForEmptyValue = "You have not entered values for " + messageAfterSplit1[0] + " and " + messageAfterSplit1[1] + "."; ;
```

```
}
```

```
else if(messageAfterSplit1.length == 3){
```

```

        messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + " and " +
messageAfterSplit1[2] + ".";
    }

    else if(messageAfterSplit1.length == 4){

        messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + ", " + messageAfterSplit1[2] +
" and " + messageAfterSplit1[3] + ".";
    }

    else if(messageAfterSplit1.length == 5){

        messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + ", " + messageAfterSplit1[2] +
", " + messageAfterSplit1[3] + " and " + messageAfterSplit1[4] + ".";
    }

    else if(messageAfterSplit1.length == 6){

        messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + ", " + messageAfterSplit1[2] +
", " + messageAfterSplit1[3] + ", " + messageAfterSplit1[4] + " and " +
messageAfterSplit1[5] + ".";
    }


    JOptionPane.showMessageDialog(mainFrame,
messageForIncorrectValue + "\n" + messageForEmptyValue, "Error" ,
JOptionPane.ERROR_MESSAGE);
}

else if(!incorrectValueForDebit && isEmptyForDebit){

    String messageForEmptyValue = ((emptyBankAccount) ? "bank
account," : "") + ((emptyIssuerBank) ? "issuer bank," : "") + ((emptyClientName) ?
"client name," : "") + ((emptyBalanceAmount) ? "balance amount," : "") +
((emptyCardId) ? "card id," : "") + ((emptyPinNumber) ? "pin number" : "");


    String messageAfterSplit1[] = messageForEmptyValue.split(",");

    if(messageAfterSplit1.length == 1){

        messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ".";
    }

```

```

    }

    else if(messageAfterSplit1.length == 2){

        messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + " and " + messageAfterSplit1[1] + "."; ;

    }

    else if(messageAfterSplit1.length == 3){

        messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + " and " +
messageAfterSplit1[2] + ".";

    }


    JOptionPane.showMessageDialog(mainFrame,
messageForEmptyValue, "Error" , JOptionPane.ERROR_MESSAGE);

}

else if(incorrectValueForDebit && !isEmptyForDebit){

    String messageForIncorrectValue = ((catchBalanceAmount) ? "balance
amount," : "") + ((catchCardId) ? "card id," : "") + ((catchPinNumber) ? "pin number" :
    "");

    String messageAfterSplit[] = messageForIncorrectValue.split(",");

    if(messageAfterSplit.length == 1){

        messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + ".";

    }

    else if(messageAfterSplit.length == 2){

        messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + " and " + messageAfterSplit[1] + "."; ;

    }

    else if(messageAfterSplit.length == 3){

        messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + ", " + messageAfterSplit[1] + " and " + messageAfterSplit[2] +
        ".";

    }
}

```

```

        JOptionPane.showMessageDialog(mainFrame,
messageForIncorrectValue, "Error" , JOptionPane.ERROR_MESSAGE);
    }
}
}

```

```

else if(e.getSource() == addCredit){

```

```

    try{

```

```

        //Throw an exception if any of the values are empty or incorrect or both.

```

```

        if(emptyBankAccount || emptyIssuerBank || emptyClientName ||
emptyBalanceAmount || emptyCardId || emptyCvcNumber || emptyInterestRate ||
catchBalanceAmount || catchCardId || catchCvcNumber || catchInterestRate){

```

```

            throw new NumberFormatException();

```

```

        }

```

```

        String bankAccount = bankAccountField.getText();

```

```

        String issuerBank = issuerBankField.getText();

```

```

        String clientName = clientNameField.getText();

```

```

        rightPanel3Year =
rightPanel3YearComboBox.getSelectedItem().toString();

```

```

        rightPanel3Month =
rightPanel3MonthComboBox.getSelectedItem().toString();

```

```

        rightPanel3Day = rightPanel3DayComboBox.getSelectedItem().toString();

```

```

        String expirationDate = rightPanel3Year + "-" + rightPanel3Month + "-" +
rightPanel3Day;

```

```

        balanceAmount = Integer.parseInt(balanceAmountField.getText());

```

```

        cardId = Integer.parseInt(cardIdField.getText());

```

```

        cvcNumber = Integer.parseInt(cvcNumberField.getText());

```

```

interestRate = Double.parseDouble(interestRateField.getText());

Iterator itr = arrayList.iterator();

if(arrayList.isEmpty()){
    CreditCard obj = new CreditCard(cardId, clientName, issuerBank,
bankAccount, balanceAmount, cvcNumber, interestRate, expirationDate);
    arrayList.add(obj);

    JOptionPane.showMessageDialog(mainFrame, "The credit card has
been added successfully.", "Inforamtion", JOptionPane.INFORMATION_MESSAGE);
}
else{
    while(itr.hasNext()){
        BankCard bankCard = (BankCard)itr.next();
        if(bankCard instanceof CreditCard){
            if(bankCard.getCardId() == cardId){
                isMatch = true;

                JOptionPane.showMessageDialog(mainFrame, "You cannot add
the same credit card twice.", "Error", JOptionPane.ERROR_MESSAGE);

                break;
            }
        }
    }

    if (isMatch == false){
        CreditCard obj = new CreditCard(cardId, clientName, issuerBank,
bankAccount, balanceAmount, cvcNumber, interestRate, expirationDate);
        arrayList.add(obj);

        JOptionPane.showMessageDialog(mainFrame, "The credit card has
been added successfully.", "Inforamtion", JOptionPane.INFORMATION_MESSAGE);
    }
}

```

```

    }
}

}

catch(NumberFormatException exception1){
    if(incorrectValueForCredit && isEmptyForCredit){

        String messageForIncorrectValue = ((catchBalanceAmount) ? "balance
amount," : "") + ((catchCardId) ? "card id," : "") + ((catchCvcNumber) ? "cvc number,"
: "") + ((catchInterestRate) ? "interest rate" : "");

        String messageAfterSplit[] = messageForIncorrectValue.split(",");

        if(messageAfterSplit.length == 1){
            messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + ".";
        }
        else if(messageAfterSplit.length == 2){
            messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + " and " + messageAfterSplit[1] + "."; ;
        }
        else if(messageAfterSplit.length == 3){
            messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + ", " + messageAfterSplit[1] + " and " + messageAfterSplit[2] +
".";
        }
        else if(messageAfterSplit.length == 4){
            messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + ", " + messageAfterSplit[1] + ", " + messageAfterSplit[2] + "
and " + messageAfterSplit[3] + ".";
        }

        String messageForEmptyValue = ((emptyBankAccount) ? "bank
account," : "") + ((emptyIssuerBank) ? "issuer bank," : "") + ((emptyClientName) ?
"client name," : "") + ((emptyBalanceAmount) ? "balance amount," : "") +

```

```

((emptyCardId) ? "card id," : "") + ((emptyCvcNumber) ? "cvc number," : "") +
((emptyInterestRate) ? "interest rate" : "");

String messageAfterSplit1[] = messageForEmptyValue.split(",");

if(messageAfterSplit1.length == 1){
    messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ".";
}
else if(messageAfterSplit1.length == 2){
    messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + " and " + messageAfterSplit1[1] + "."; ;
}
else if(messageAfterSplit1.length == 3){
    messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + " and " +
messageAfterSplit1[2] + ".";
}
else if(messageAfterSplit1.length == 4){
    messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + ", " + messageAfterSplit1[2] +
" and " + messageAfterSplit1[3] + ".";
}
else if(messageAfterSplit1.length == 5){
    messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + ", " + messageAfterSplit1[2] +
", " + messageAfterSplit1[3] + " and " + messageAfterSplit1[4] + ".";
}
else if(messageAfterSplit1.length == 6){
    messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + ", " + messageAfterSplit1[2] +
", " + messageAfterSplit1[3] + ", " + messageAfterSplit1[4] + " and " +
messageAfterSplit1[5] + ".";
}
else if(messageAfterSplit1.length == 7){

```

```

        messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + ", " + messageAfterSplit1[2] +
", " + messageAfterSplit1[3] + ", " + messageAfterSplit1[4] + " and " +
messageAfterSplit1[5] + ", " + messageAfterSplit1[6] + ".";

```

```

    }

```

```

        JOptionPane.showMessageDialog(mainFrame,
messageForIncorrectValue + "\n" + messageForEmptyValue, "Error" ,
JOptionPane.ERROR_MESSAGE);

```

```

    }

```

```

    else if(!incorrectValueForCredit && isEmptyForCredit){

```

```

        String messageForEmptyValue = ((emptyBankAccount) ? "bank
account," : "") + ((emptyIssuerBank) ? "issuer bank," : "") + ((emptyClientName) ?
"client name," : "") + ((emptyBalanceAmount) ? "balance amount," : "") +
((emptyCardId) ? "card id," : "") + ((emptyCvcNumber) ? "cvc number," : "") +
((emptyInterestRate) ? "interest rate" : "");

```

```

        String messageAfterSplit1[] = messageForEmptyValue.split(",");

```

```

        if(messageAfterSplit1.length == 1){

```

```

            messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ".";

```

```

        }

```

```

        else if(messageAfterSplit1.length == 2){

```

```

            messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + " and " + messageAfterSplit1[1] + "."; ;

```

```

        }

```

```

        else if(messageAfterSplit1.length == 3){

```

```

            messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + " and " +
messageAfterSplit1[2] + ".";

```

```

        }

```

```

        else if(messageAfterSplit1.length == 4){

```

```

            messageForEmptyValue = "You have not entered values for " +
messageAfterSplit1[0] + ", " + messageAfterSplit1[1] + ", " + messageAfterSplit1[2] +
" and " + messageAfterSplit1[3] + ".";

```

```

    }

    JOptionPane.showMessageDialog(mainFrame,
messageForEmptyValue, "Error" , JOptionPane.ERROR_MESSAGE);

    }

    else if(incorrectValueForCredit && !isEmptyForCredit){

        String messageForIncorrectValue = ((catchBalanceAmount) ? "balance
amount," : "") + ((catchCardId) ? "card id," : "") + ((catchCvcNumber) ? "cvc number,"
: "") + ((catchInterestRate) ? "interest rate" : "");

        String messageAfterSplit[] = messageForIncorrectValue.split(",");

        if(messageAfterSplit.length == 1){

            messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + ".";

        }

        else if(messageAfterSplit.length == 2){

            messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + " and " + messageAfterSplit[1] + "."; ;

        }

        else if(messageAfterSplit.length == 3){

            messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + ", " + messageAfterSplit[1] + " and " + messageAfterSplit[2] +
".";

        }

        else if(messageAfterSplit.length == 4){

            messageForIncorrectValue = "You have entered wrong values for " +
messageAfterSplit[0] + ", " + messageAfterSplit[1] + ", " + messageAfterSplit[2] + "
and " + messageAfterSplit[3] + ".";

        }

        JOptionPane.showMessageDialog(mainFrame,
messageForIncorrectValue, "Error" , JOptionPane.ERROR_MESSAGE);

    }

```

```

    }
}

else if (e.getSource() == withdraw){
    try{
        cardId = Integer.parseInt(cardIdField.getText());
        pinNumber = Integer.parseInt(pinNumberField.getText());
        withdrawalAmount = Integer.parseInt(withdrawalAmountField.getText());

        boolean isMatchCardId = false;
        boolean isMatchPinNumber = false;

        rightPanel2Year =
rightPanel2YearComboBox.getSelectedItem().toString();

        rightPanel2Month =
rightPanel2MonthComboBox.getSelectedItem().toString();

        rightPanel2Day = rightPanel2DayComboBox.getSelectedItem().toString();

        String dateOfWithdrawal = rightPanel2Year + "-" + rightPanel2Month + "-"
+ rightPanel2Day;

        if(arrayList.isEmpty()){
            JOptionPane.showMessageDialog(mainFrame, "There are no entries in
the database. Please add an object of debit card before withdrawing.", "Error",
JOptionPane.ERROR_MESSAGE);
        }
        else{
            Iterator itr = arrayList.iterator();
            while(itr.hasNext()){
                BankCard bankCard = (BankCard)itr.next();
                if(bankCard instanceof DebitCard){
                    DebitCard debitCard = (DebitCard)bankCard;

```

```

        if(debitCard.getCardId() == cardId || debitCard.getPinNumber() ==
pinNumber){
            if(debitCard.getCardId() == cardId && debitCard.getPinNumber()
!= pinNumber){

                isMatchCardId = true;
                isMatchPinNumber = false;
                break;
            }
            else if(debitCard.getCardId() != cardId){
                isMatchCardId = false;
            }
            else if(debitCard.getCardId() == cardId &&
debitCard.getPinNumber() == pinNumber){
                debitCard.withdraw(withdrawalAmount, dateOfWithdrawal,
pinNumber);

                isMatchCardId = true;
                isMatchPinNumber = true;

                if(debitCard.getHasWithdrawn() == true){
                    JOptionPane.showMessageDialog(mainFrame, "The
withdrawal was successful." + "\n" + "\n" + "The current balance amount is " +
debitCard.getBalanceAmount() + "." + "\n" + "The date of withdrawal is " +
debitCard.getDateOfWithdrawal() + ".", "Information",
JOptionPane.INFORMATION_MESSAGE);
                }
                else if(debitCard.getHasWithdrawn() == false){
                    JOptionPane.showMessageDialog(mainFrame, "The
withdrawal was not successful." + "\n" + "\n" + "Please enter a valid amount.",
"Error", JOptionPane.ERROR_MESSAGE);
                }
                break;
            }
        }
    }
}

```

```

        }

    }

    if(isMatchCardId == false){
        JOptionPane.showMessageDialog(mainFrame, "A valid card Id was
not entered.", "Error", JOptionPane.ERROR_MESSAGE);
    }

    else if(isMatchCardId == true && isMatchPinNumber == false){
        JOptionPane.showMessageDialog(mainFrame, "The pin number of
the entered card Id was not valid.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

}

}

catch(NumberFormatException exception1){

    JOptionPane.showMessageDialog(mainFrame, "You have not entered a
correct value. Please enter a correct value.", "Error",
JOptionPane.ERROR_MESSAGE);

}

}

else if (e.getSource() == setCreditLimit){
    try{
        cardId = Integer.parseInt(cardIdField.getText());
        gracePeriod = Integer.parseInt(gracePeriodField.getText());
        creditLimit = Double.parseDouble(creditLimitField.getText());
        boolean isMatchCredit = false;

        if(arrayList.isEmpty()){

            JOptionPane.showMessageDialog(mainFrame, "There are no entries in
the database. Please add an object of credit card before setting its limit.", "Error",
JOptionPane.ERROR_MESSAGE);

```

```

    }
    else{
        Iterator itr = arrayList.iterator();
        while(itr.hasNext()){
            BankCard bankCard = (BankCard)itr.next();
            if(bankCard instanceof CreditCard){
                CreditCard creditCard = (CreditCard)bankCard;
                if(creditCard.getCardId() != cardId){
                    isMatchCredit = false;
                }
                else if(creditCard.getCardId() == cardId){
                    creditCard.setCreditLimit(creditLimit, gracePeriod);
                    isMatchCredit = true;
                    if(creditCard.getIsGranted() == true){
                        JOptionPane.showMessageDialog(mainFrame, "The credit
limit has been set successfully." + "\n" + "\n" + "Your new credit limit is " +
creditCard.getCreditLimit() + "." + "\n" + "Your new grace period is " +
creditCard.getGracePeriod() + ".", "Information",
JOptionPane.INFORMATION_MESSAGE);
                    }
                    else if(creditCard.getIsGranted() == false){
                        JOptionPane.showMessageDialog(mainFrame, "The credit
limit could not be set." + "\n" + "\n" + "Please enter a valid credit limit. ", "Error",
JOptionPane.ERROR_MESSAGE);
                    }
                    break;
                }
            }
        }
        if(isMatchCredit == false){
            JOptionPane.showMessageDialog(mainFrame, "There is no entry of
credit card of entered card id in the database.", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

```

        }
    }
}

catch(NumberFormatException exception1){
    JOptionPane.showMessageDialog(mainFrame, "You have not entered a
correct value. Please enter a correct value.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}

else if(e.getSource() == cancelCredit){
    try{
        cardId = Integer.parseInt(cardIdField.getText());
        Iterator itr = arrayList.iterator();
        boolean isMatchCredit = false;
        boolean isGranted = false;

        if(arrayList.isEmpty()){
            JOptionPane.showMessageDialog(mainFrame, "There are no entries in
the database. Please add an object of credit card before cancelling its limit.", "Error",
JOptionPane.ERROR_MESSAGE);
        }
        else{
            while(itr.hasNext()){
                BankCard bankCard = (BankCard)itr.next();
                if(bankCard instanceof CreditCard){
                    CreditCard creditCard = (CreditCard) bankCard; //Downcasting to
credit card type to access cancelCreditCard() method.
                    if(creditCard.getCardId() != cardId){
                        isMatchCredit = false;
                    }
                    else if(creditCard.getCardId() == cardId){
                        isMatchCredit = true;

```

```

        if(creditCard.getIsGranted() == true){
            creditCard.cancelCreditCard();
            isGranted = true;

            JOptionPane.showMessageDialog(mainFrame, "The credit limit
of the entered card id has been successfully cancelled." + "\n" + "\n" + "The cvc
number is " + creditCard.getCvcNumber() + "." + "\n" + "The credit limit is " +
creditCard.getCreditLimit() + "." + "\n" + "The grace period is " +
creditCard.getGracePeriod() + ".", "Information",
JOptionPane.INFORMATION_MESSAGE);

            break;
        }
        else{
            isGranted = false;
        }
    }
}

if(isMatchCredit == false){
    JOptionPane.showMessageDialog(mainFrame, "There is no entry of
credit card of the entered card id in the database.", "Information",
JOptionPane.INFORMATION_MESSAGE);
}
else{
    if(isGranted == false){
        JOptionPane.showMessageDialog(mainFrame, "Credit limit has not
been granted or it has been cancelled before." + "\n" + "Hence, the system is unable
to cancel the credit card." , "Error", JOptionPane.ERROR_MESSAGE);
    }
}
}
}

catch(NumberFormatException exception1){

```

```

        JOptionPane.showMessageDialog(mainFrame, "You have not entered a
correct value. Please enter a correct value.", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}
else if (e.getSource() == displayDebit){

    try{
        cardId = Integer.parseInt(cardIdField.getText());
        isMatch = false;
        Iterator itr = arrayList.iterator();

        if(arrayList.isEmpty()){
            JOptionPane.showMessageDialog(mainFrame, "There are no entries in
the database." + "\n" + "Please add a debit card to the database before viewing the
details.", "Error", JOptionPane.ERROR_MESSAGE);
        }
        else{
            while(itr.hasNext()){
                BankCard bankCard = (BankCard)itr.next();
                if(bankCard instanceof DebitCard){
                    DebitCard debitCard = (DebitCard)bankCard; //Downcasting to
access display() method of debit card.
                    if(debitCard.getCardId() == cardId){
                        debitCard.display();
                        isMatch = true;

                        JOptionPane.showMessageDialog(mainFrame, "The details of
the debit card were displayed successfully.", "Information",
JOptionPane.INFORMATION_MESSAGE);
                    }
                    else{

```

```

        isMatch = false;
    }
}
}
if(isMatch == false){
    JOptionPane.showMessageDialog(mainFrame, "There is no debit
card of the entered card Id. Please enter a correct card id.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}
}
catch(NumberFormatException exception1){
    JOptionPane.showMessageDialog(mainFrame, "You have not entered a
correct value. Please enter a correct value.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}
else if(e.getSource() == displayCredit){
    try{
        cardId = Integer.parseInt(cardIdField.getText());
        isMatch = false;
        Iterator itr = arrayList.iterator();

        if(arrayList.isEmpty()){
            JOptionPane.showMessageDialog(mainFrame, "There are no entries in
the database." + "\n" + "Please add a credit card to the database before viewing the
details.", "Error", JOptionPane.ERROR_MESSAGE);
        }
        else{
            while(itr.hasNext()){
                BankCard bankCard = (BankCard)itr.next();
                if(bankCard instanceof CreditCard){

```

```
        CreditCard creditCard = (CreditCard)bankCard; //Upcasting to  
access display() method of credit card.
```

```
        if(creditCard.getCardId() == cardId){  
            creditCard.display();  
            isMatch = true;
```

```
            JOptionPane.showMessageDialog(mainFrame, "The details of  
the credit card were displayed successfully.", "Error",  
JOptionPane.ERROR_MESSAGE);
```

```
        }  
        else{  
            isMatch = false;  
        }  
    }  
}
```

```
    if(isMatch == false){
```

```
        JOptionPane.showMessageDialog(mainFrame, "There is no credit  
card of the entered card Id. Please enter a correct card id.", "Error",  
JOptionPane.ERROR_MESSAGE);
```

```
    }
```

```
}
```

```
}
```

```
catch(NumberFormatException exception1){
```

```
    JOptionPane.showMessageDialog(mainFrame, "You have not entered a  
correct value. Please enter a correct value.", "Error",  
JOptionPane.ERROR_MESSAGE);
```

```
}
```

```
}
```

```
else if(e.getSource() == rightPanel1ClearButton){
```

```
    clientNameField.setText(""); // Setting the text field to an empty string clears  
out the field.
```

```
    cardIdField.setText("");
```

```
    bankAccountField.setText("");
```

```

        issuerBankField.setText("");
        balanceAmountField.setText("");
    }
    else if(e.getSource() == clearDebit){
        pinNumberField.setText("");
        withdrawalAmountField.setText("");
    }
    else if(e.getSource() == clearCredit1){
        cvcNumberField.setText("");
        interestRateField.setText("");
    }
    else if(e.getSource() == clearCredit2){
        creditLimitField.setText("");
        gracePeriodField.setText("");
    }
    else if(e.getSource() == withdrawalAmountField){
        try{
            rightPanel2Year =
rightPanel2YearComboBox.getSelectedItem().toString();

            rightPanel2Month =
rightPanel2MonthComboBox.getSelectedItem().toString();

            rightPanel2Day = rightPanel2DayComboBox.getSelectedItem().toString();

            String dateOfWithdrawal = rightPanel2Year + "-" + rightPanel2Month + "-"
+ rightPanel2Day;

```

```

withdrawalAmount = Integer.parseInt(withdrawalAmountField.getText());
cardId = Integer.parseInt(cardIdField.getText());
Iterator itr = arrayList.iterator();

if(arrayList.isEmpty()){
}
else{
    while(itr.hasNext()){
        BankCard bankCard = (BankCard)itr.next();
        if(bankCard instanceof DebitCard && bankCard.getCardId() ==
cardId){
            if(bankCard.getBalanceAmount() >= withdrawalAmount){
                validWithdrawalAmount = true;

                JOptionPane.showMessageDialog(mainFrame, "A valid
withdrawal amount was entered.", "Information",
JOptionPane.INFORMATION_MESSAGE);

                break;
            }
            else{
                validWithdrawalAmount = false;
            }
        }
    }
}

if (validCardIdDebit && validPinNumber && validWithdrawalAmount){
    JOptionPane.showMessageDialog(mainFrame, "The values entered
were correct." + "\n" + "\n" + "The card Id is " + cardIdField.getText() + "." + "\n" +
"The withdrawal amount is " + withdrawalAmountField.getText() + "." + "\n" + "The
date of withdrawal is " + dateOfWithdrawal + "." + "\n" + "The pin Number is " +

```

```

pinNumberField.getText() + ".", "Information",
JOptionPane.INFORMATION_MESSAGE);
    }
}
catch(NumberFormatException exception1){
    JOptionPane.showMessageDialog(mainFrame, "A valid withdrawal
amount was not entered.", "Error", JOptionPane.ERROR_MESSAGE);
}
}
}

```

```

if(e.getSource() == darkMode){
    Color darkModeColorLeftPanel = new Color(54, 21, 0);
    Color darkModeColorRightPanel = new Color(83, 55, 16);
    Color darkModeColorComboBox = new Color(160, 120, 50);

    leftPanel.setBackground(darkModeColorLeftPanel);
    detailsButton.setBackground(darkModeColorLeftPanel);
    debitCardButton.setBackground(darkModeColorLeftPanel);
    creditCardButton.setBackground(darkModeColorLeftPanel);
    settingButton.setBackground(darkModeColorLeftPanel);

    rightPanel1.setBackground(darkModeColorRightPanel);
    clientNameField.setBackground(darkModeColorRightPanel);
    cardIdField.setBackground(darkModeColorRightPanel);
    bankAccountField.setBackground(darkModeColorRightPanel);
    issuerBankField.setBackground(darkModeColorRightPanel);
    balanceAmountField.setBackground(darkModeColorRightPanel);
    rightPanel1ClearButton.setBackground(darkModeColorRightPanel);

    rightPanel2.setBackground(darkModeColorRightPanel);
}
}
}

```

```
pinNumberField.setBackground(darkModeColorRightPanel);
withdrawalAmountField.setBackground(darkModeColorRightPanel);
rightPanel2YearComboBox.setBackground(darkModeColorComboBox);
rightPanel2MonthComboBox.setBackground(darkModeColorComboBox);
rightPanel2DayComboBox.setBackground(darkModeColorComboBox);
```

```
addDebit.setBackground(darkModeColorRightPanel);
clearDebit.setBackground(darkModeColorRightPanel);
withdraw.setBackground(darkModeColorRightPanel);
displayDebit.setBackground(darkModeColorRightPanel);
```

```
rightPanel3.setBackground(darkModeColorRightPanel);
cvcNumberField.setBackground(darkModeColorRightPanel);
interestRateField.setBackground(darkModeColorRightPanel);
rightPanel3YearComboBox.setBackground(darkModeColorComboBox);
rightPanel3MonthComboBox.setBackground(darkModeColorComboBox);
rightPanel3DayComboBox.setBackground(darkModeColorComboBox);
```

```
creditLimitField.setBackground(darkModeColorRightPanel);
gracePeriodField.setBackground(darkModeColorRightPanel);
```

```
addCredit.setBackground(darkModeColorRightPanel);
clearCredit1.setBackground(darkModeColorRightPanel);
clearCredit2.setBackground(darkModeColorRightPanel);
cancelCredit.setBackground(darkModeColorRightPanel);
displayCredit.setBackground(darkModeColorRightPanel);
setCreditLimit.setBackground(darkModeColorRightPanel);
```

```
rightPanel4.setBackground(darkModeColorRightPanel);
darkMode.setBackground(darkModeColorRightPanel);
```

```

        lightMode.setBackground(darkModeColorRightPanel);
    }
    else if(e.getSource() == lightMode){
        Color lightModeColorLeftPanel = new Color(118, 160, 30);
        Color lightModeColorRightPanel = new Color(165, 85, 64);
        Color lightModeColorComboBox = new Color(255, 109, 70);

        leftPanel.setBackground(lightModeColorLeftPanel);
        detailsButton.setBackground(lightModeColorLeftPanel);
        debitCardButton.setBackground(lightModeColorLeftPanel);
        creditCardButton.setBackground(lightModeColorLeftPanel);
        settingButton.setBackground(lightModeColorLeftPanel);

        rightPanel1.setBackground(lightModeColorRightPanel);
        clientNameField.setBackground(lightModeColorRightPanel);
        cardIdField.setBackground(lightModeColorRightPanel);
        bankAccountField.setBackground(lightModeColorRightPanel);
        issuerBankField.setBackground(lightModeColorRightPanel);
        balanceAmountField.setBackground(lightModeColorRightPanel);
        rightPanel1ClearButton.setBackground(lightModeColorRightPanel);

        rightPanel2.setBackground(lightModeColorRightPanel);
        pinNumberField.setBackground(lightModeColorRightPanel);
        withdrawalAmountField.setBackground(lightModeColorRightPanel);
        rightPanel2YearComboBox.setBackground(lightModeColorComboBox);
        rightPanel2MonthComboBox.setBackground(lightModeColorComboBox);
        rightPanel2DayComboBox.setBackground(lightModeColorComboBox);

        addDebit.setBackground(lightModeColorRightPanel);
        clearDebit.setBackground(lightModeColorRightPanel);
    }
}

```

```
withdraw.setBackground(lightModeColorRightPanel);  
displayDebit.setBackground(lightModeColorRightPanel);
```

```
rightPanel3.setBackground(lightModeColorRightPanel);  
cvcNumberField.setBackground(lightModeColorRightPanel);  
interestRateField.setBackground(lightModeColorRightPanel);  
rightPanel3YearComboBox.setBackground(lightModeColorComboBox);  
rightPanel3MonthComboBox.setBackground(lightModeColorComboBox);  
rightPanel3DayComboBox.setBackground(lightModeColorComboBox);
```

```
creditLimitField.setBackground(lightModeColorRightPanel);  
gracePeriodField.setBackground(lightModeColorRightPanel);
```

```
addCredit.setBackground(lightModeColorRightPanel);  
clearCredit1.setBackground(lightModeColorRightPanel);  
clearCredit2.setBackground(lightModeColorRightPanel);  
cancelCredit.setBackground(lightModeColorRightPanel);  
displayCredit.setBackground(lightModeColorRightPanel);  
setCreditLimit.setBackground(lightModeColorRightPanel);
```

```
rightPanel4.setBackground(lightModeColorRightPanel);  
darkMode.setBackground(lightModeColorRightPanel);  
lightMode.setBackground(lightModeColorRightPanel);
```

```
}
```

```
}
```

```
@Override
```

```
public void keyPressed(KeyEvent e){  
}
```

@Override

```
public void keyTyped(KeyEvent e){
```

```
}
```

@Override

```
public void keyReleased(KeyEvent e){
```

```
    if (e.getSource() == cardIdField){
```

```
        try{
```

```
            rightPanel2Year =  
rightPanel2YearComboBox.getSelectedItem().toString();
```

```
            rightPanel2Month =  
rightPanel2MonthComboBox.getSelectedItem().toString();
```

```
            rightPanel2Day = rightPanel2DayComboBox.getSelectedItem().toString();
```

```
            String dateOfWithdrawal = rightPanel2Year + "-" + rightPanel2Month + "-"  
+ rightPanel2Day;
```

```
            int cardId = Integer.parseInt(cardIdField.getText());
```

```
            Iterator itr = arrayList.iterator();
```

```
            Iterator itr1 = arrayList.iterator();
```

```
            if(arrayList.isEmpty()){
```

```
            }
```

```
            else{
```

```
                while(itr.hasNext()){
```

```
                    BankCard bankCard = (BankCard)itr.next();
```

```
                    if(bankCard instanceof DebitCard){
```

```
                        if(bankCard.getCardId() == cardId){
```

```
                            validCardIdDebit = true;
```

```
                            JOptionPane.showMessageDialog(mainFrame, "A valid card Id  
for debit card was entered.", "Information",  
JOptionPane.INFORMATION_MESSAGE);
```

```

        break;
    }
    else{
        validCardIdDebit = false;
    }
}
}
}

```

```

    if (validCardIdDebit && validPinNumber && validWithdrawalAmount){
        JOptionPane.showMessageDialog(mainFrame, "The values entered
were correct." + "\n" + "\n" + "The card Id is " + cardIdField.getText() + "." + "\n" +
"The withdrawal amount is " + withdrawalAmountField.getText() + "." + "\n" + "The
date of withdrawal is " + dateOfWithdrawal + "." + "\n" + "The pin Number is " +
pinNumberField.getText() + ".", "Information",
JOptionPane.INFORMATION_MESSAGE);
    }

```

```

while(itr1.hasNext()){
    BankCard bankCard = (BankCard)itr1.next();
    if (bankCard instanceof CreditCard){
        if(bankCard.getCardId() == cardId){
            validCardIdCredit = true;

            JOptionPane.showMessageDialog(mainFrame, "A valid card Id
for credit card was entered.", "Information",
JOptionPane.INFORMATION_MESSAGE);

            break;
        }
        else{
            validCardIdCredit = false;
        }
    }
}
}

```

```

        if(validCardIdCredit){
            double creditLimit = Double.parseDouble(creditLimitField.getText());
            int gracePeriod = Integer.parseInt(gracePeriodField.getText());

            JOptionPane.showMessageDialog(mainFrame, "The values entered
are -" + "\n" + "\n" + "The credit limit is " + creditLimit + "." + "\n" + "The grace period
is " + gracePeriod + ".");
        }

    }
}
catch(NumberFormatException exception){
}
}

else if(e.getSource() == withdrawalAmountField){
    try{
        rightPanel2Year =
rightPanel2YearComboBox.getSelectedItem().toString();

        rightPanel2Month =
rightPanel2MonthComboBox.getSelectedItem().toString();

        rightPanel2Day = rightPanel2DayComboBox.getSelectedItem().toString();

        String dateOfWithdrawal = rightPanel2Year + "-" + rightPanel2Month + "-"
+ rightPanel2Day;

        int withdrawalAmount =
Integer.parseInt(withdrawalAmountField.getText());

        int cardId = Integer.parseInt(cardIdField.getText());

        Iterator itr = arrayList.iterator();

```

```

        if(arrayList.isEmpty()){
        }
        else{
            while(itr.hasNext()){
                BankCard bankCard = (BankCard)itr.next();
                if(bankCard instanceof DebitCard && bankCard.getCardId() ==
cardId){
                    if(bankCard.getBalanceAmount() >= withdrawalAmount){
                        validWithdrawalAmount = true;
                        break;
                    }
                    else{
                        validWithdrawalAmount = false;
                    }
                }
            }
        }
    }
    catch(NumberFormatException exception){
    }
}

else if(e.getSource() == pinNumberField){
    try{

        rightPanel2Year =
rightPanel2YearComboBox.getSelectedItem().toString();

        rightPanel2Month =
rightPanel2MonthComboBox.getSelectedItem().toString();

        rightPanel2Day = rightPanel2DayComboBox.getSelectedItem().toString();

        String dateOfWithdrawal = rightPanel2Year + "-" + rightPanel2Month + "-"
+ rightPanel2Day;

```

```

int cardId = Integer.parseInt(cardIdField.getText());
int pinNumber = Integer.parseInt(pinNumberField.getText());
Iterator itr = arrayList.iterator();

if(arrayList.isEmpty()){
}
else{
    while(itr.hasNext()){
        BankCard bankCard = (BankCard)itr.next();

        if(bankCard instanceof DebitCard){
            if(bankCard.getCardId() == cardId){
                DebitCard debitCard = (DebitCard)bankCard;
                if(debitCard.getPinNumber() == pinNumber){
                    validPinNumber = true;

                    JOptionPane.showMessageDialog(mainFrame, "A valid pin
number was entered.", "Information", JOptionPane.INFORMATION_MESSAGE);

                    break;
                }
            }
            else{
                validPinNumber = false;
            }
        }
    }
}

if (validCardIdDebit && validPinNumber && validWithdrawalAmount){
    JOptionPane.showMessageDialog(mainFrame, "The values entered
were correct." + "\n" + "\n" + "The card Id is " + cardIdField.getText() + "." + "\n" +
"The withdrawal amount is " + withdrawalAmountField.getText() + "." + "\n" + "The
date of withdrawal is " + dateOfWithdrawal + "." + "\n" + "The pin Number is " +

```

```
pinNumberField.getText() + ".", "Information",
JOptionPane.INFORMATION_MESSAGE);
    }
}
catch(NumberFormatException exception){
}
}
}

public static void main(String[] args) {
    new BankGUI();
}

}
```